

Pontifícia Universidade Católica de Campinas

Estruturas e Recuperação de Dados II



GRAFOS



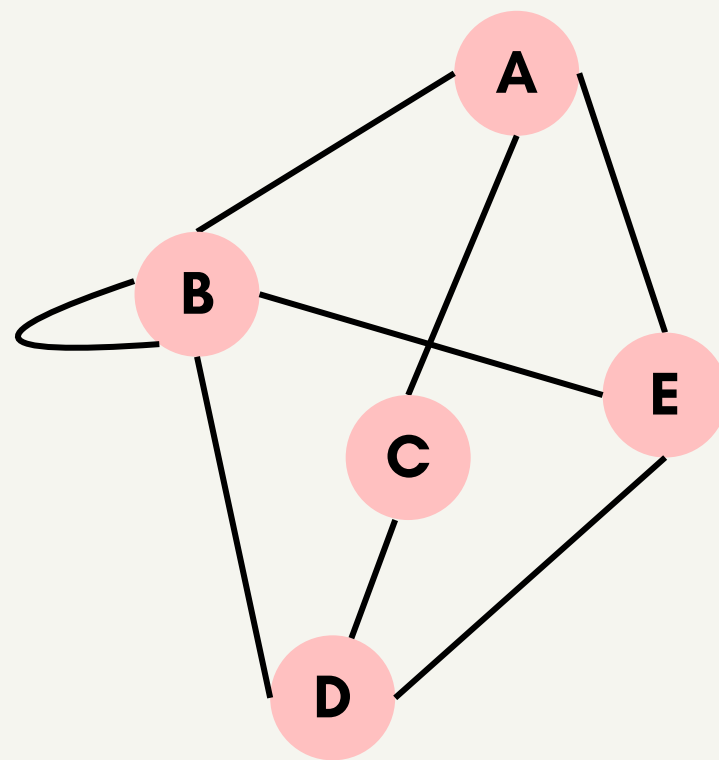
Prof. Me. Darcy H.F.R. Moreira



Grafos Não-Direcionados

GRAU DE UM VÉRTICE $d(v)$: Quantidade de arestas incidentes a ele.

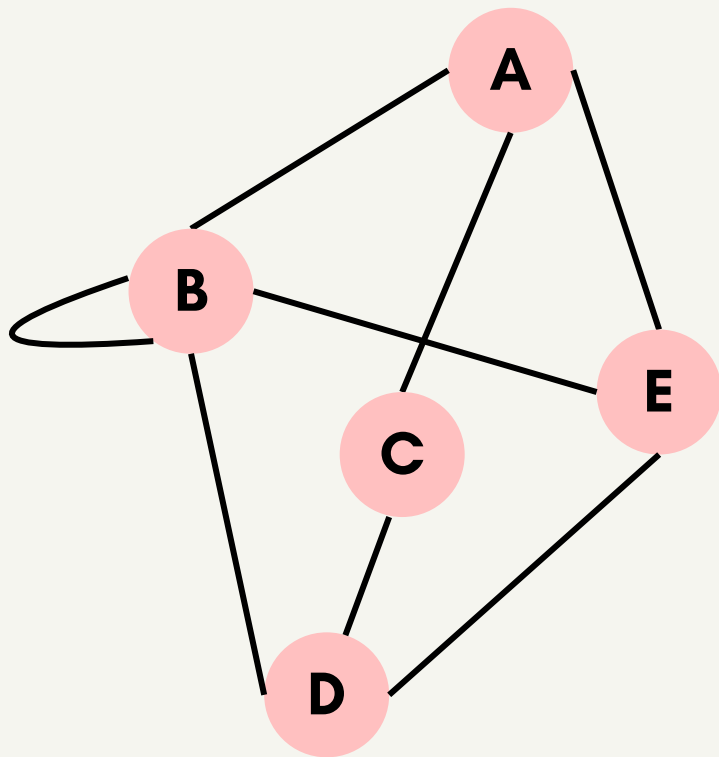
OBS: O LAÇO CONTA 2 VEZES!!



Grafos Não-Direcionados

GRAU DE UM VÉRTICE $d(v)$: Quantidade de arestas incidentes a ele.

OBS: O LAÇO CONTA 2 VEZES!!



TEOREMA DE EULER: A SOMA DOS GRAUS DOS VÉRTICES É DUAS VEZES O NÚMERO DE ARESTAS.

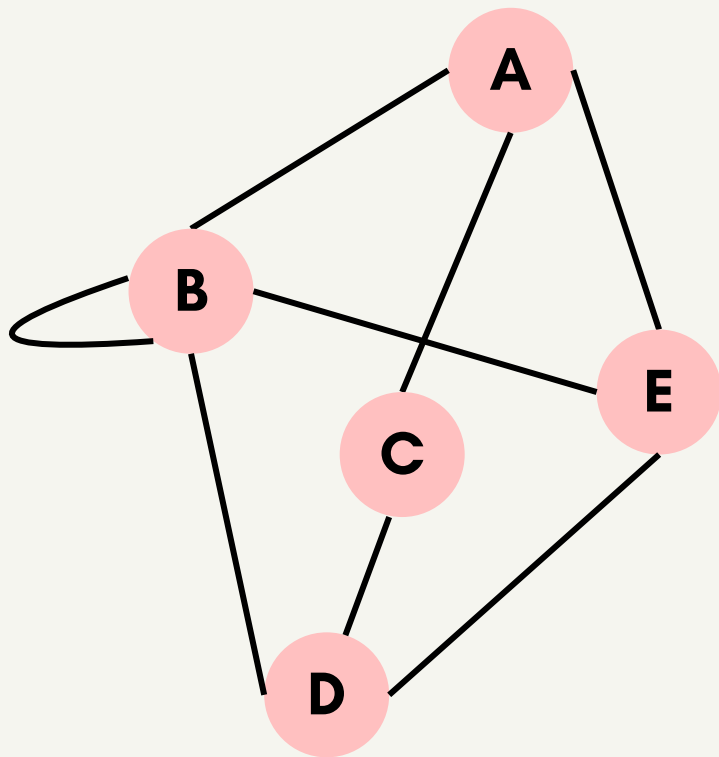
$$2|A| = \underbrace{\sum_{v_i \in V} d(v_i)}_{\text{SEMPRE PAR}}$$

POSSUI UMA QUANTIDADE PAR DE VÉRTICES DE GRAU ÍMPAR.

Grafos Não-Direcionados

GRAU DE UM VÉRTICE $d(v)$: Quantidade de arestas incidentes a ele.

OBS: O LAÇO CONTA 2 VEZES!!



$|A| = 8$

TEOREMA DE EULER: A SOMA DOS GRAUS DOS VÉRTICES É DUAS VEZES O NÚMERO DE ARESTAS.

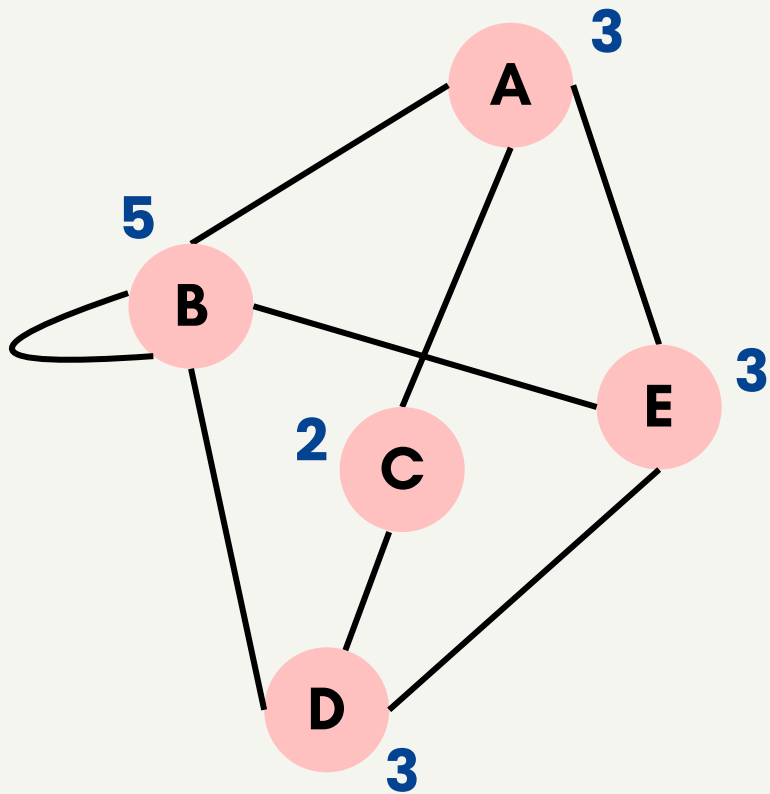
$$2|A| = \underbrace{\sum_{v_i \in V} d(v_i)}_{\text{SEMPRE PAR}}$$

POSSUI UMA QUANTIDADE PAR DE VÉRTICES DE GRAU ÍMPAR.

Grafos Não-Direcionados

GRAU DE UM VÉRTICE $d(v)$: Quantidade de arestas incidentes a ele.

OBS: O LAÇO CONTA 2 VEZES!!



$|A| = 8$

$2|A| = \sum_{v_i \in V} d(v_i) = 16$

TEOREMA DE EULER: A SOMA DOS GRAUS DOS VÉRTICES É DUAS VEZES O NÚMERO DE ARESTAS.

$$2|A| = \underbrace{\sum_{v_i \in V} d(v_i)}_{\text{SEMPRE PAR}}$$

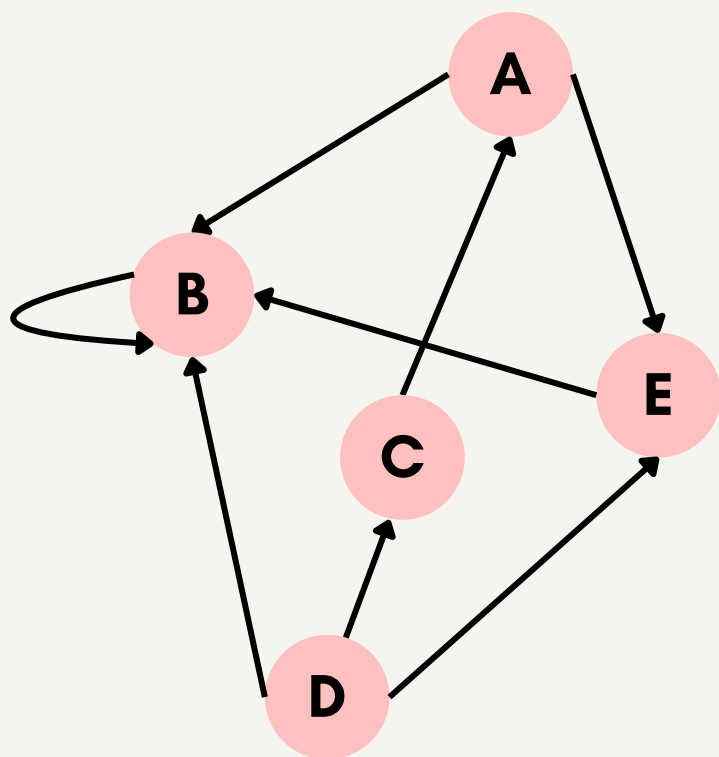
POSSUI UMA QUANTIDADE PAR DE VÉRTICES DE GRAU ÍMPAR.

Grafos direcionados (Dígrafos)

Grau de entrada (din): Quantidade de arestas que chegam ao vértice.

Grau de saída (dout): Quantidade de arestas que saem do vértice.

$d(v) = \text{din}(v) + \text{dout}(v)$

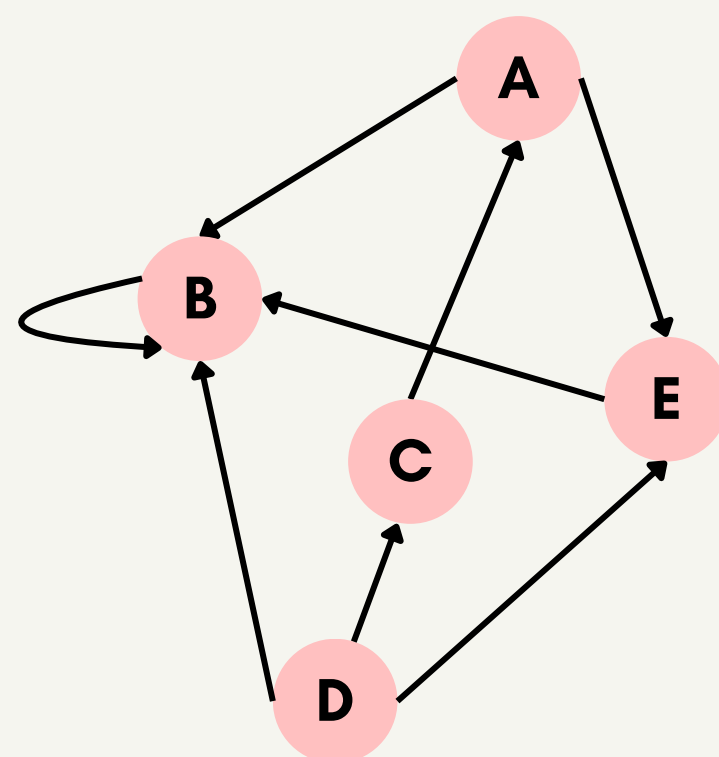


Grafos direcionados (Dígrafos)

Grau de entrada (din): Quantidade de arestas que chegam ao vértice.

Grau de saída (dout): Quantidade de arestas que saem do vértice.

$$d(v) = \text{din}(v) + \text{dout}(v)$$



A SOMA DOS GRAUS DE SAÍDA DOS VÉRTICES É IGUAL À SOMA DOS GRAUS DE ENTRADA DOS VÉRTICES, QUE EQUIVALE AO NÚMERO DE ARESTAS DO GRAFO.

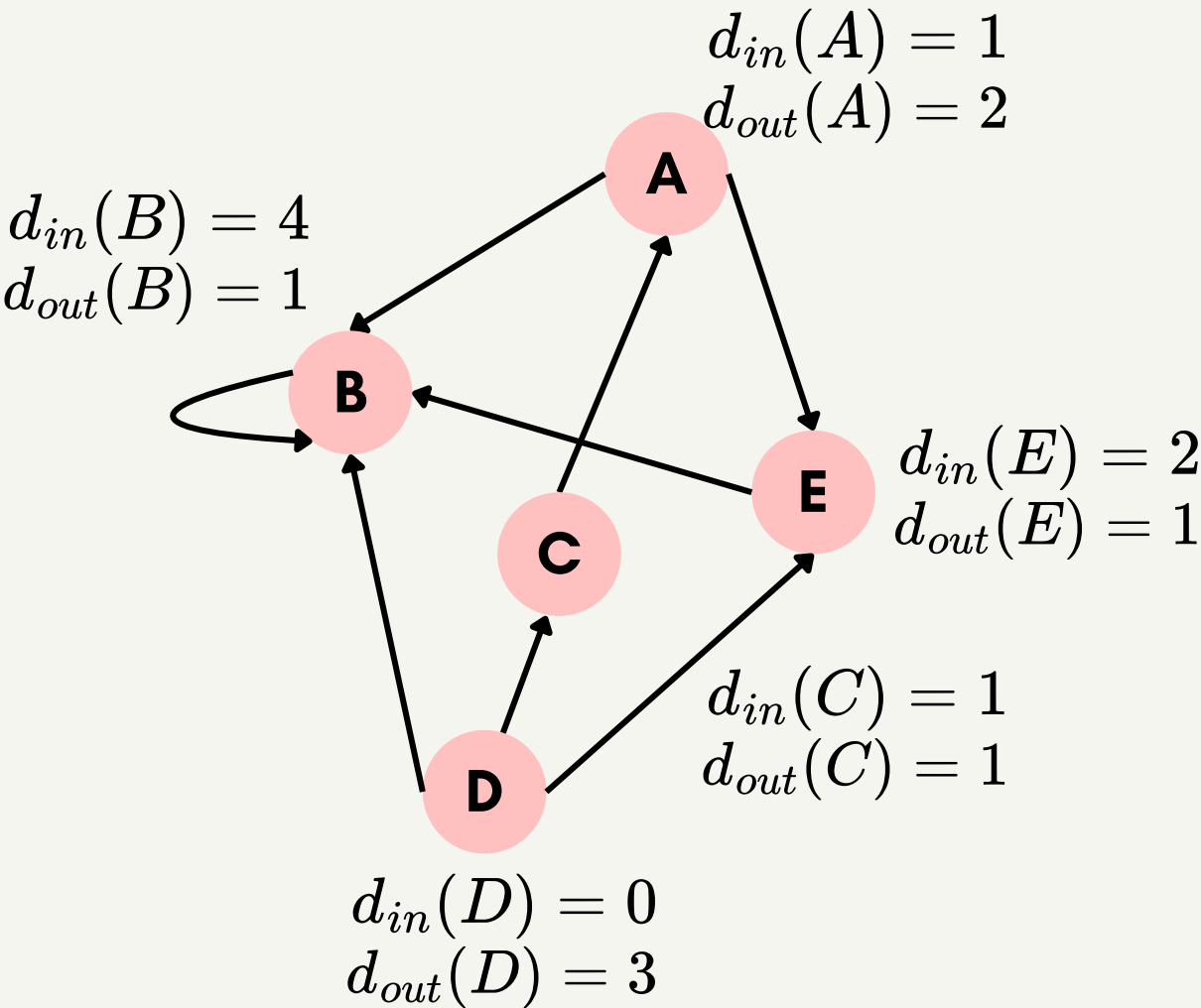
$$|A| = \sum_{i=1}^n d_{in}(v_i) = \sum_{i=1}^n d_{out}(v_i)$$

Grafos direcionados (Dígrafos)

Grau de entrada (din): Quantidade de arestas que chegam ao vértice.

Grau de saída (dout): Quantidade de arestas que saem do vértice.

$d(v) = \text{din}(v) + \text{dout}(v)$



A SOMA DOS GRAUS DE SAÍDA DOS VÉRTICES É IGUAL À SOMA DOS GRAUS DE ENTRADA DOS VÉRTICES, QUE EQUIVALE AO NÚMERO DE ARESTAS DO GRAFO.

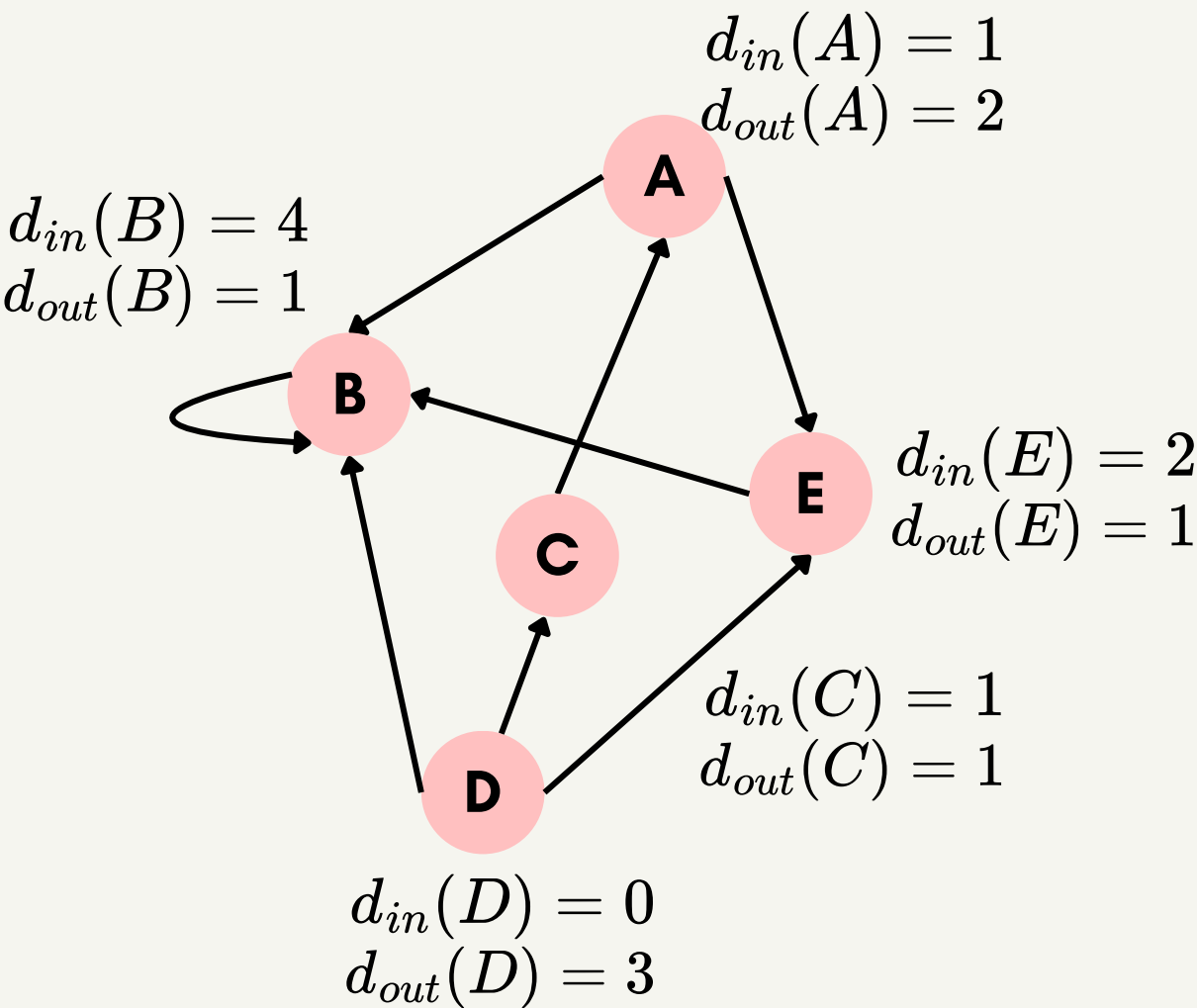
$$|A| = \sum_{i=1}^n d_{in}(v_i) = \sum_{i=1}^n d_{out}(v_i)$$

Grafos direcionados (Dígrafos)

Grau de entrada (din): Quantidade de arestas que chegam ao vértice.

Grau de saída (dout): Quantidade de arestas que saem do vértice.

$$d(v) = \text{din}(v) + \text{dout}(v)$$



A SOMA DOS GRAUS DE SAÍDA DOS VÉRTICES É IGUAL À SOMA DOS GRAUS DE ENTRADA DOS VÉRTICES, QUE EQUIVALE AO NÚMERO DE ARESTAS DO GRAFO.

$$|A| = \sum_{i=1}^n d_{in}(v_i) = \sum_{i=1}^n d_{out}(v_i)$$

$$|A| = \sum_{i=1}^n d_{in}(v_i) = \sum_{i=1}^n d_{out}(v_i) = 8$$

Grafos Completos (não direcionado)

Para cada par de vértices existe uma aresta ligando-os.

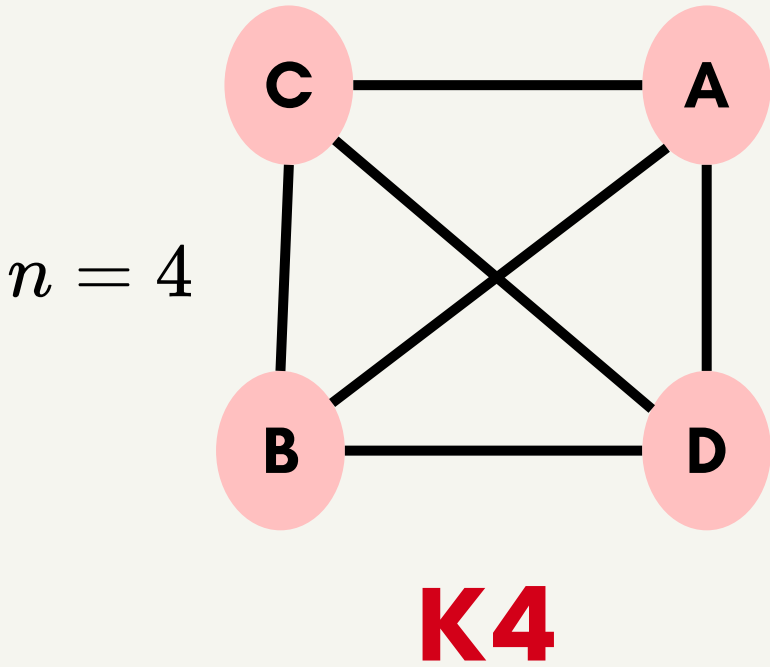
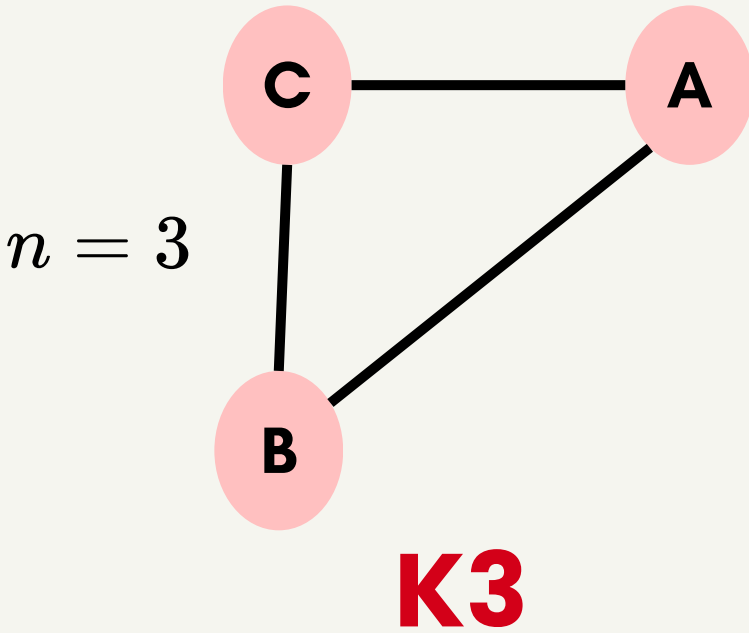
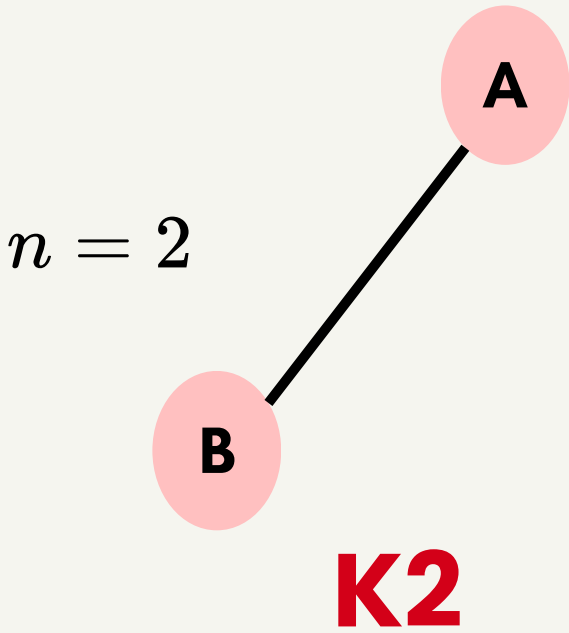
Grafos Completos (não direcionado)

Para cada par de vértices existe uma aresta ligando-os.

K_n

$n \rightarrow$ número de vértices

$n-1 \rightarrow$ grau de todos os vértices

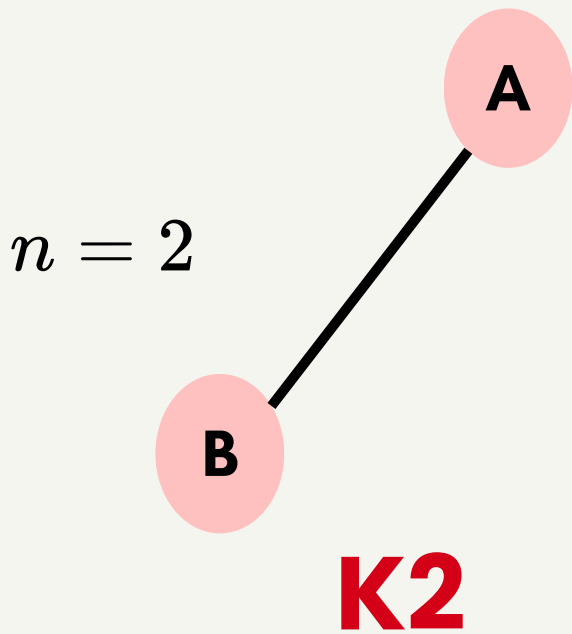


Grafos Completos (não direcionado)

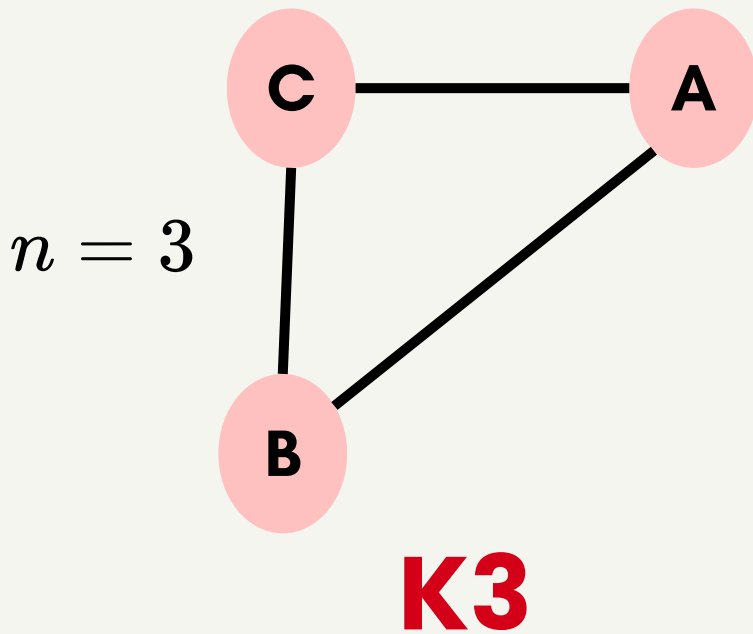
Para cada par de vértices existe uma aresta ligando-os.

K_n

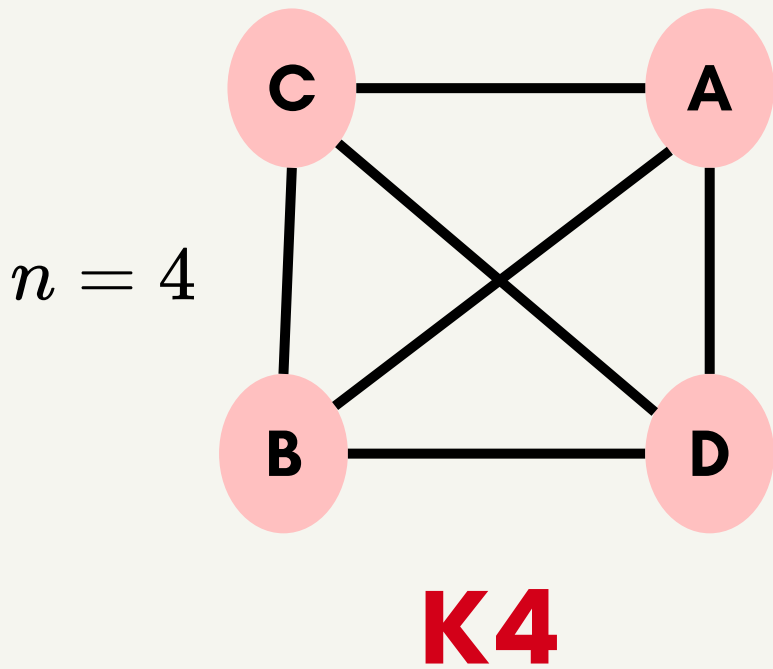
$n \rightarrow$ número de vértices
 $n-1 \rightarrow$ grau de todos os vértices



$d(v) = 2 - 1 = 1$



$d(v) = 3 - 1 = 2$



$d(v) = 4 - 1 = 3$

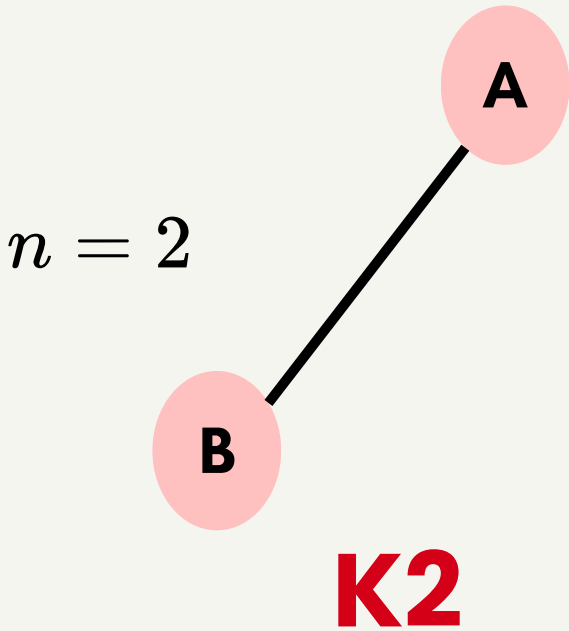
Grafos Completos (não direcionado)

Para cada par de vértices existe uma aresta ligando-os.

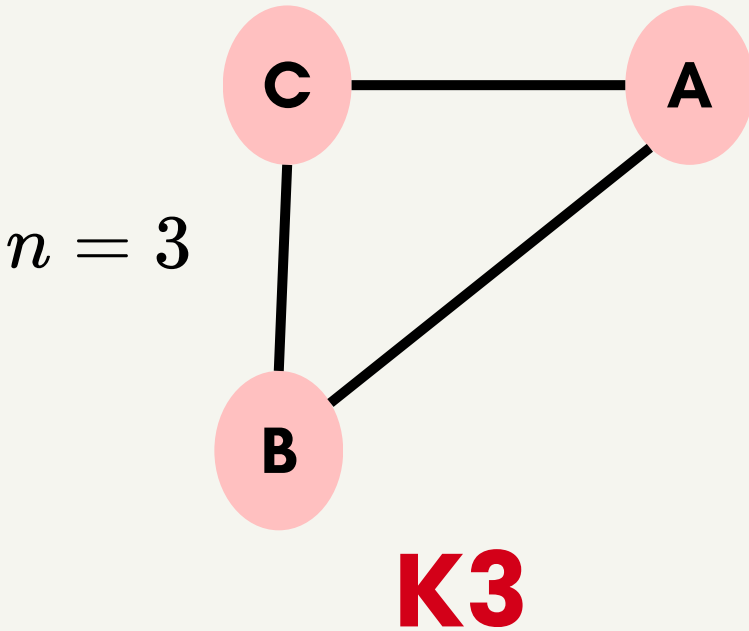
K_n

$n \rightarrow$ número de vértices
 $n-1 \rightarrow$ grau de todos os vértices

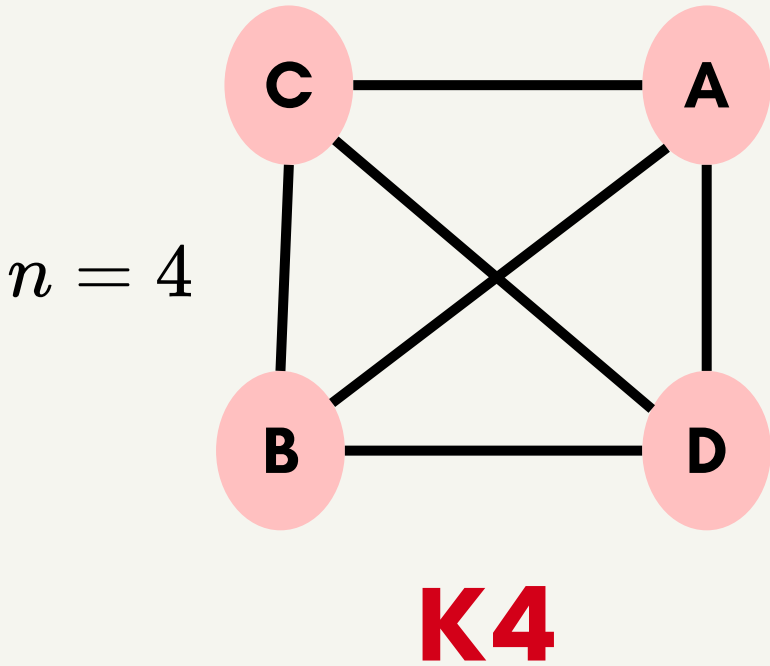
$|A| = \frac{n(n - 1)}{2}$



$d(v) = 2 - 1 = 1$



$d(v) = 3 - 1 = 2$



$d(v) = 4 - 1 = 3$

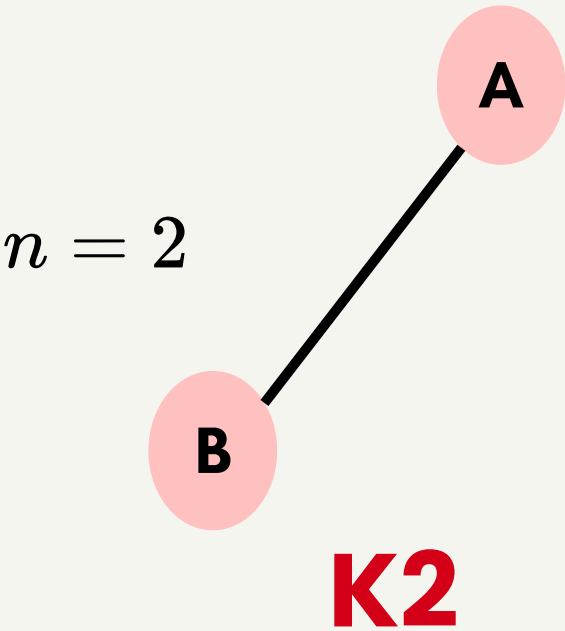
Grafos Completos (não direcionado)

Para cada par de vértices existe uma aresta ligando-os.

K_n

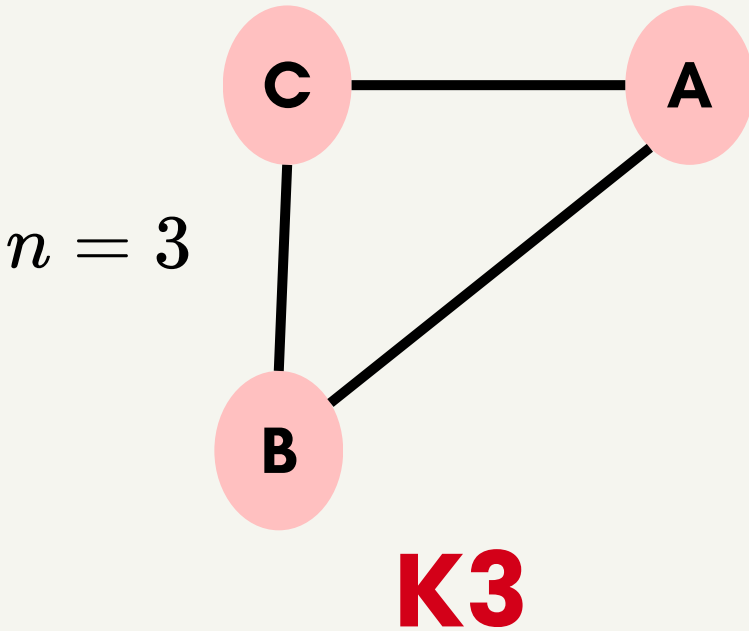
$n \rightarrow$ número de vértices
 $n-1 \rightarrow$ grau de todos os vértices

$$|A| = \frac{n(n - 1)}{2}$$



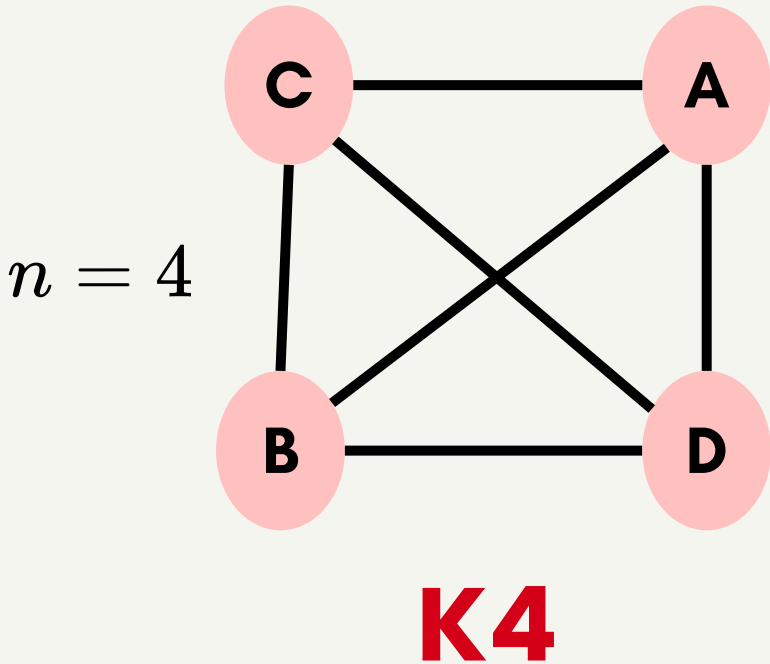
$$d(v) = 2 - 1 = 1$$

$$|A| = \frac{2(2 - 1)}{2} = 1$$



$$d(v) = 3 - 1 = 2$$

$$|A| = \frac{3(3 - 1)}{2} = 3$$

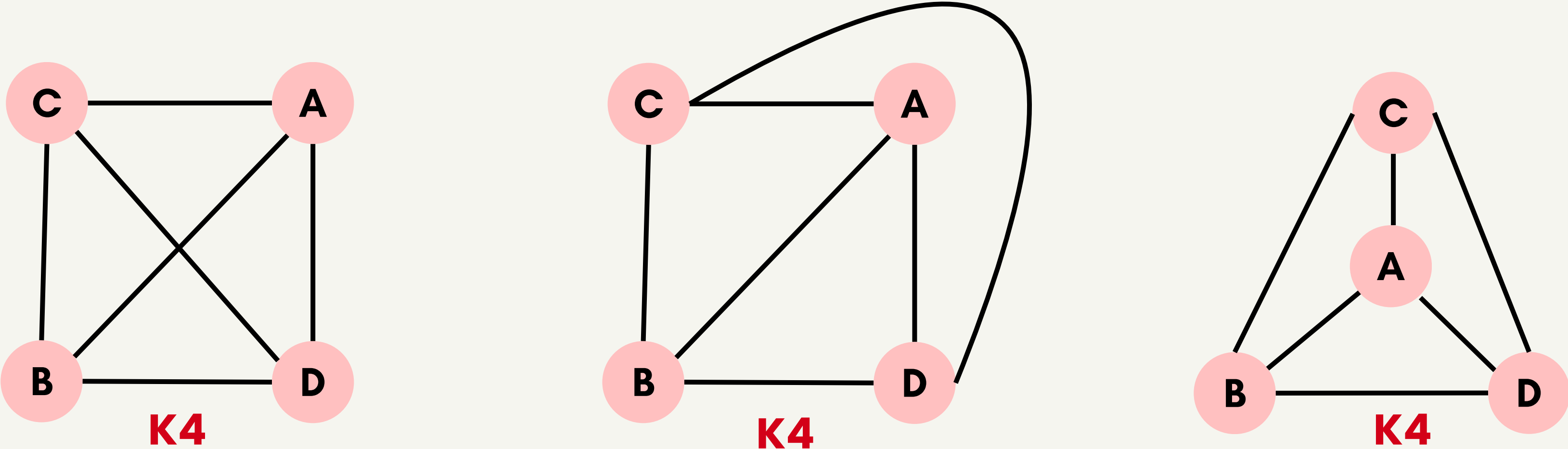


$$d(v) = 4 - 1 = 3$$

$$|A| = \frac{4(4 - 1)}{2} = 6$$

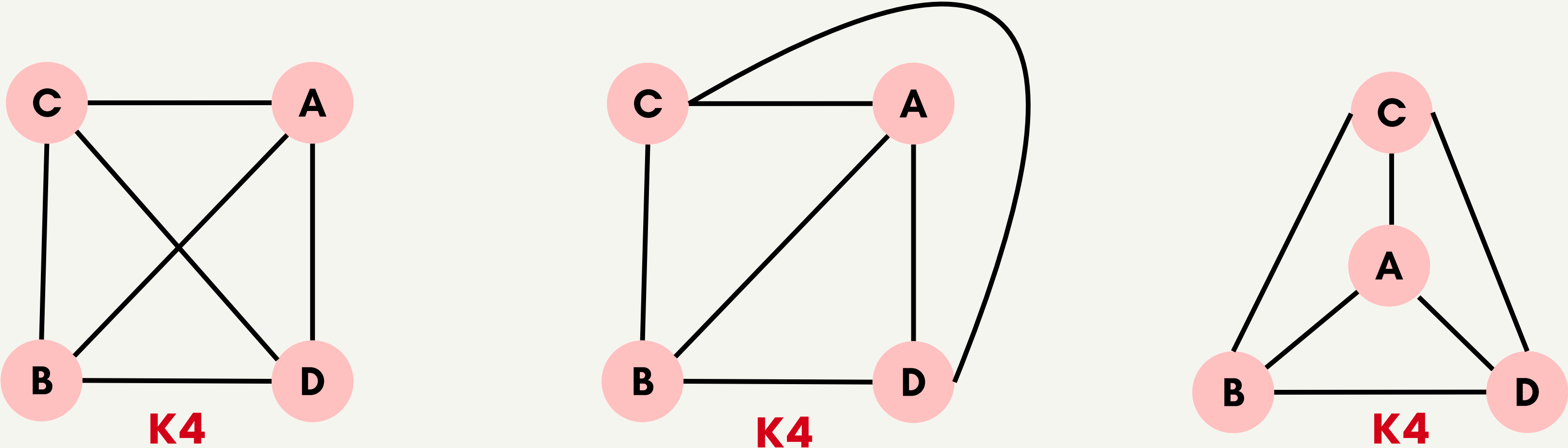
GRAFOS ISOMORFOS

Quando existe uma correspondência entre seus vértices que preserve as ligações.
Representações gráficas distintas para um mesmo grafo



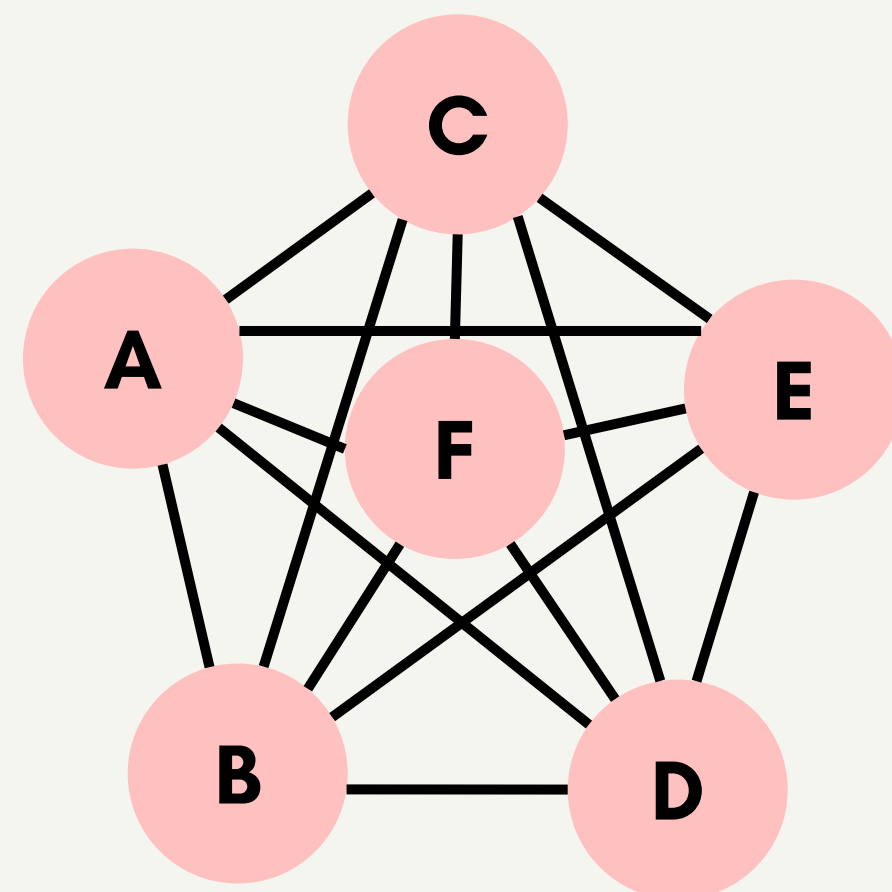
GRAFOS ISOMORFOS

Quando existe uma correspondência entre seus vértices que preserve as ligações.
Representações gráficas distintas para um mesmo grafo



MESMO NÚMERO DE VÉRTICES;
MESMO NÚMERO DE ARESTAS;
MESMA SEQUÊNCIA DE GRAUS;
MESMA RELAÇÃO DE ADJACÊNCIA ENTRE OS VÉRTICES.

GRAFOS DENSOS x ESPARSOS



GRAFO DENSO

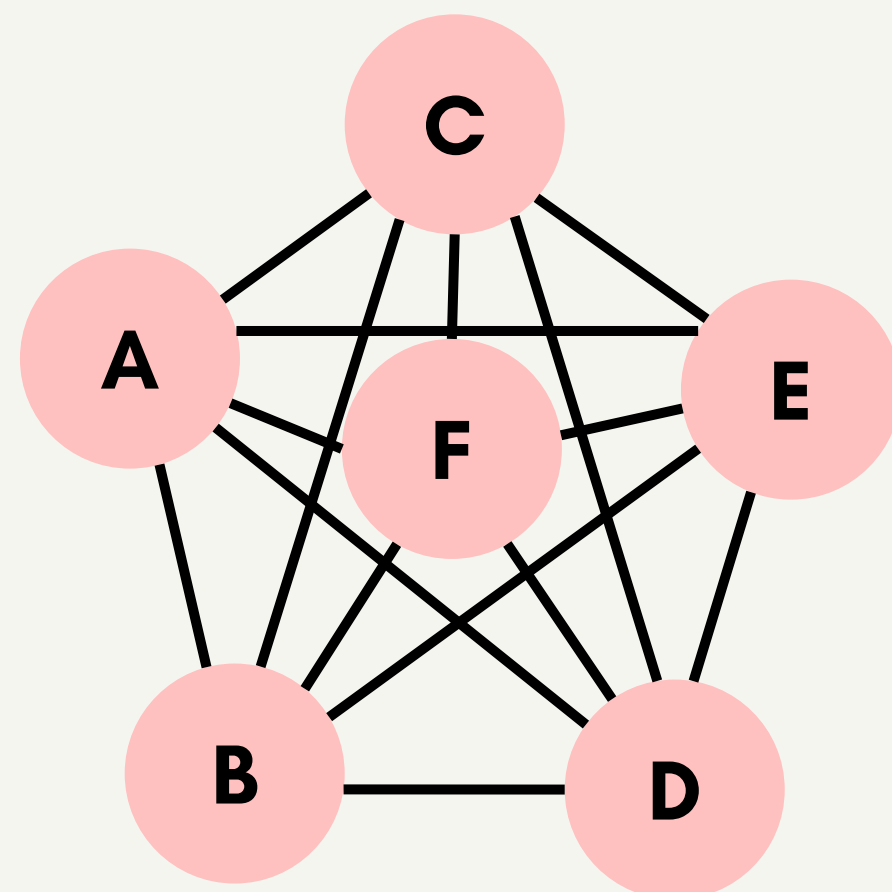
Muitos relacionamentos em relação aos nós

Muitas arestas

Possuem **matriz de adjacências densa** →

Muitos campos preenchidos

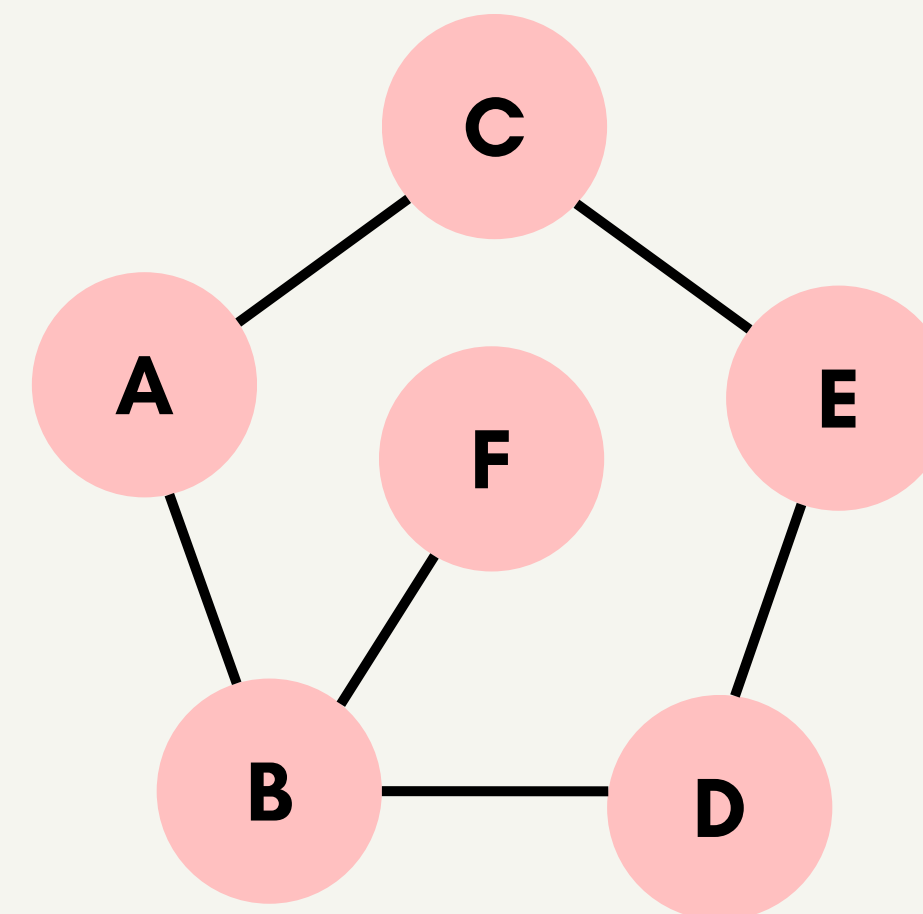
GRAFOS DENSOS x ESPARSOS



GRAFO DENSO

Muitos relacionamentos em relação aos nós
Muitas arestas

Possuem **matriz de adjacências densa** →
Muitos campos preenchidos



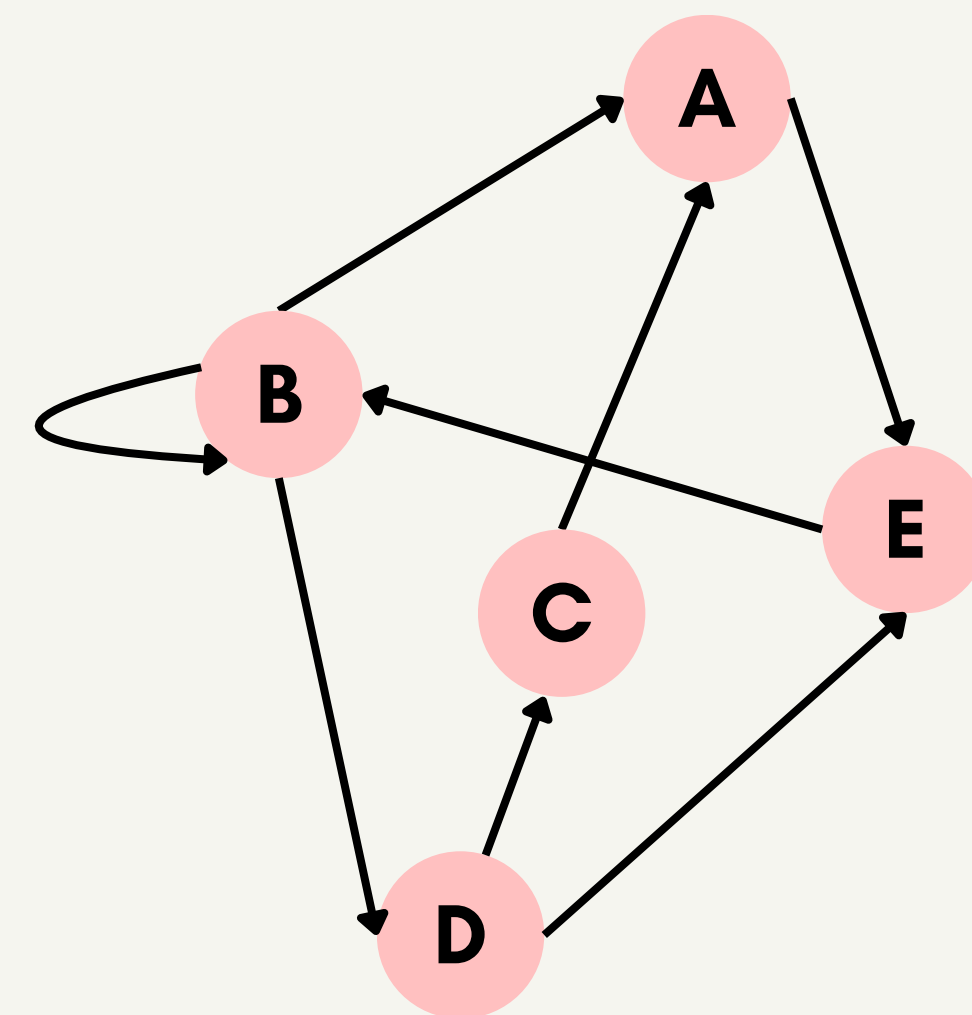
GRAFO ESPARSO

Poucos relacionamentos em relação aos nós
Poucas arestas

Possuem **matriz de adjacências esparsa** →
a maioria dos elementos é zero, nula ou vazia.

Passeio → Sequência de vértices ligados por arestas, podendo repetir vértices e arestas.

Passeio B à A → **(B,D), (D,E),(E,B),(B,A)**



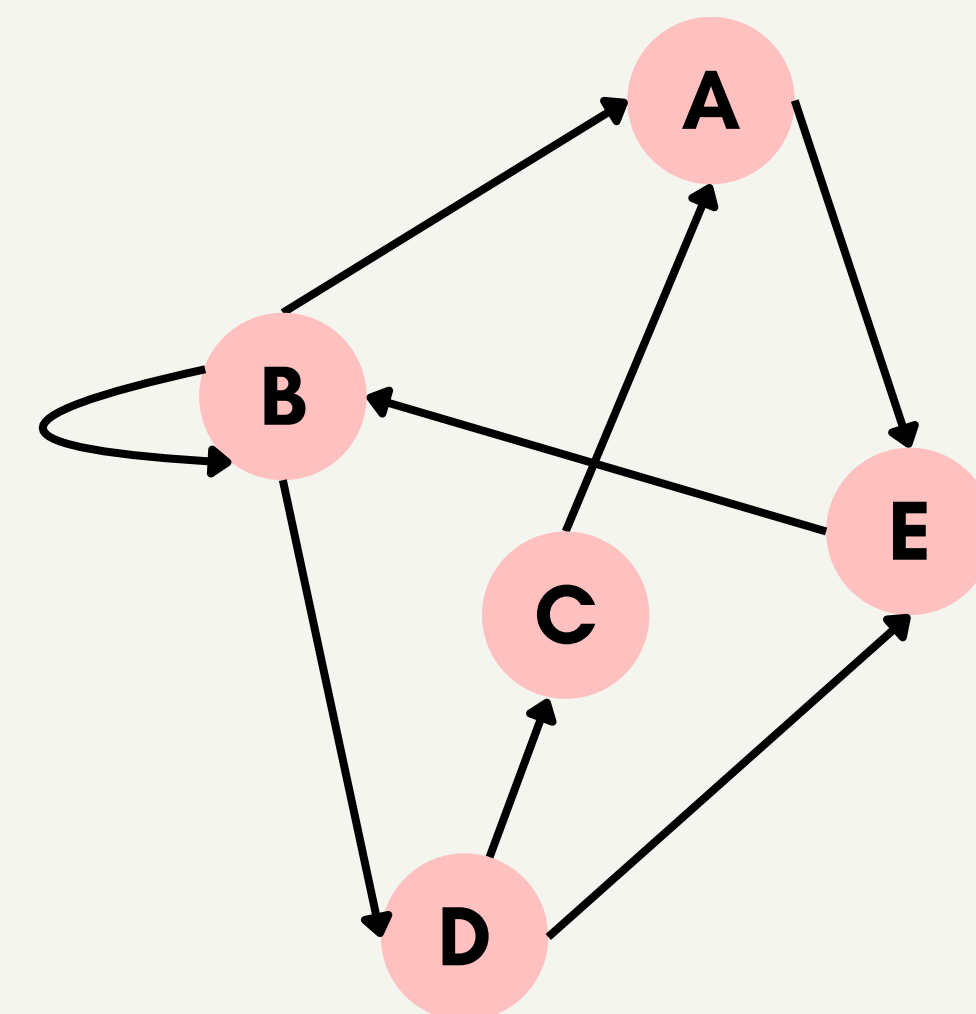
Passeio → Sequência de vértices ligados por arestas, podendo repetir vértices e arestas.

Passeio B à A → $(B,D), (D,E), (E,B), (B,A)$

Trilha → É um passeio sem repetição de **arestas**.

Trilha B à A → $(B,D), (D,E), (E,B), (B,A)$

Trilha fechada: vértice de saída = vértice de entrada



Passeio → Sequência de vértices ligados por arestas, podendo repetir vértices e arestas.

Passeio B à A → (B,D), (D,E),(E,B),(B,A)

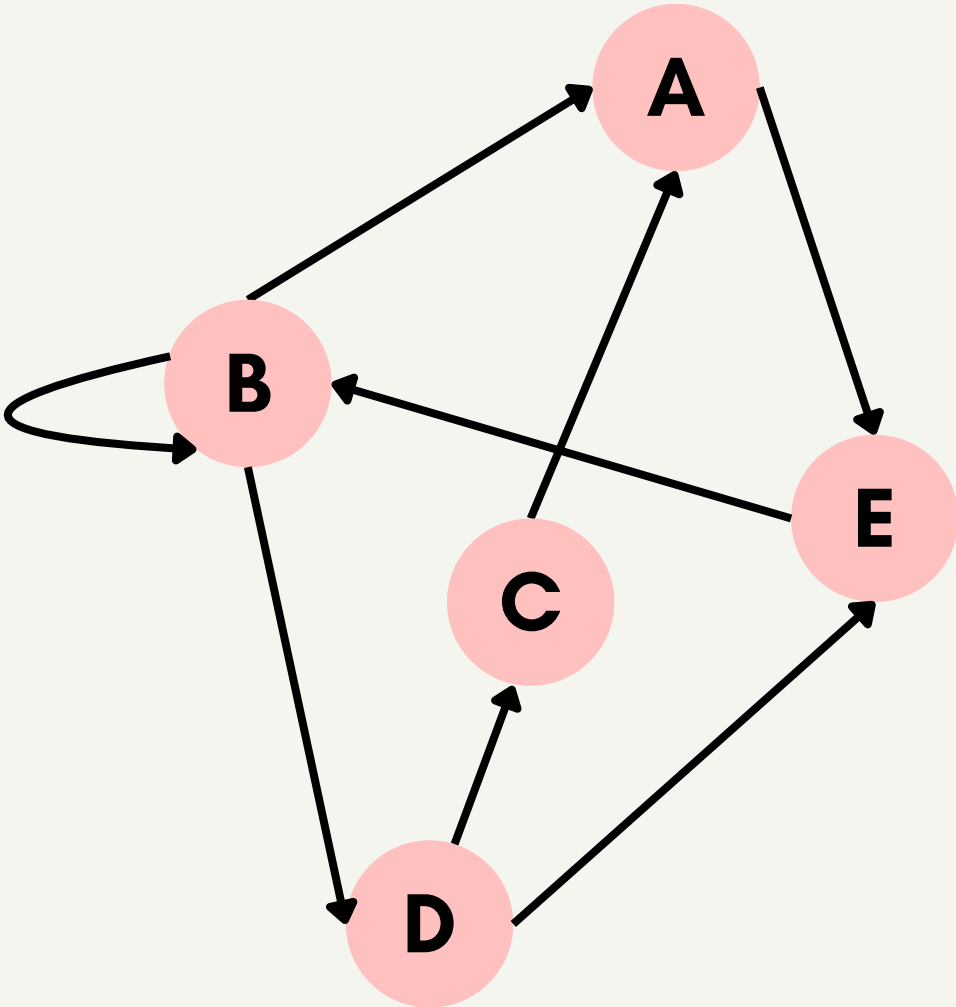
Trilha → É um passeio sem repetição de **arestas**.

Trilha B à A → (B,D), (D,E),(E,B),(B,A)

Trilha fechada: vértice de saída = vértice de entrada

Caminho → É um passeio sem repetição de **vértices** e **arestas**.

Caminho B à A → (B,D), (D,C),(C,A)



Passeio → Sequência de vértices ligados por arestas, podendo repetir vértices e arestas.

Passeio B à A → $(B,D), (D,E), (E,B), (B,A)$

Trilha → É um passeio sem repetição de **arestas**.

Trilha B à A → $(B,D), (D,E), (E,B), (B,A)$

Trilha fechada: vértice de saída = vértice de entrada

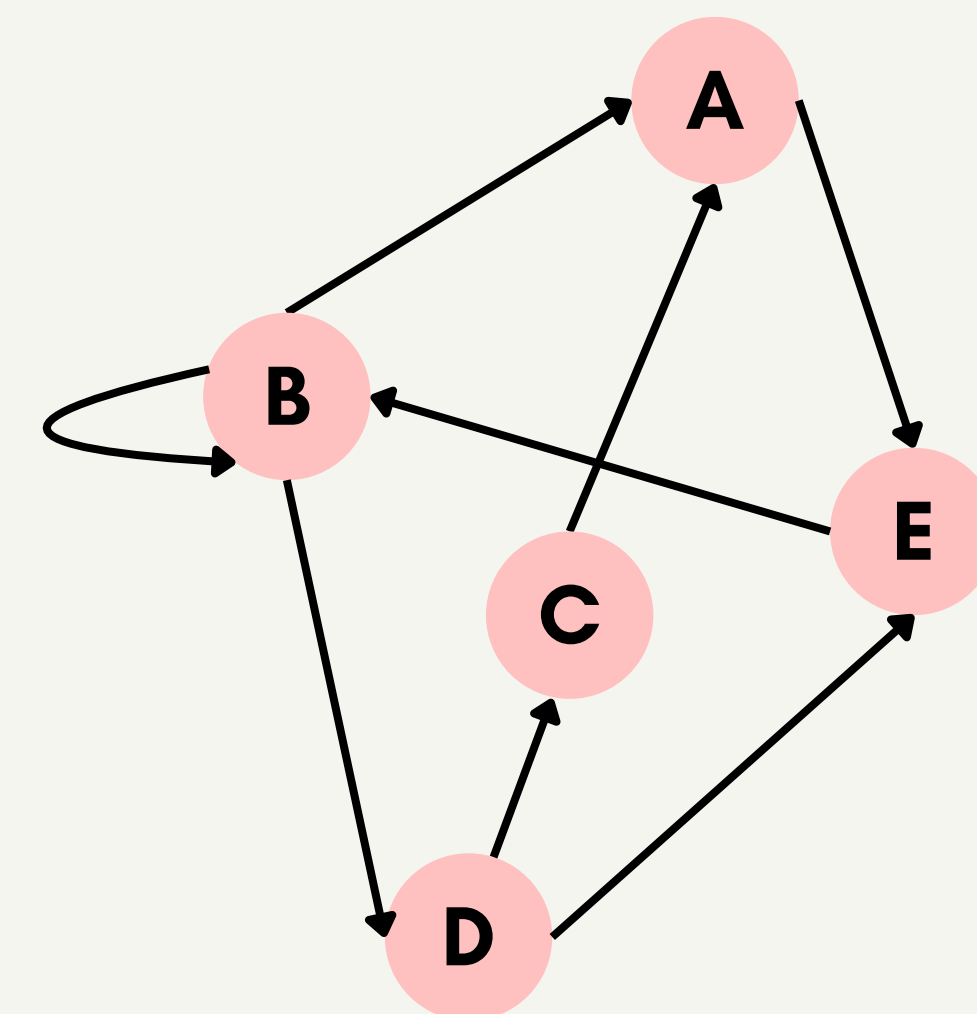
Caminho → É um passeio sem repetição de **vértices** e **arestas**.

Caminho B à A → $(B,D), (D,C), (C,A)$

Ciclo → É um **caminho** em que vértice de saída = vértice de entrada

$(B,D), (D,E), (E,B)$

Laços são ciclos de comprimento 1



Grafo Euleriano

Se existe uma **trilha fechada** (não repete arestas/ começa e termina no mesmo vértice) que passa por **todas** as **arestas** de G .

Grafo Euleriano

Se existe uma **trilha fechada** (não repete arestas/ começa e termina no mesmo vértice) que passa por **todas** as **arestas** de G .

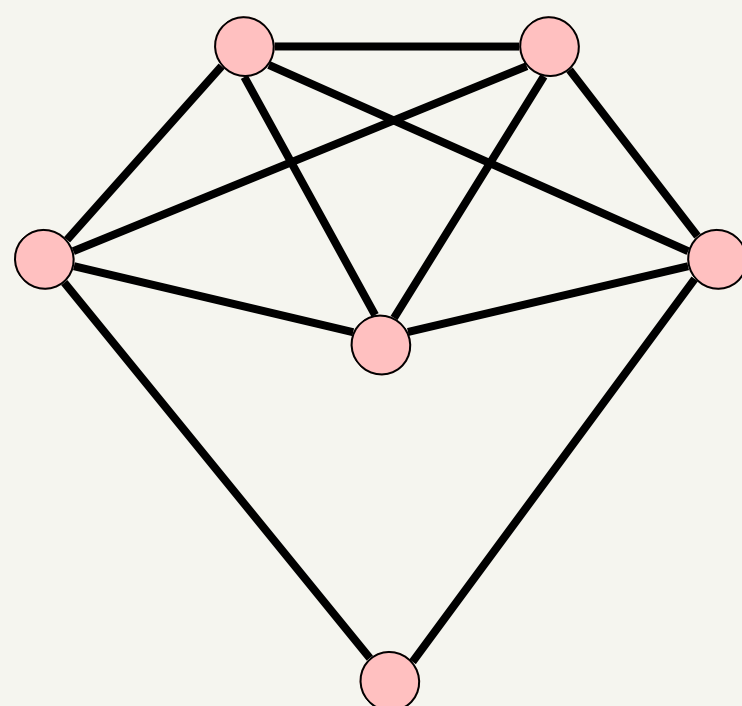
Teorema: Um grafo conexo não-direcionado G é Euleriano se, e somente se, **todos** os seus vértices tem **grau par**. (nenhum de grau ímpar)

Grafo Euleriano

Se existe uma **trilha fechada** (não repete arestas/ começa e termina no mesmo vértice) que passa por **todas** as **arestas** de G .

Teorema: Um grafo conexo não-direcionado G é Euleriano se, e somente se, **todos** os seus vértices tem **grau par**. (nenhum de grau ímpar)

É euleriano?

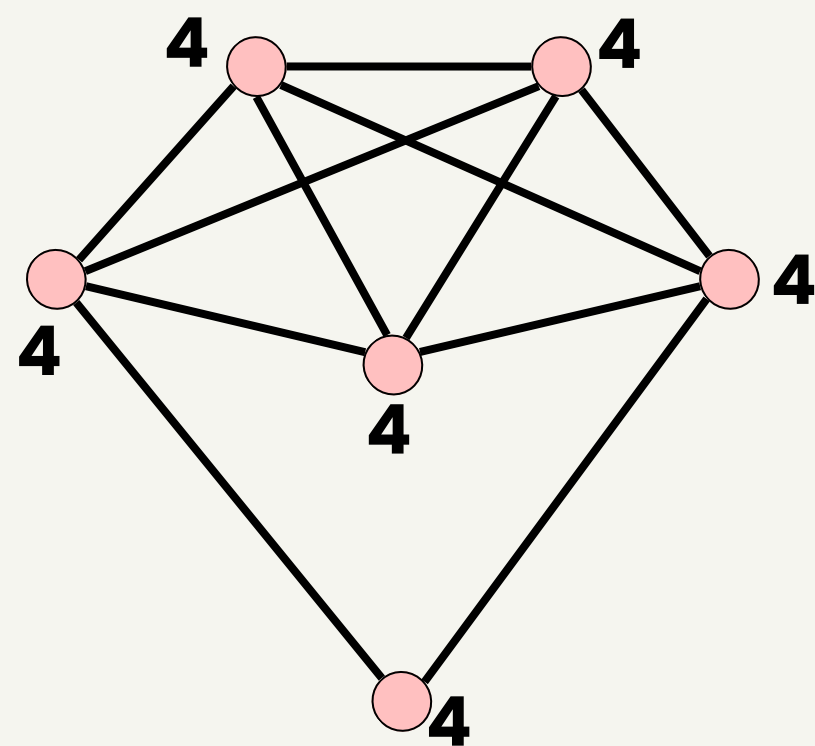


Grafo Euleriano

Se existe uma **trilha fechada** (não repete arestas/ começa e termina no mesmo vértice) que passa por **todas** as **arestas** de G .

Teorema: Um grafo conexo não-direcionado G é Euleriano se, e somente se, **todos** os seus vértices tem **grau par**. (nenhum de grau ímpar)

É euleriano?

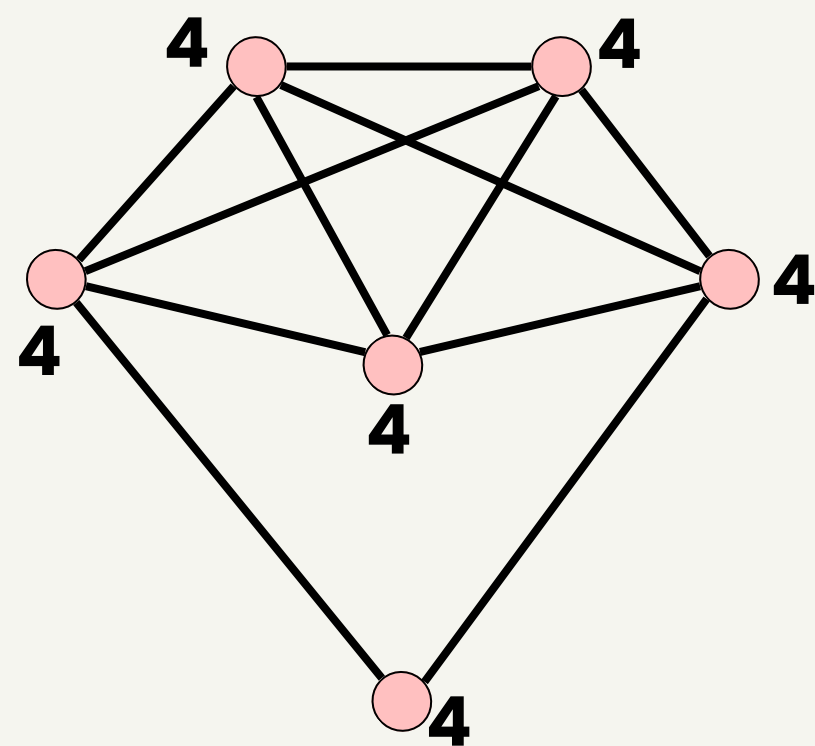


Grafo Euleriano

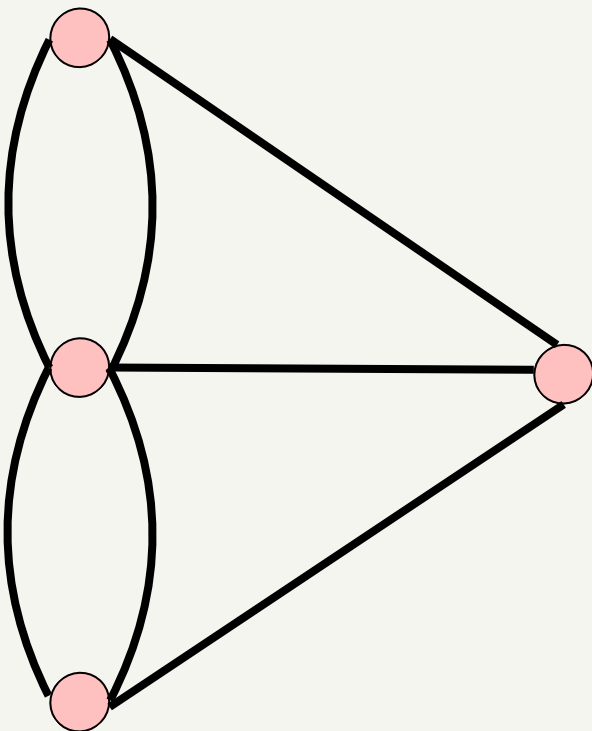
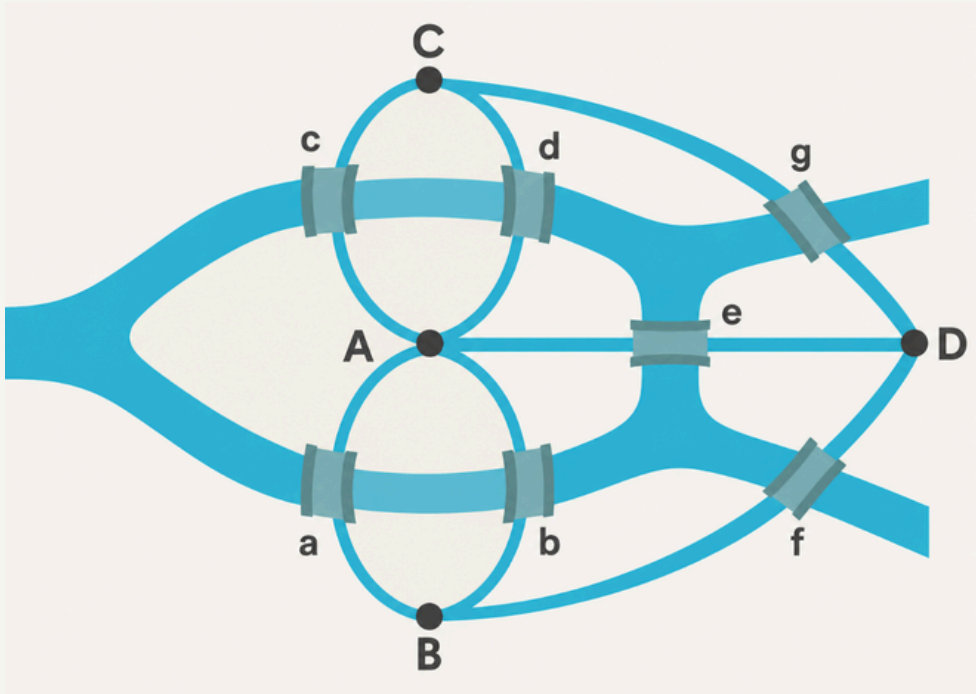
Se existe uma **trilha fechada** (não repete arestas/ começa e termina no mesmo vértice) que passa por **todas** as **arestas** de G .

Teorema: Um grafo conexo não-direcionado G é Euleriano se, e somente se, **todos** os seus vértices tem **grau par**. (nenhum de grau ímpar)

É euleriano?



É euleriano?

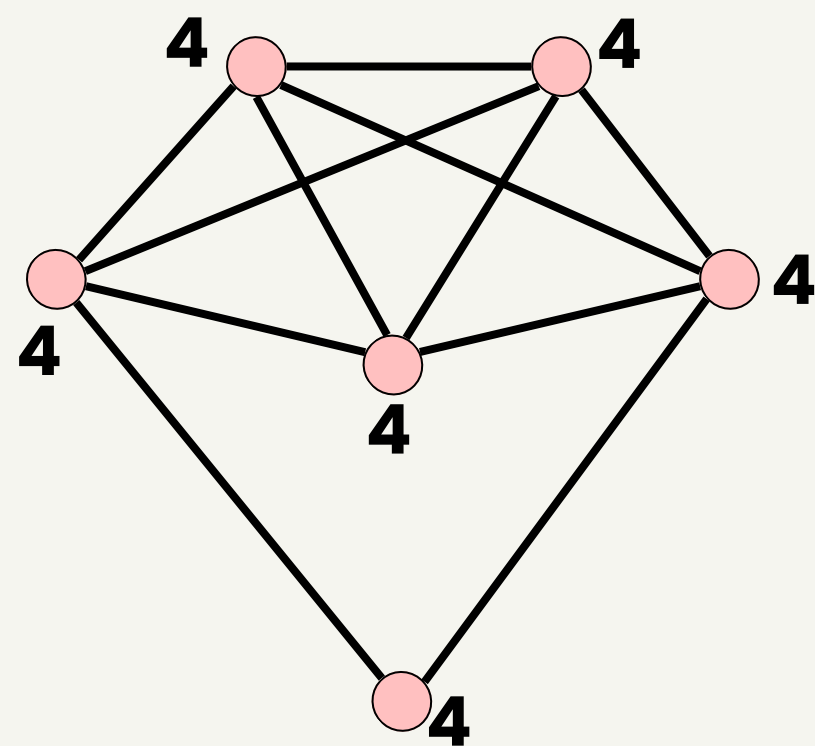


Grafo Euleriano

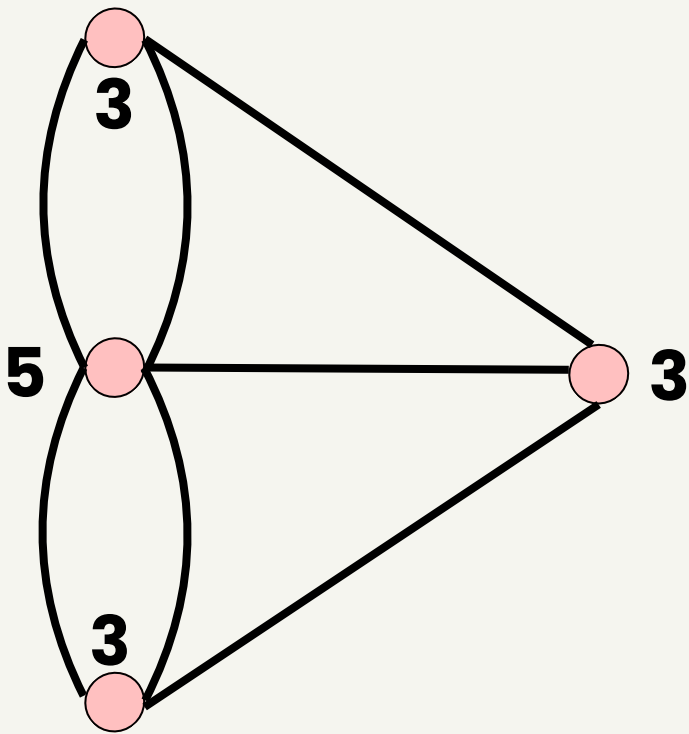
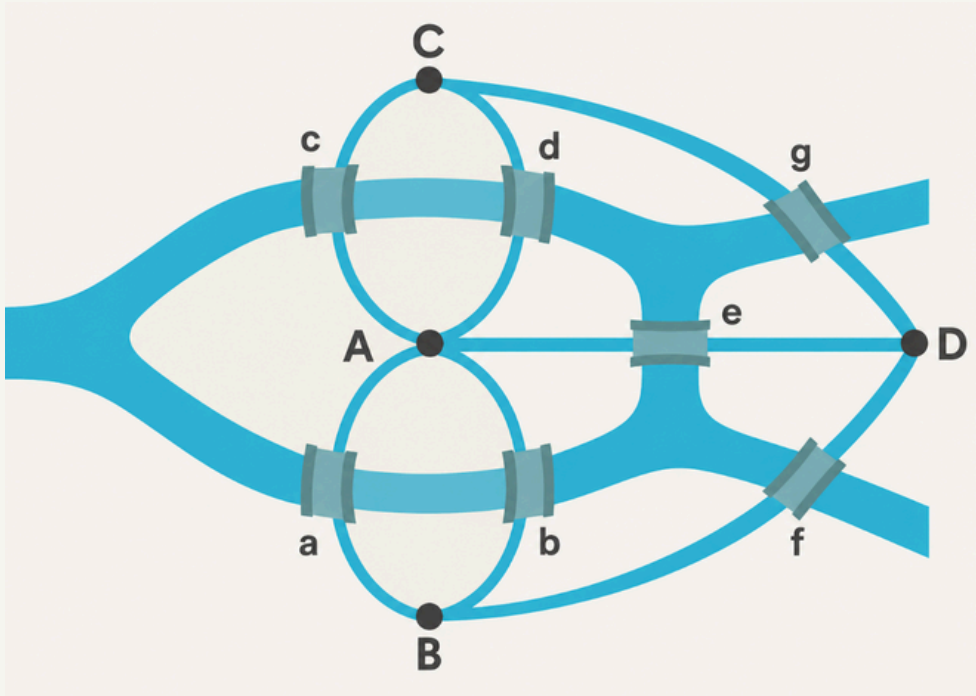
Se existe uma **trilha fechada** (não repete arestas/ começa e termina no mesmo vértice) que passa por **todas** as **arestas** de G .

Teorema: Um grafo conexo não-direcionado G é Euleriano se, e somente se, **todos** os seus vértices tem **grau par**. (nenhum de grau ímpar)

É euleriano?



É euleriano?



Grafo Semieuleriano

Se existe uma **trilha aberta** (não repete arestas/ começa e termina em vértices diferentes) que passa por **todas** as **arestas** de G .

Grafo Semieuleriano

Se existe uma **trilha aberta** (não repete arestas/ começa e termina em vértices diferentes) que passa por **todas** as **arestas** de G .

Teorema: Um grafo conexo não direcionado G é Semieuleriano se, e somente se, G tem exatamente **DOIS** vértices de **grau ímpar**

Grafo Semieuleriano

Se existe uma **trilha aberta** (não repete arestas/ começa e termina em vértices diferentes) que passa por **todas** as **arestas** de G .

Teorema: Um grafo conexo não direcionado G é Semieuleriano se, e somente se, G tem exatamente **DOIS** vértices de **grau ímpar**

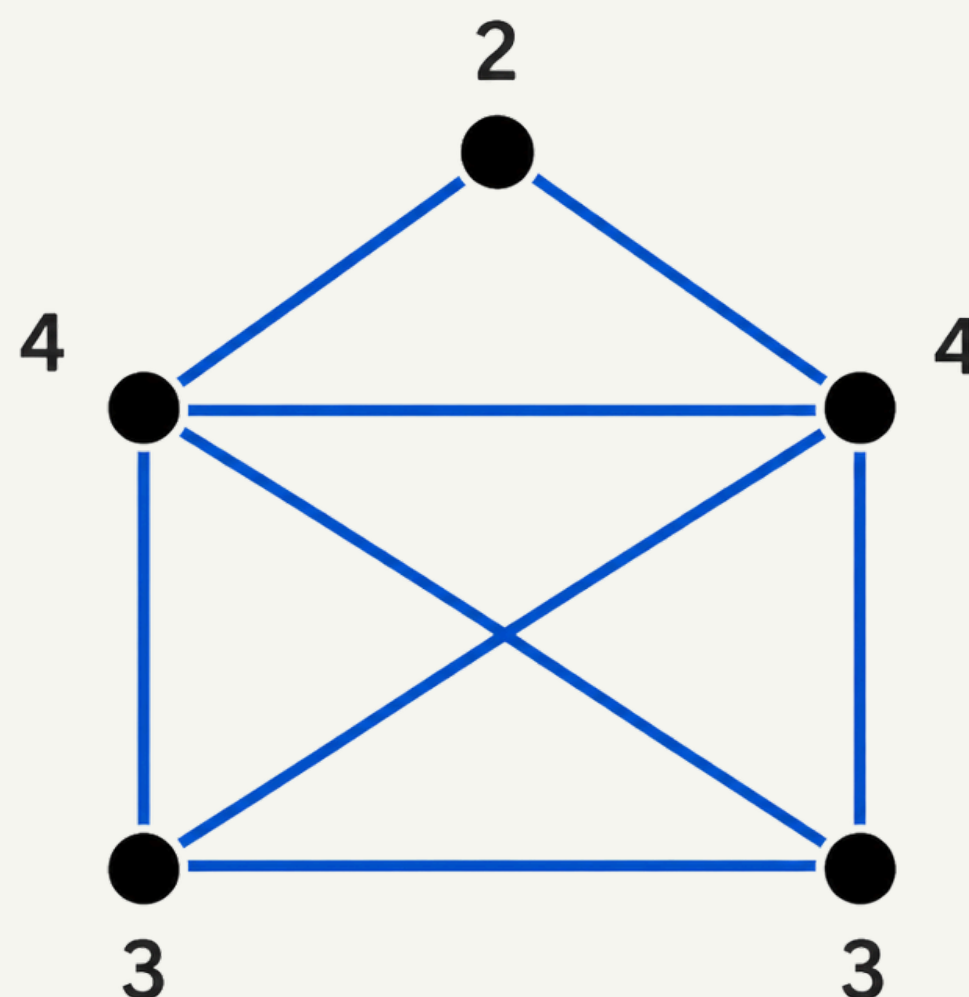
A trilha começa em um vértice de grau ímpar e termina em outro ímpar.

Grafo Semieuleriano

Se existe uma **trilha aberta** (não repete arestas/ começa e termina em vértices diferentes) que passa por **todas** as **arestas** de G .

Teorema: Um grafo conexo não direcionado G é Semieuleriano se, e somente se, G tem exatamente **DOIS** vértices de **grau ímpar**

A trilha começa em um vértice de grau ímpar e termina em outro ímpar.



É semi-euleriano?

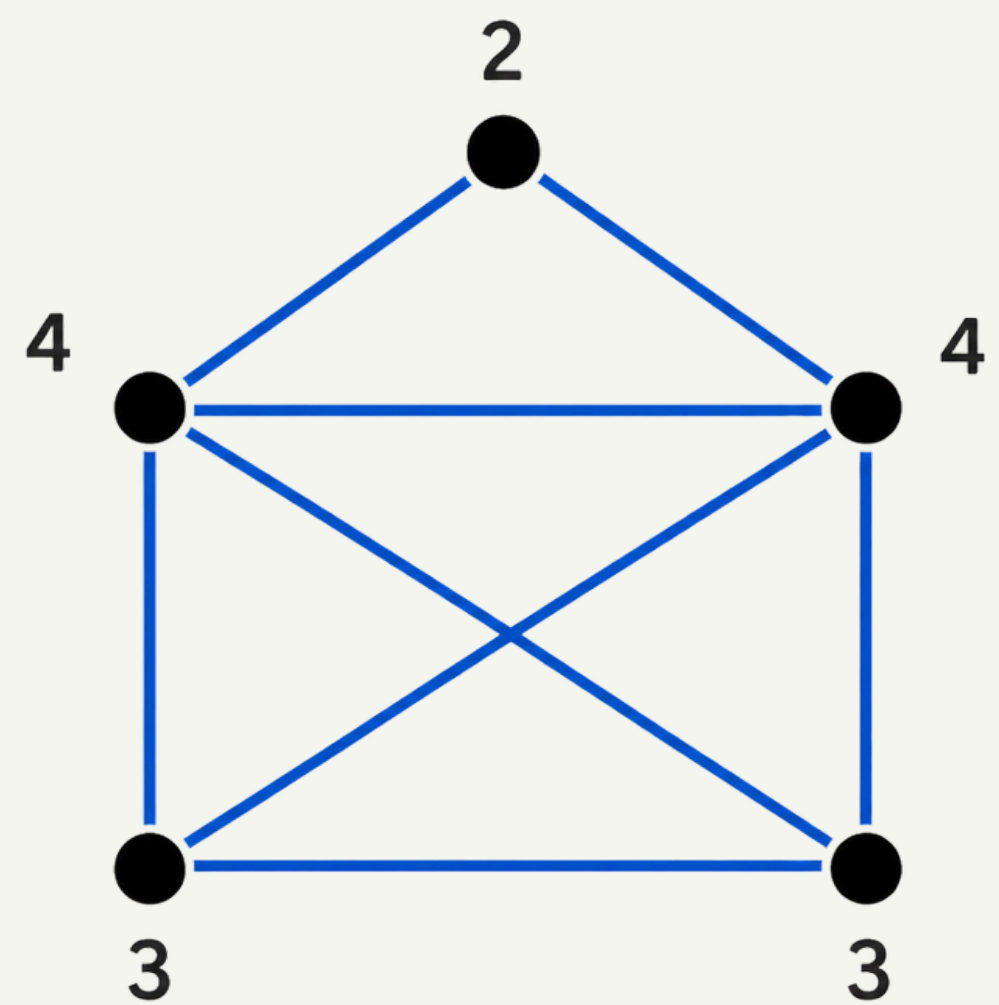
Grafo Semieuleriano

Se existe uma **trilha aberta** (não repete arestas/ começa e termina em vértices diferentes) que passa por **todas** as **arestas** de G .

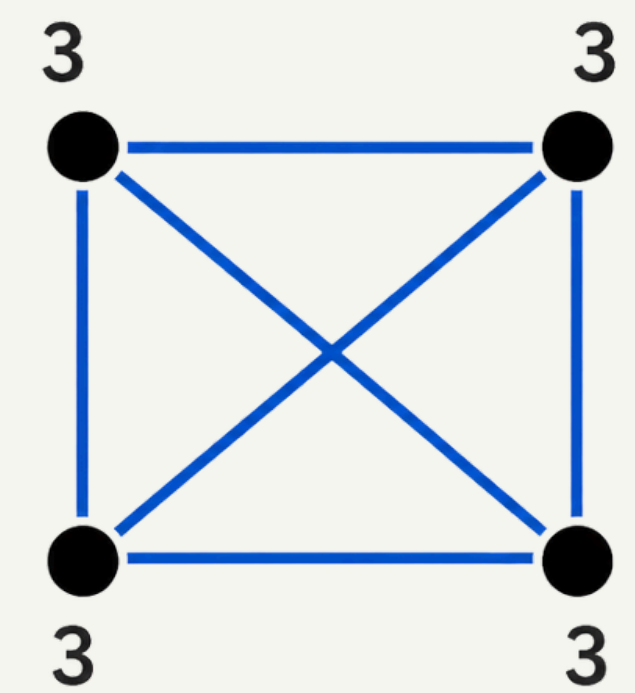
Teorema: Um grafo conexo não direcionado G é Semieuleriano se, e somente se, G tem exatamente **DOIS** vértices de **grau ímpar**

A trilha começa em um vértice de grau ímpar e termina em outro ímpar.

É semi-euleriano?



É semi-euleriano?



Trilha euleriana → Passa por todas as arestas exatamente **uma** vez

Trilha euleriana → Passa por todas as arestas exatamente **uma** vez

Possui **trilha euleriana fechada (0 vértices com grau ímpar)** → **Grafo euleriano**

Possui **trilha euleriana aberta (2 vértices com grau ímpar)** → **Grafo semi-euleriano**

Trilha euleriana → Passa por todas as arestas exatamente uma vez

Possui **trilha euleriana fechada (0 vértices com grau ímpar)** → **Grafo euleriano**

Possui **trilha euleriana aberta (2 vértices com grau ímpar)** → **Grafo semi-euleriano**

Mais de 2 vértices com grau ímpar → **IMPOSSÍVEL** passar por todas as arestas exatamente uma vez

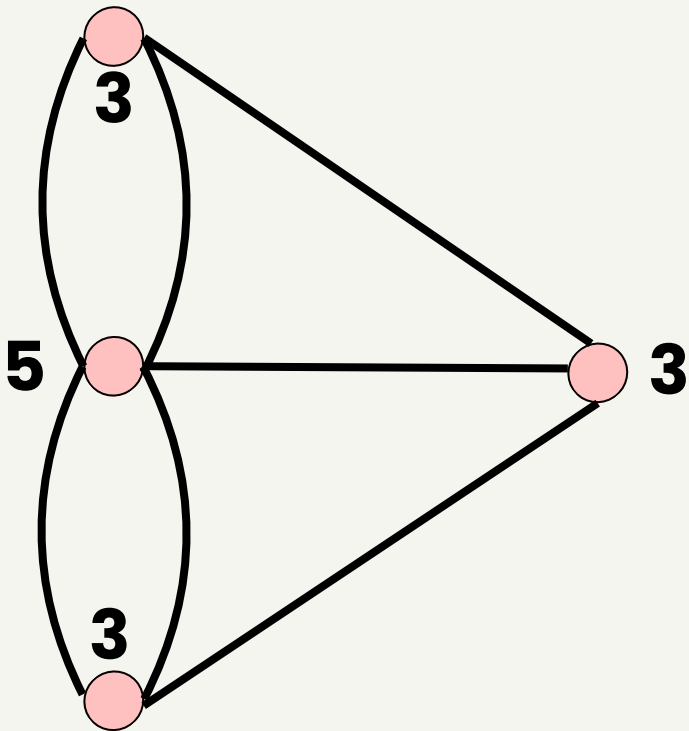
Trilha euleriana → Passa por todas as arestas exatamente uma vez

Possui **trilha euleriana fechada** (0 vértices com grau ímpar) → **Grafo euleriano**

Possui **trilha euleriana aberta** (2 vértices com grau ímpar) → **Grafo semi-euleriano**

Mais de 2 vértices com grau ímpar → **IMPOSSÍVEL** passar por todas as arestas exatamente uma vez

As Sete Pontes de Königsberg



Trilha euleriana → Passa por todas as arestas exatamente uma vez

Possui **trilha euleriana fechada** (0 vértices com grau ímpar) → **Grafo euleriano**

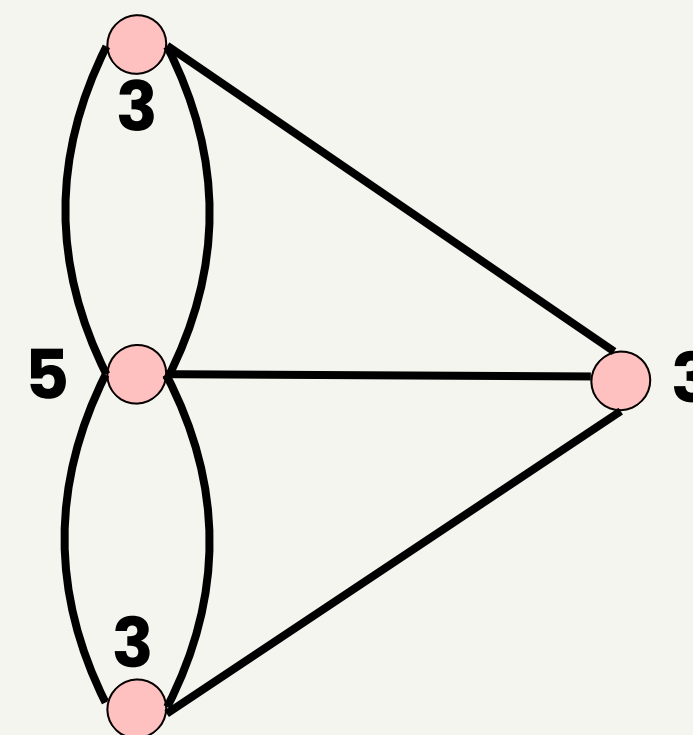
Possui **trilha euleriana aberta** (2 vértices com grau ímpar) → **Grafo semi-euleriano**

Mais de 2 vértices com grau ímpar → **IMPOSSÍVEL** passar por todas as arestas exatamente uma vez

As Sete Pontes de Königsberg

4 vértices com grau ímpar

- Não é euleriano
- não é semieuleriano



Caminho Hamiltoniano

É um **caminho** (não repete vértices intermediários e nem aresta) que contém **todos os vértices** de um grafo exatamente **uma vez**.

Caminho Hamiltoniano

É um **caminho** (não repete vértices intermediários e nem aresta) que contém **todos os vértices** de um grafo exatamente **uma vez**.

Ciclo Hamiltoniano

É um caminho hamiltoniano em que **o primeiro e o último vértices são iguais**.

Caminho Hamiltoniano

É um **caminho** (não repete vértices intermediários e nem aresta) que contém **todos os vértices** de um grafo exatamente **uma vez**.

Ciclo Hamiltoniano

É um caminho hamiltoniano em que **o primeiro e o último vértices são iguais**.

Grafo Hamiltoniano

Se existe um **ciclo Hamiltoniano**

Caminho Hamiltoniano

É um **caminho** (não repete vértices intermediários e nem aresta) que contém **todos os vértices** de um grafo exatamente **uma vez**.

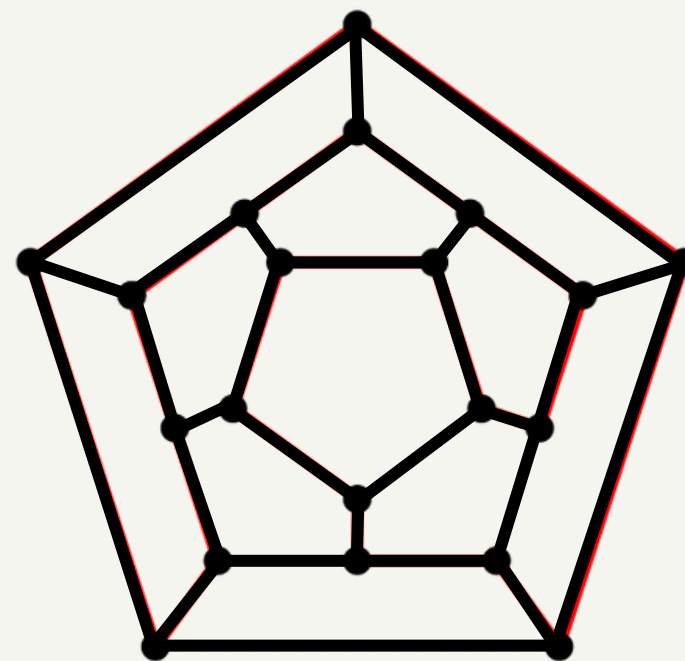
Ciclo Hamiltoniano

É um caminho hamiltoniano em que **o primeiro e o último vértices são iguais**.

Grafo Hamiltoniano

Se existe um **ciclo Hamiltoniano**

É um um grafo
Hamiltoniano?



Caminho Hamiltoniano

É um **caminho** (não repete vértices intermediários e nem aresta) que contém **todos os vértices** de um grafo exatamente **uma vez**.

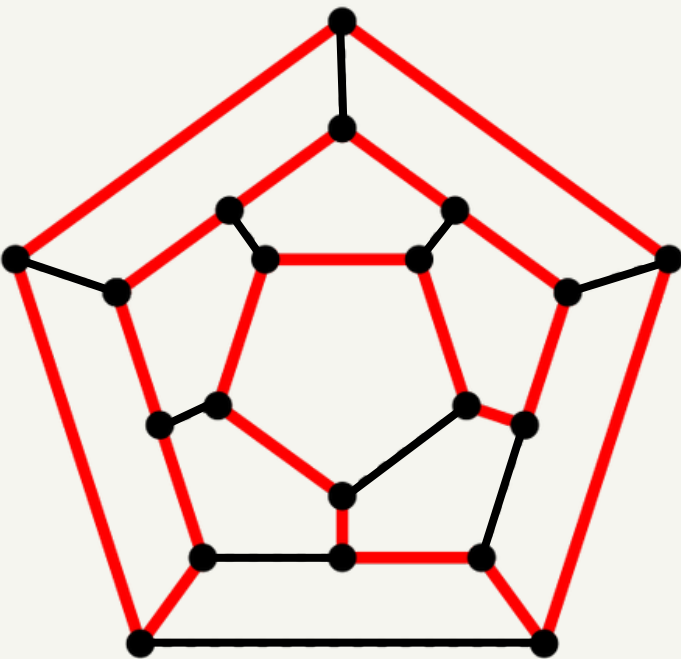
Ciclo Hamiltoniano

É um caminho hamiltoniano em que **o primeiro e o último vértices são iguais**.

Grafo Hamiltoniano

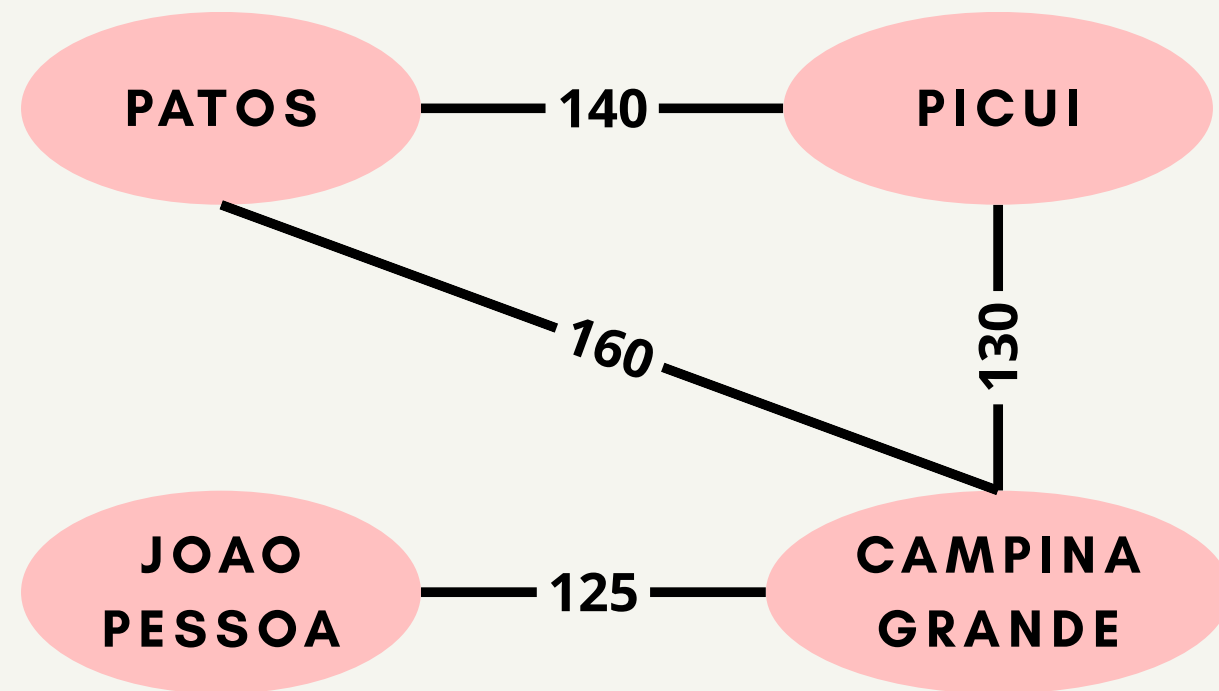
Se existe um **ciclo Hamiltoniano**

É um um grafo Hamiltoniano?



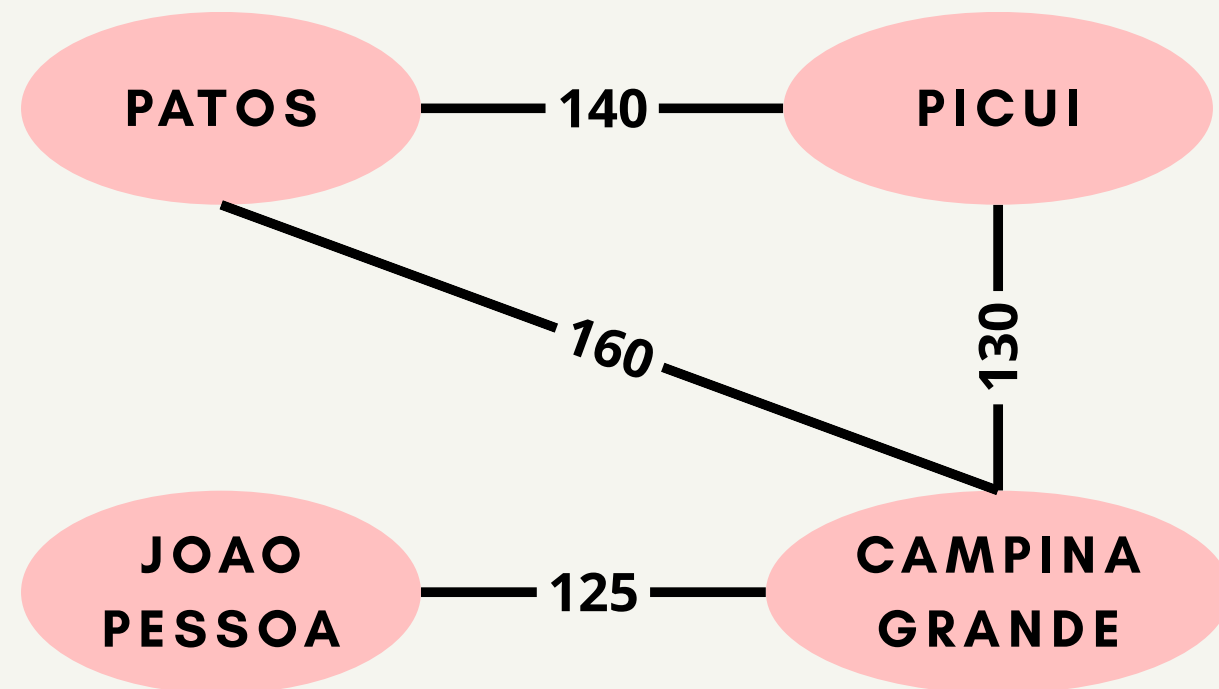
Representação computacional do grafo – Matriz de adjacências

Grafo simples não-direcionado ponderado



Representação computacional do grafo – Matriz de adjacências

Grafo simples não-direcionado ponderado

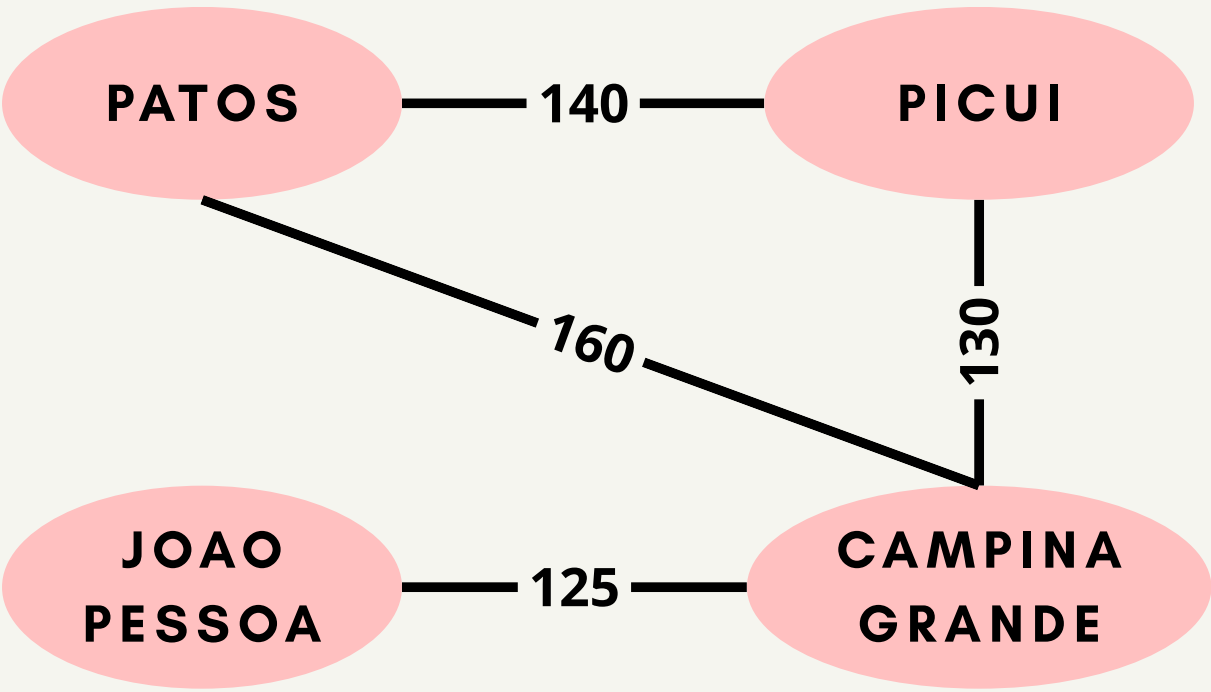


$V = \{JP, CG, Patos, Picui\}$

$A = \{(JP,CG), (CG,Picui), (CG, Patos), (Patos, Picui)\}$

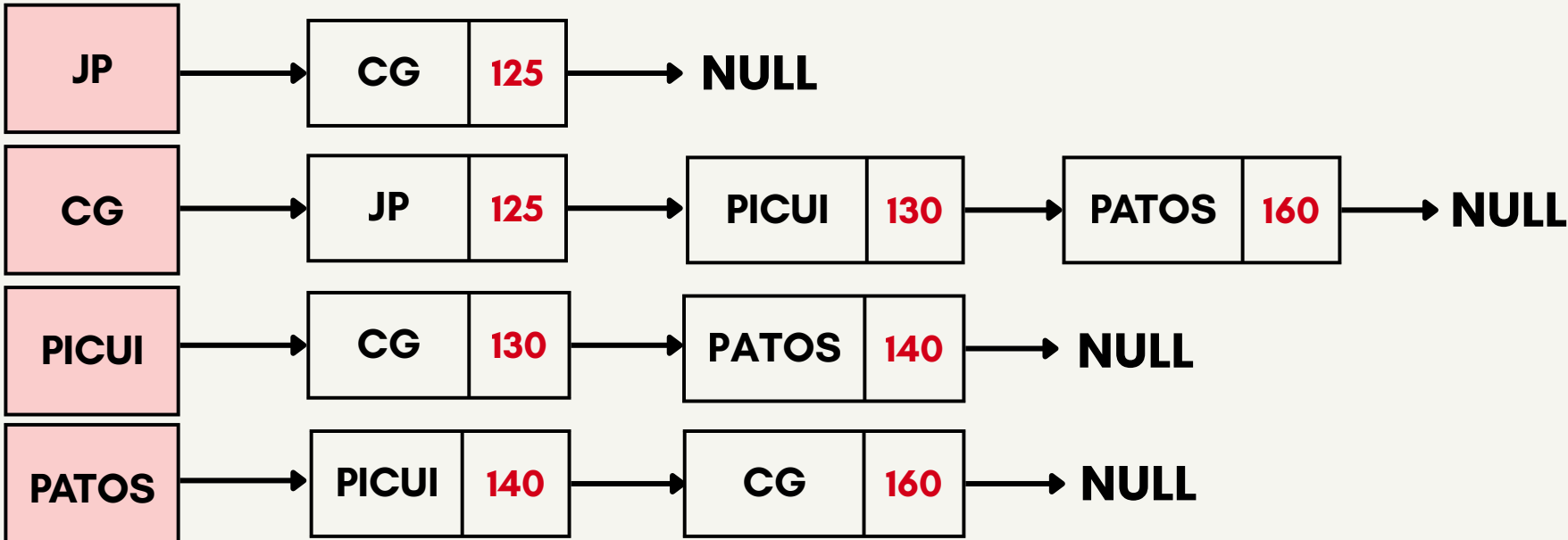
Representação computacional do grafo – Matriz de adjacências

Grafo simples não-direcionado ponderado



$V = \{JP, CG, Patos, Picui\}$

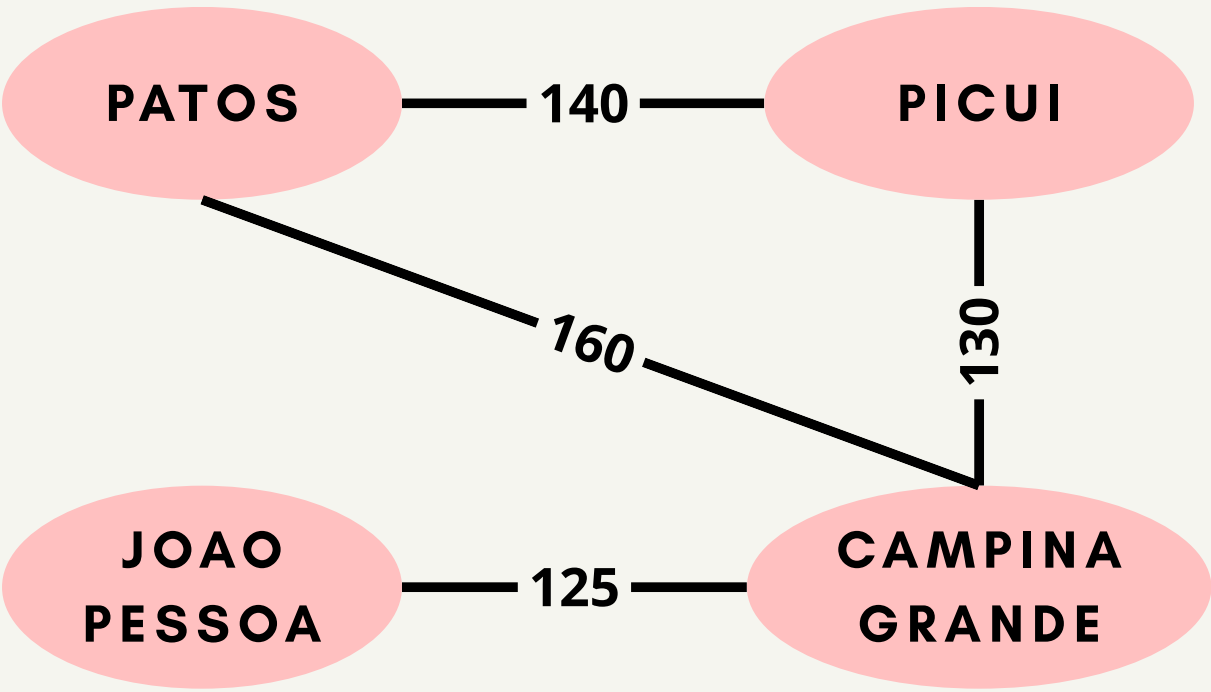
$A = \{(JP,CG), (CG,Picui), (CG, Patos), (Patos, Picui)\}$



Lista de adjacências

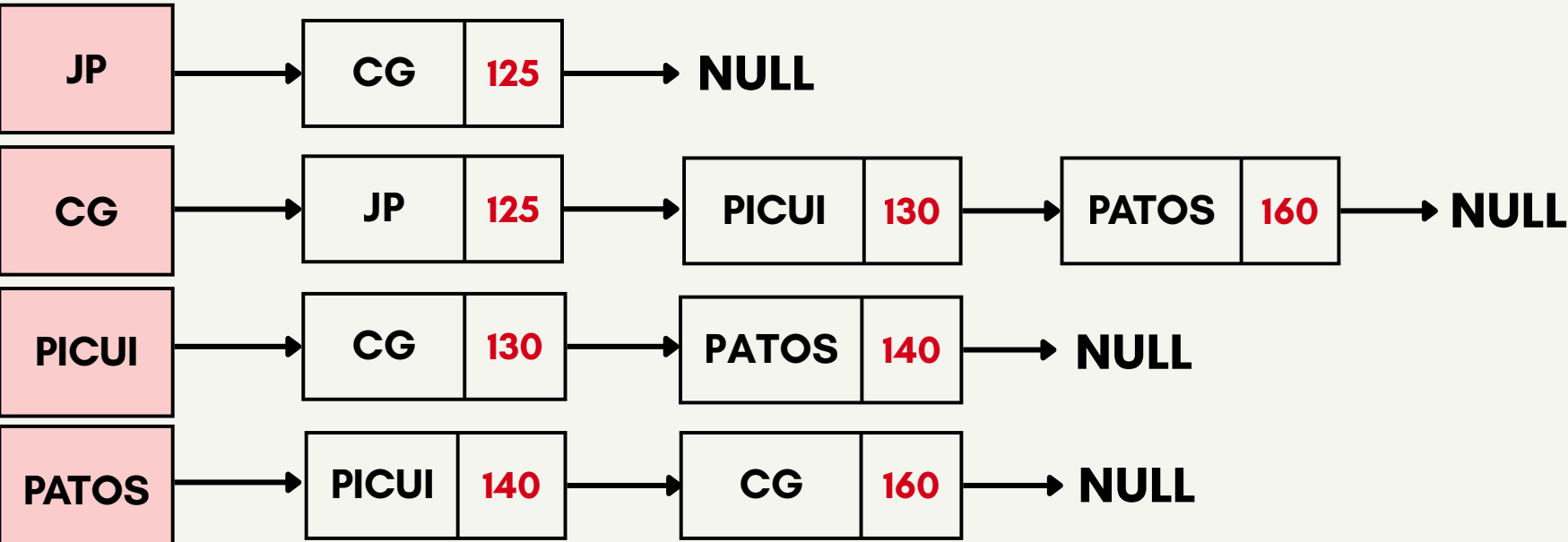
Representação computacional do grafo – Matriz de adjacências

Grafo simples não-direcionado ponderado



$V = \{JP, CG, Patos, Picui\}$

$A = \{(JP,CG), (CG,Picui), (CG, Patos), (Patos, Picui)\}$



Lista de adjacências

	JP	CG	Patos	Picui
JP	0	125	0	0
CG	125	0	160	130
Patos	0	160	0	140
Picui	0	130	140	0

Matriz de adjacências

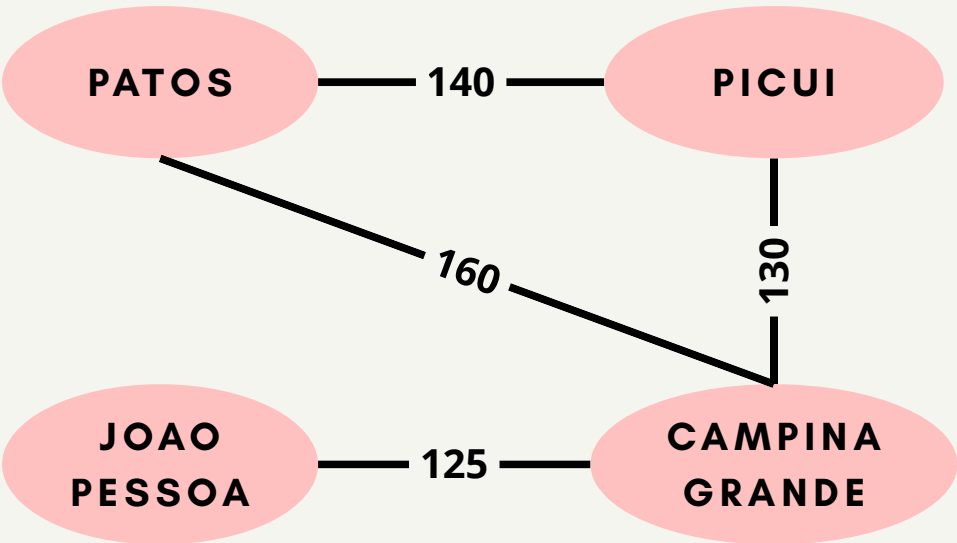
Representação computacional do grafo – Matriz de adjacências

Consideração:

Aresta inexistente (nula) → 0
Peso nulo → **não permitido**

```
typedef char TipoItem[50];
```

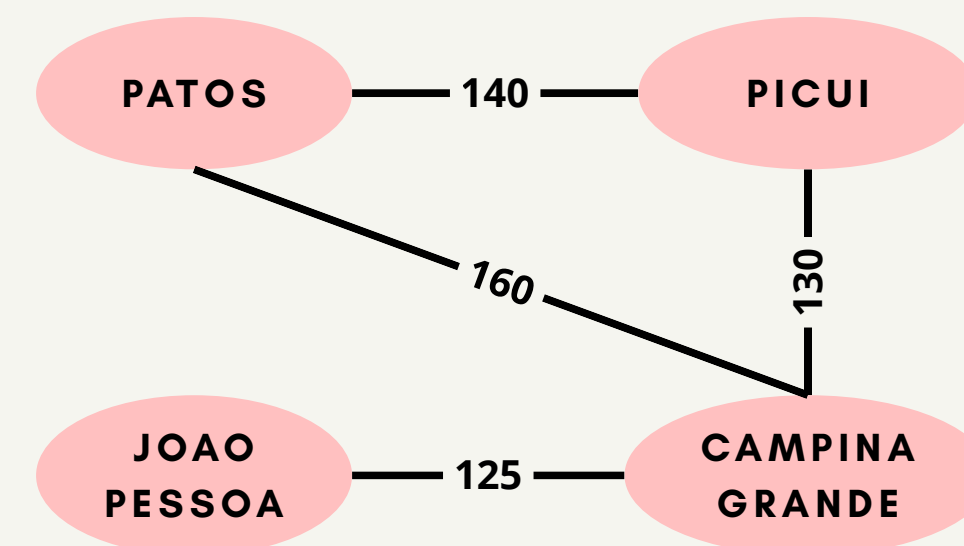
```
typedef struct Grafo {
    int arestanula;
    int maxvertices;
    int numvertices;
    TipoItem *vertices;
    int **matrizadjacencias;
} Grafo;
```



Representação computacional do grafo – Matriz de adjacências

```
typedef char TipoItem[50];
```

Tipo do valor armazenado no vértice



```
typedef struct Grafo {
```

```
    int arestanula;
```

```
    int maxvertices;
```

```
    int numvertices;
```

```
    TipoItem *vertices;
```

```
    int **matrizadjacencias;
```

```
} Grafo;
```

valor que representa ausência de aresta

quantidade máxima de vértices permitida

quantidade atual de vertices cadastrados

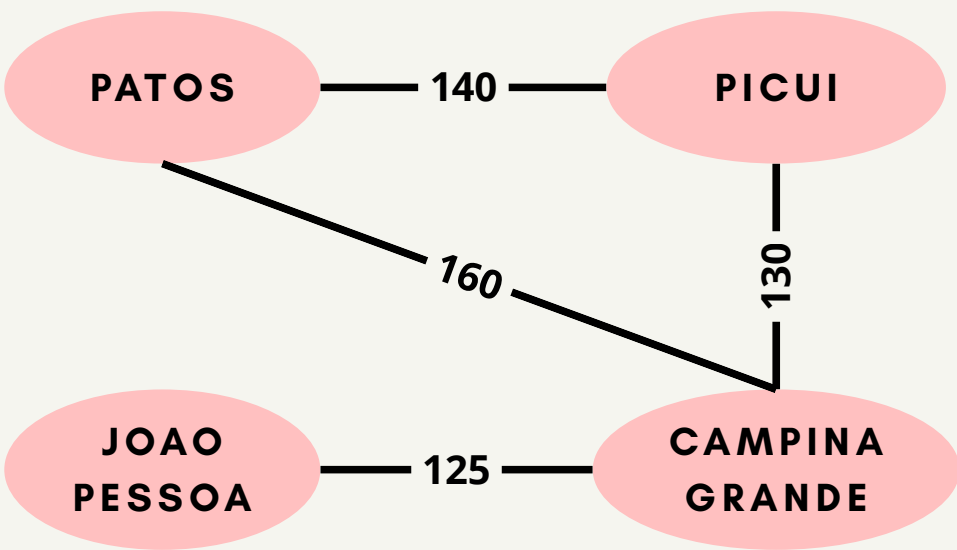
vetor de vertices dinâmico

matriz de adjacencias (vetor de vetores dinâmico)

O tamanho da **matriz** e **vetor** é definido em **tempo de execução**

Criar o Grafo → alocar memória e inicializar os campos

Desconsiderando falhas de alocação na memória:



linhas da matriz
(ponteiros para int)

colunas da matriz
(int)

Inicializa cada
componente da matriz

```
Grafo* criarGrafo(int maxVertices, int valorarestanula) { //
    Grafo *grafo = malloc(sizeof(Grafo));

    grafo->numvertices = 0;
    grafo->maxvertices = maxVertices;
    grafo->arestanula = valorarestanula;

    grafo->vertices = malloc(maxVertices * sizeof(TipoItem));

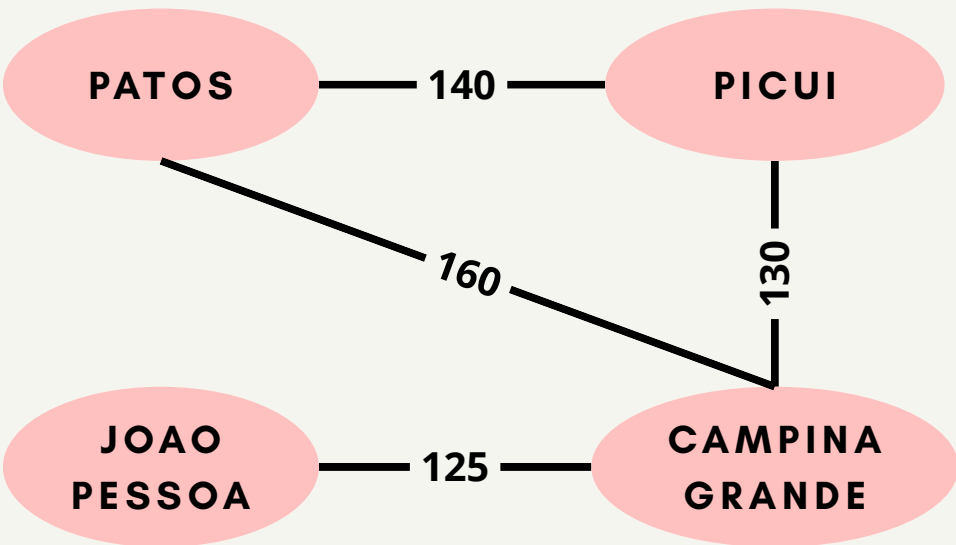
    grafo->matrizadjacencias = (int**) malloc(maxVertices * sizeof(int*));

    for (int i = 0; i < maxVertices; i++) {
        grafo->matrizadjacencias[i] = (int*) malloc(maxVertices * sizeof(int));
    }

    for (int i = 0; i < maxVertices; i++) {
        for (int j = 0; j < maxVertices; j++) {
            grafo->matrizadjacencias[i][j] = valorarestanula;
        }
    }

    return grafo;
}
```

Criar o Grafo → alocar memória e inicializar os campos



Caso usasse o calloc e valorarestanula = 0, não é necessário

Desconsiderando falhas de alocação na memória:

```
Grafo* criarGrafo(int maxVertices, int valorarestanula) { //
    Grafo *grafo = malloc(sizeof(Grafo));

    grafo->numvertices = 0;
    grafo->maxvertices = maxVertices;
    grafo->arestanula = valorarestanula;

    grafo->vertices = malloc(maxVertices * sizeof(TipoItem));
    grafo->matrizadjacencias = (int**) malloc(maxVertices * sizeof(int*));

    for (int i = 0; i < maxVertices; i++) {
        grafo->matrizadjacencias[i] = (int*) malloc(maxVertices * sizeof(int));
    }

    for (int i = 0; i < maxVertices; i++) {
        for (int j = 0; j < maxVertices; j++) {
            grafo->matrizadjacencias[i][j] = valorarestanula;
        }
    }

    return grafo;
}
```

Representação computacional do grafo – Matriz de adjacências

Obtém o índice de um vertice no vetor de vertices do grafo

Retorna o **índice** do vertice
Retorna **-1**, se não encontrar

```
int obterindice(const Grafo *grafo, const TipoItem item) {  
    if (grafo == NULL) {  
        return -1;  
    }  
  
    for (int i = 0; i < grafo->numvertices; i++) {  
        if (strcmp(item, grafo->vertices[i]) == 0) {  
            return i;  
        }  
    }  
  
    return -1;  
}
```

Representação computacional do grafo – Matriz de adjacências

Obtém o índice de um vertice no vetor de vertices do grafo

Retorna o **índice** do vertice
Retorna **-1**, se não encontrar

```
int obterindice(const Grafo *grafo, const TipoItem item) {
    if (grafo == NULL) {
        return -1;
    }

    for (int i = 0; i < grafo->numvertices; i++) {
        if (strcmp(item, grafo->vertices[i]) == 0) {
            return i;
        }
    }

    return -1;
}
```

Verifica se o grafo já atingiu a quantidade máxima de vertices

```
int estacheio(const Grafo *grafo) {
    if (grafo == NULL) {
        return 0;
    }

    return grafo->numvertices == grafo->maxvertices;
}
```

Representação computacional do grafo – Matriz de adjacências

Inserir um novo vertice no grafo

Testa se o grafo existe

Testa se o grafo está cheio

Testa se o o vértice já existe no grafo

Adiciona o vértice ao vetor de vértices

Incrementa o número de vértices

```

void inserevertice(Grafo *grafo, const TipoItem item) {
    if (grafo == NULL) {
        return;
    }

    if (estacheio(grafo)) {
        printf("O numero maximo de vertices foi alcançado!\n");
        return;
    }

    if (obterindice(grafo, item) != -1) {
        printf("Esse vertice ja existe no grafo!\n");
        return;
    }

    strcpy(grafo->vertices[grafo->numvertices], item);
    grafo->numvertices++;
}

```

Inserir um nova aresta no grafo

Preenche a posição na matriz com o **peso** da aresta

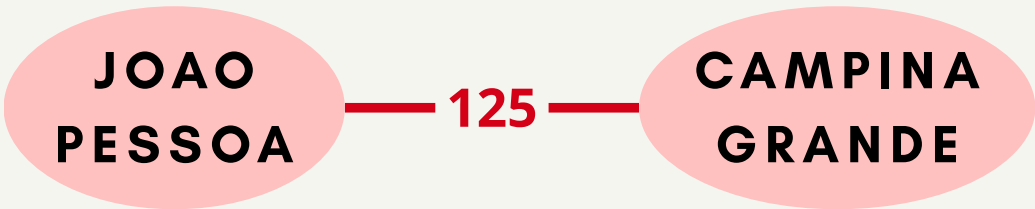
JOAO
PESSOA

CAMPINA
GRANDE

	JP	CG	Patos	Picui
JP	0	0	0	0
CG	0	0	0	0
Patos	0	0	0	0
Picui	0	0	0	0

Inserir um nova aresta no grafo

Preenche a posição na matriz com o **peso** da aresta



	JP	CG	Patos	Picui
JP	0	125	0	0
CG	125	0	0	0
Patos	0	0	0	0
Picui	0	0	0	0

Inserir um nova aresta no grafo

Preenche a posição na matriz com o **peso** da aresta

obtém o índice de inserção na matriz

Exemplo: se arestanula = 0, então uma aresta não pode ter peso 0.

Grafo não direcionado: $a_{ij} = a_{ji}$

```

void inserearesta(Grafo *grafo, const TipoItem NoSaida, const TipoItem NoEntrada, int peso) {
    if (grafo == NULL) {
        return;
    }

    int linha = obterindice(grafo, NoSaida);
    int coluna = obterindice(grafo, NoEntrada);

    if (linha == -1 || coluna == -1) {
        printf("Um ou ambos os vertices nao existem no grafo!\n");
        return;
    }

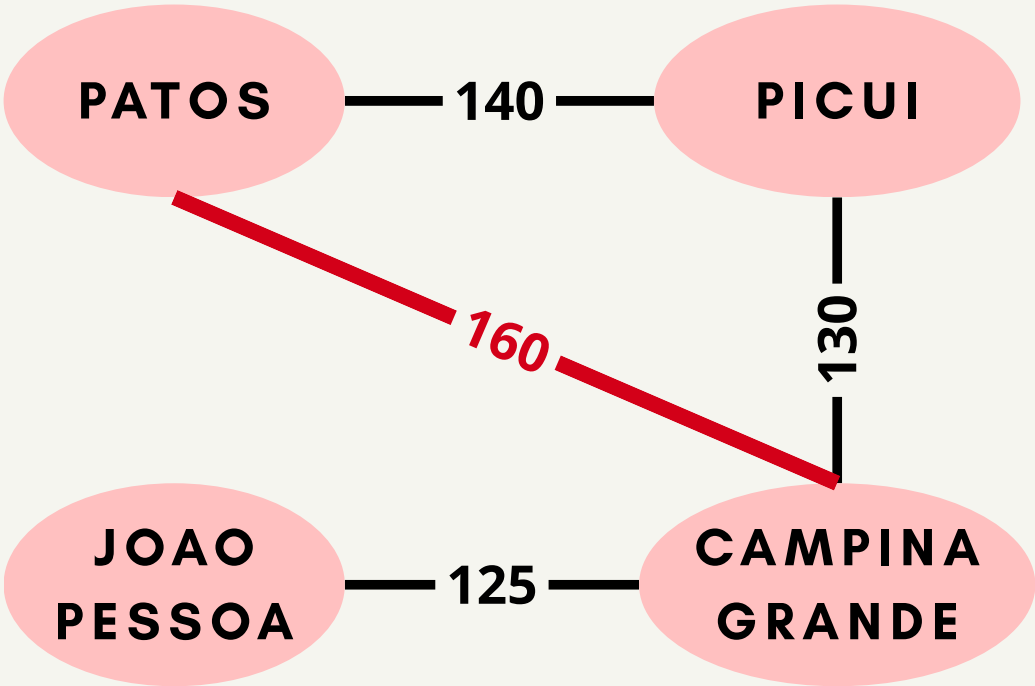
    if (linha == coluna) {
        printf("Grafo simples nao permite laco!\n");
        return;
    }

    if (peso == grafo->arestanula) {
        printf("0 peso nao pode ser igual ao valor que representa ausencia de aresta!\n");
        return;
    }

    grafo->matrizadjacencias[linha][coluna] = peso;
    grafo->matrizadjacencias[coluna][linha] = peso;
}
    
```

Remover uma aresta

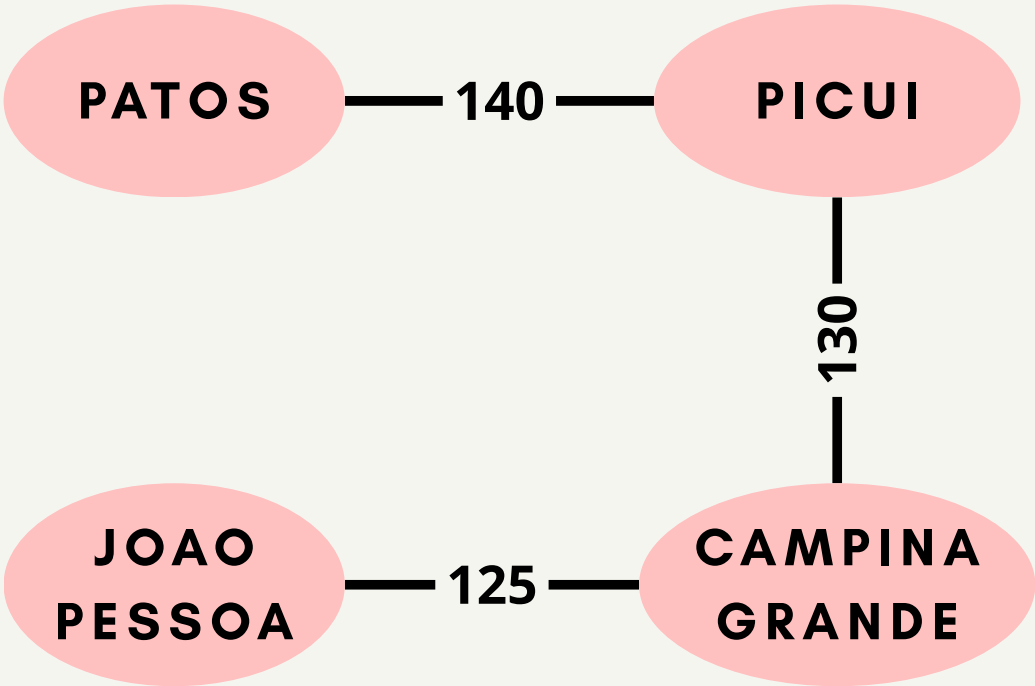
Preenche a posição na matriz com **arestaNula**



	JP	CG	Patos	Picui
JP	0	125	0	0
CG	125	0	160	130
Patos	0	160	0	140
Picui	0	130	140	0

Remover uma aresta

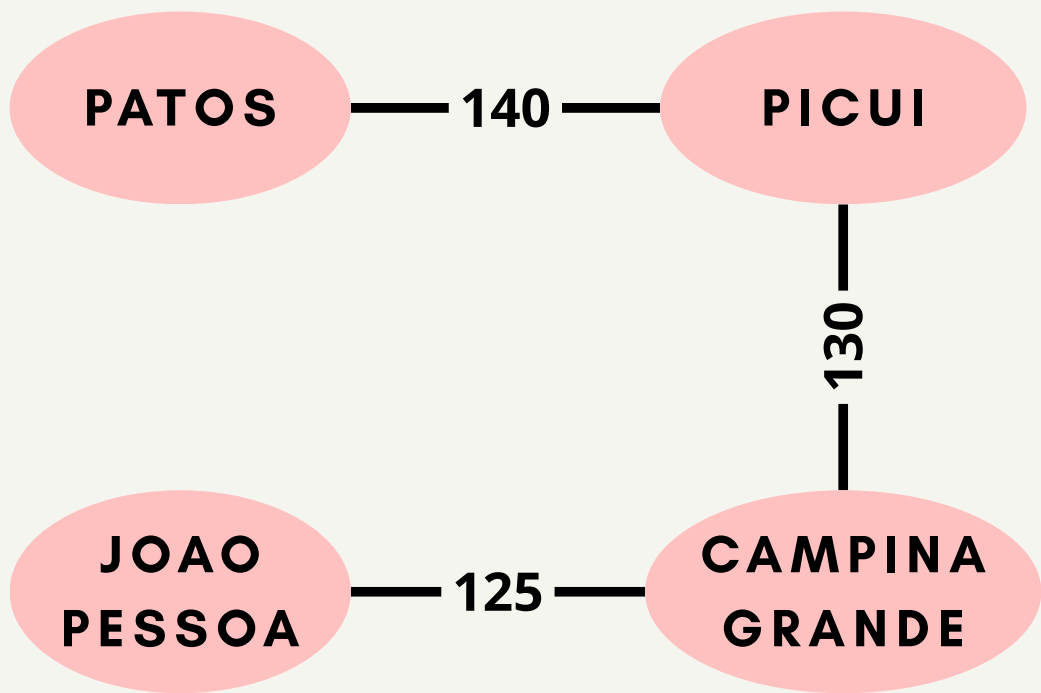
Preenche a posição na matriz com **arestaNula**



	JP	CG	Patos	Picui
JP	0	125	0	0
CG	125	0	0	130
Patos	0	0	0	140
Picui	0	130	140	0

Remover uma aresta

Preenche a posição na matriz com **arestaNula**



	JP	CG	Patos	Picui
JP	0	125	0	0
CG	125	0	0	130
Patos	0	0	0	140
Picui	0	130	140	0

```
void removerAresta(Grafo *grafo, const TipoItem NoSaida, const TipoItem NoEntrada) {  
    if (grafo == NULL) {  
        return;  
    }  
  
    int linha = obterindice(grafo, NoSaida);  
    int coluna = obterindice(grafo, NoEntrada);  
  
    if (linha == -1 || coluna == -1) {  
        printf("Um ou ambos os vertices nao existem no grafo!\n");  
        return;  
    }  
  
    grafo->matrizadjacencias[linha][coluna] = grafo->arestanula;  
    grafo->matrizadjacencias[coluna][linha] = grafo->arestanula;  
}
```

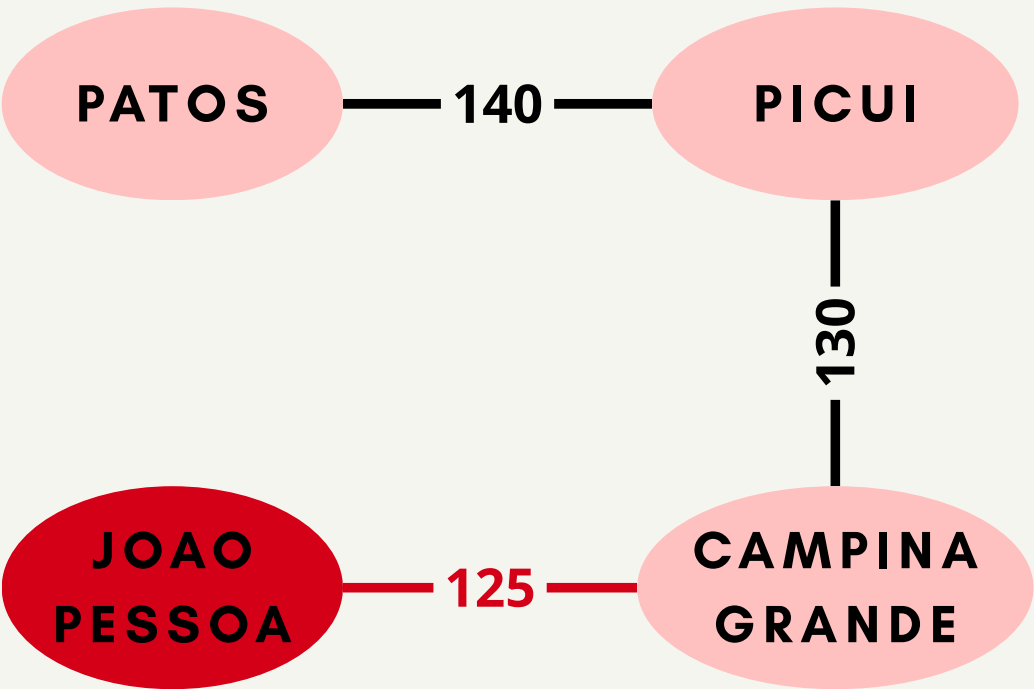
→ **obtem os indices de remoção na matriz**

Remover um vértice

1. Remove o vertice do **vetor de vertices**, e **decrementa** a quantidade de vértices
2. Preencher com **arestaNula** a **linha** e a **coluna** correspondente ao **vértice**.

Remover um vértice

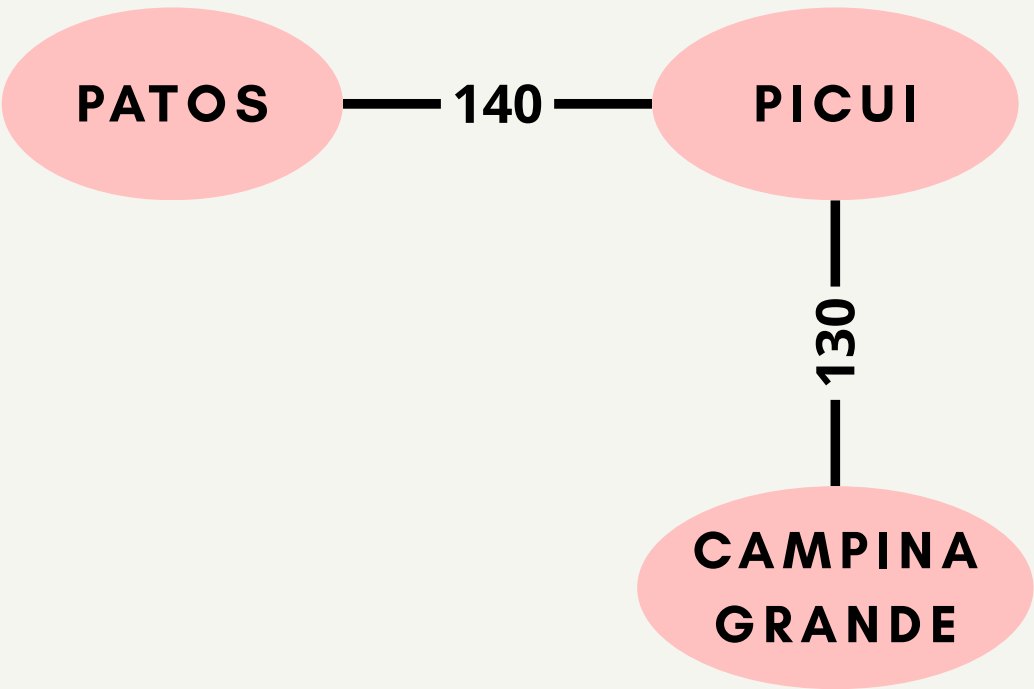
1. Remove o vertice do **vetor de vertices**, e **decrementa** a quantidade de vértices
2. Preencher com **arestaNula** a **linha** e a **coluna** correspondente ao **vértice**.



	JP	CG	Patos	Picui
JP	0	125	0	0
CG	125	0	0	130
Patos	0	0	0	140
Picui	0	130	140	0

Remover um vértice

1. Remove o vertice do **vetor de vertices**, e **decrementa** a quantidade de vértices
2. Preencher com **arestaNula** a **linha** e a **coluna** correspondente ao **vértice**.



	JP	CG	Patos	Picui
JP	0	0	0	0
CG	0	0	0	130
Patos	0	0	0	140
Picui	0	130	140	0

Remover um vértice

1. Remove o vertice do **vetor de vertices**, e **decrementa** a quantidade de vértices
2. Preencher com **arestaNula** a **linha** e a **coluna** correspondente ao **vértice**.

```
void removerVertice(Grafo *grafo, const TipoItem item) {
    if (grafo == NULL) {
        return;
    }

    int indice = obterindice(grafo, item);

    if (indice == -1) {
        printf("Vertice nao encontrado!\n");
        return;
    }

    for (int i = indice; i < grafo->numvertices - 1; i++) {
        strcpy(grafo->vertices[i], grafo->vertices[i + 1]);
    }
    grafo->numvertices--;

    for (int i = 0; i < grafo->numvertices; i++) {
        grafo->matrizadjacencias[indice][i] = grafo->arestanula;
        grafo->matrizadjacencias[i][indice] = grafo->arestanula;
    }
}
```

Remove o vertice do vetor de vertices e desloca os vértices seguintes a ele

Remove as arestas associadas ao vertice removido, preenchendo a linha e coluna correspondentes com arestaNula

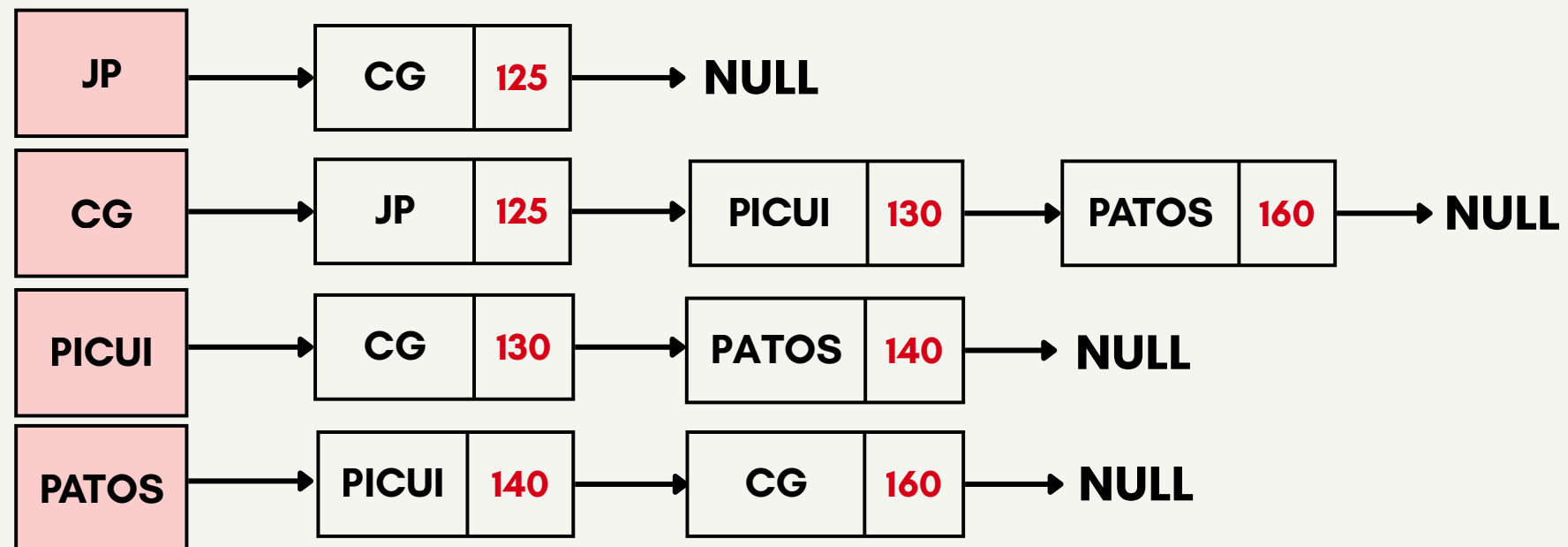
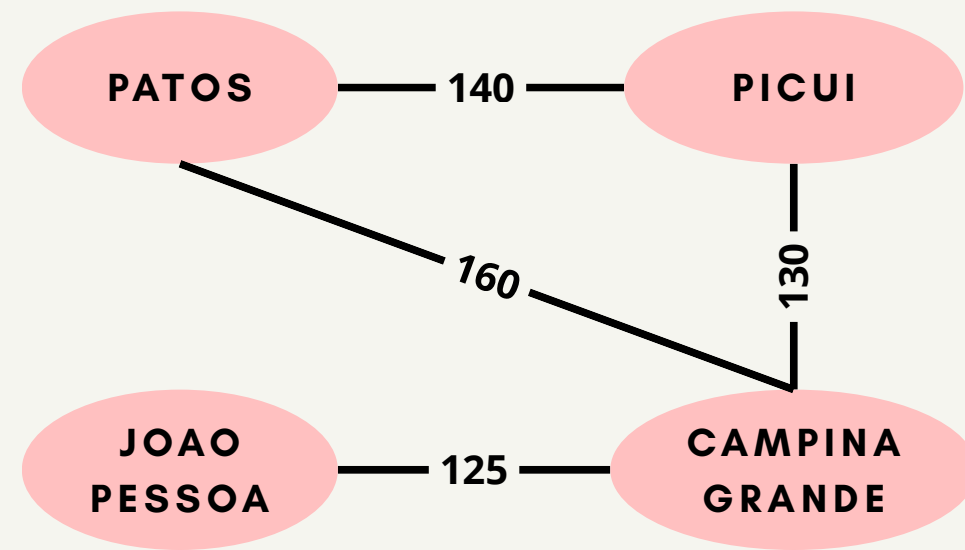
Imprimir a Matriz de adjacências

```
void imprimirmatriz(const Grafo *grafo) {  
    if (grafo == NULL) {  
        return;  
    }  
  
    printf("Matriz de adjacencias:\n");  
  
    for (int i = 0; i < grafo->numvertices; i++) {  
        for (int j = 0; j < grafo->numvertices; j++) {  
            printf("%d ", grafo->matrizadjacencias[i][j]);  
        }  
        printf("\n");  
    }  
}
```

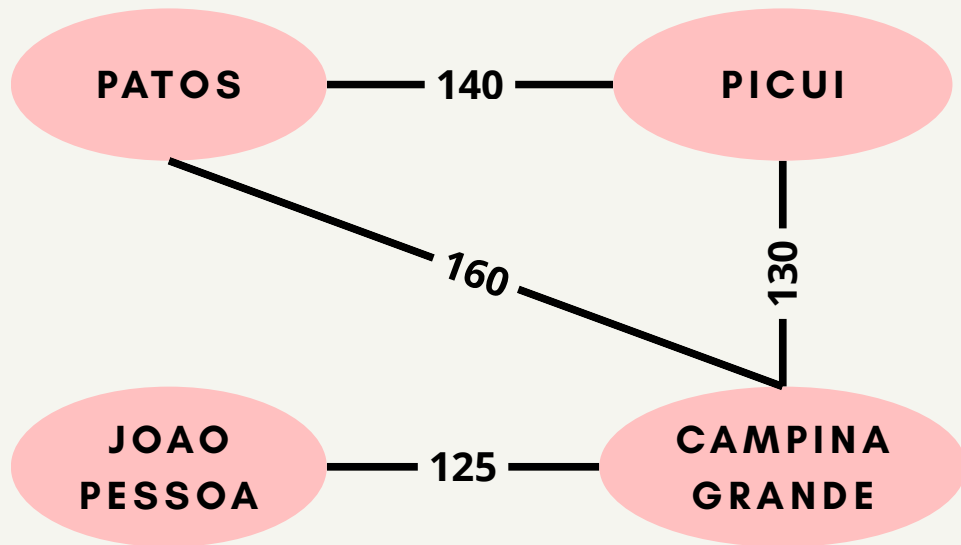
Imprimir a Matriz de adjacências

```
void imprimirvertices(const Grafo *grafo) {  
    if (grafo == NULL) {  
        return;  
    }  
  
    printf("Lista de vertices:\n");  
  
    for (int i = 0; i < grafo->numvertices; i++) {  
        printf("%d: %s\n", i, grafo->vertices[i]);  
    }  
}
```

Representação computacional do grafo – Lista de adjacências

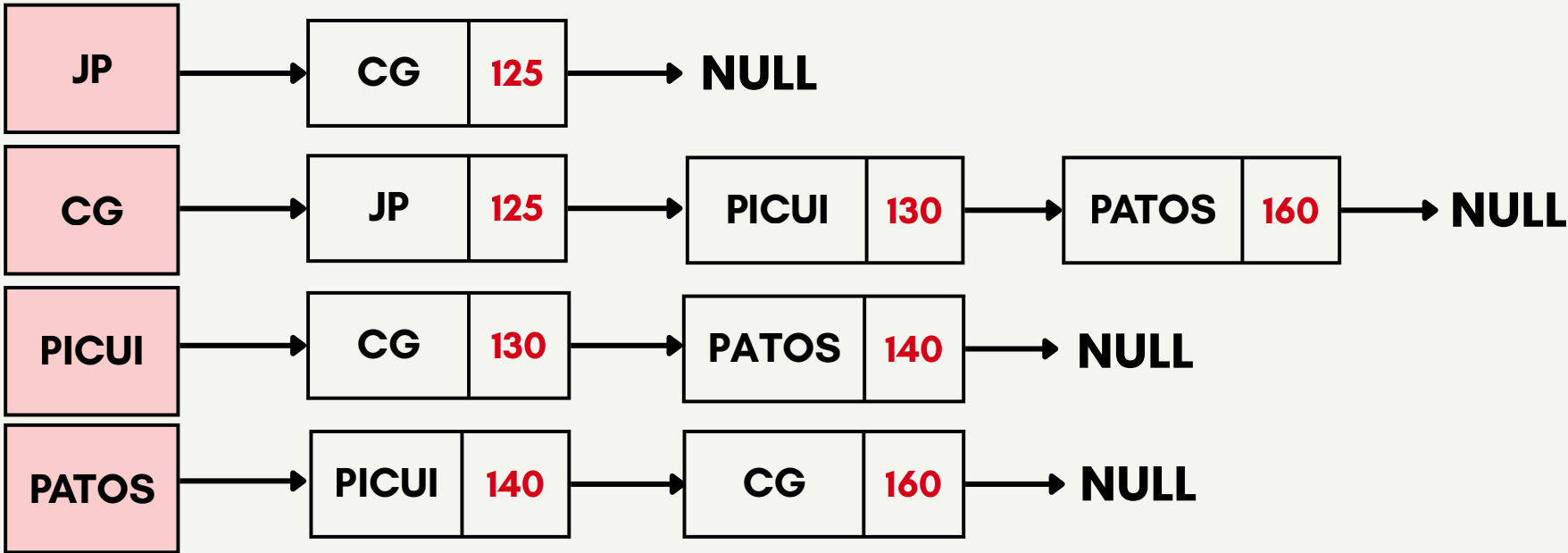


Representação computacional do grafo - Lista de adjacências

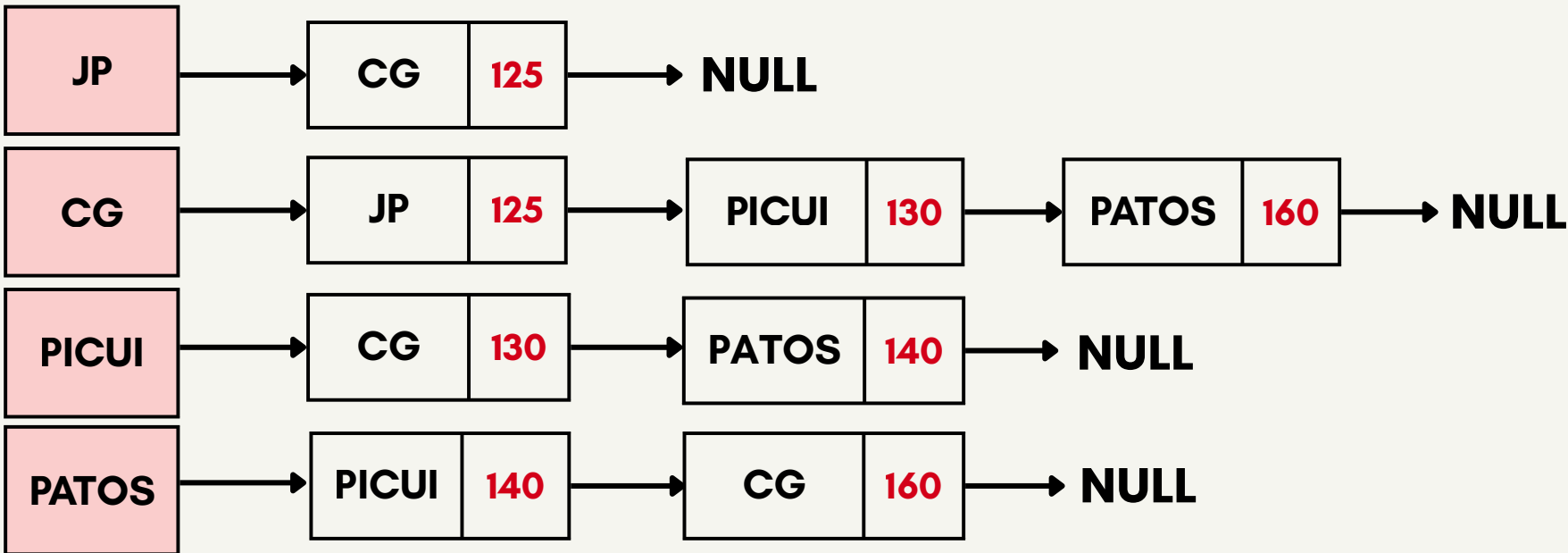
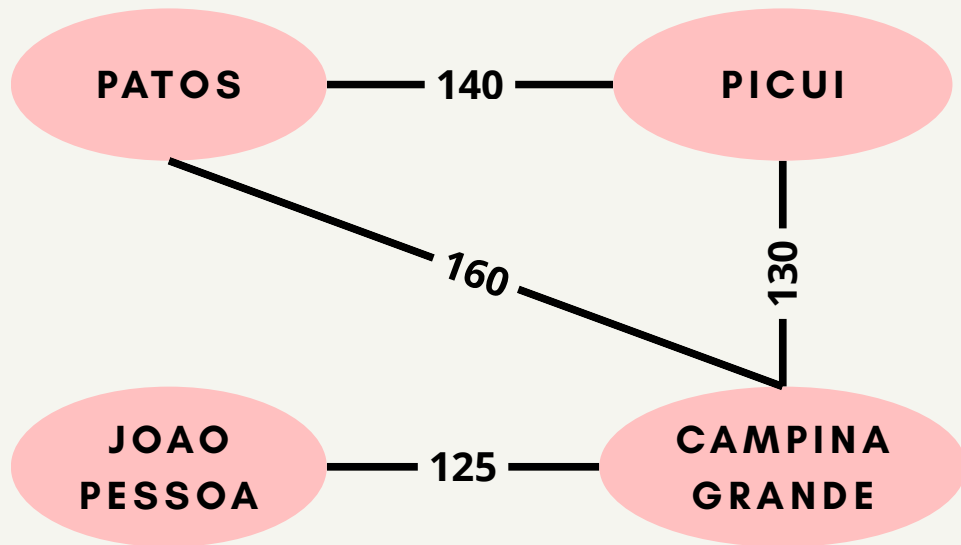


```
typedef struct No {
    int vertice;
    int peso;
    struct No *proximo;
} No;

typedef struct Lista {
    No *inicio;
} Lista;
```



Representação computacional do grafo - Lista de adjacências



```
typedef struct No {
    int vertice;
    int peso;
    struct No *proximo;
} No;

typedef struct Lista {
    No *inicio;
} Lista;

typedef struct Grafo {
    int numvertices;
    int arestanula;
    Lista *listasadjacencias;
} Grafo;
```

Vetor (dinâmico) de listas de adjacências
Cada posição do vetor é um vértice

Matriz de adjacências: Acessa diretamente o peso da aresta entre dois vértices com **`matrizadjacencias[i][j]`**.

Lista de adjacências: Percorre a lista de vizinhos de um vértice para acessar as arestas.

Matriz de adjacências: Acessa diretamente o peso da aresta entre dois vértices com **matrizadjacencias[i][j]**.

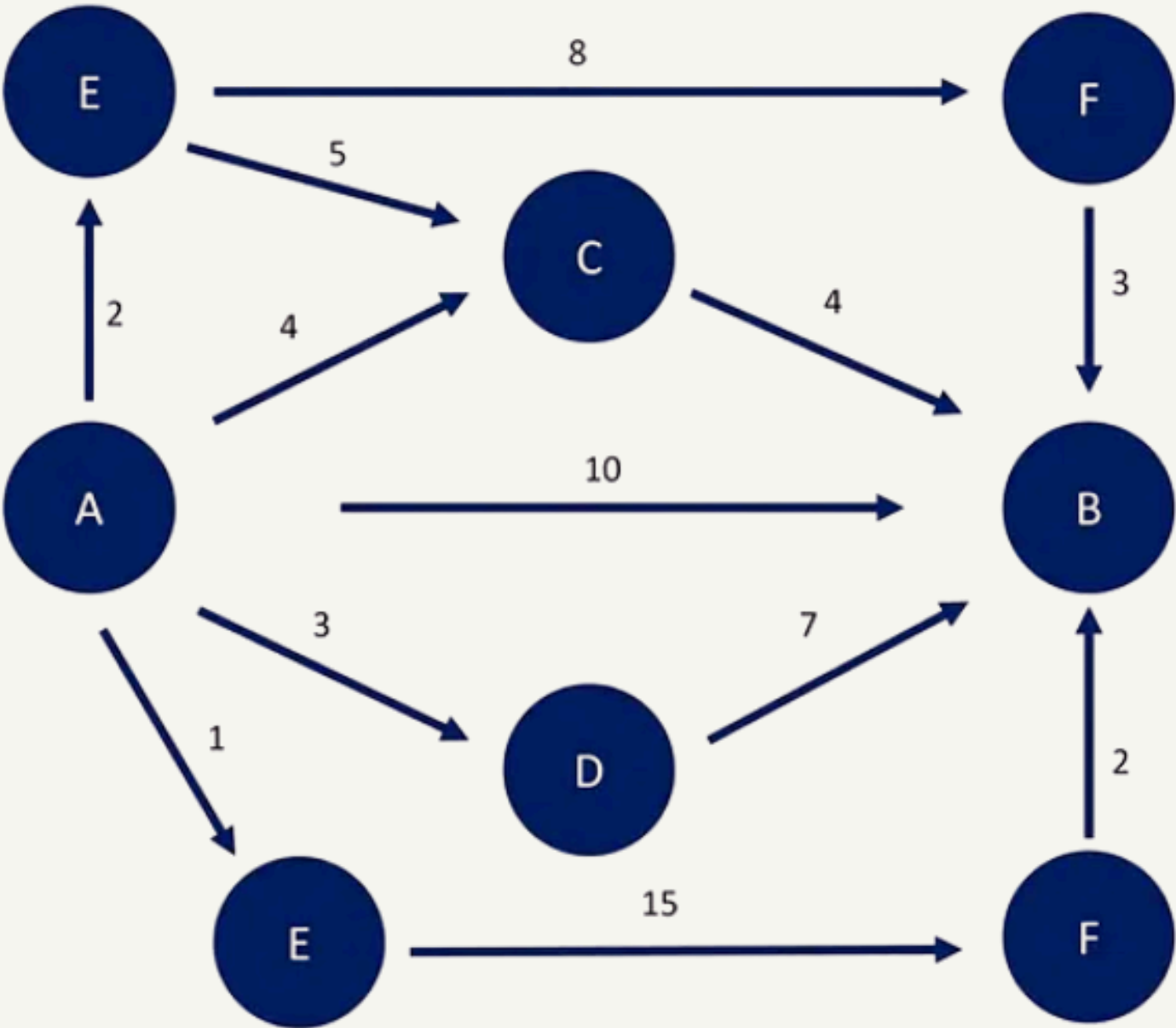
Lista de adjacências: Percorre a lista de vizinhos de um vértice para acessar as arestas.

	Matriz de Adjacências	Lista de Adjacências
Tipo de grafo	✔ Grafos densos (muitas arestas)	✔ Grafos esparsos (poucas arestas)
Acessibilidade (arestas)	✔ Acesso direto	⚠ Precisa percorrer a lista
Gasto de memória	⚠ Gasta mais memória	✔ Economiza memória
Implementação	✔ Mais simples	⚠ Mais trabalhosa



Métodos de Busca em Grafos

Têm como objetivo encontrar a melhor forma de ir de um nó a outro dentro do grafo.



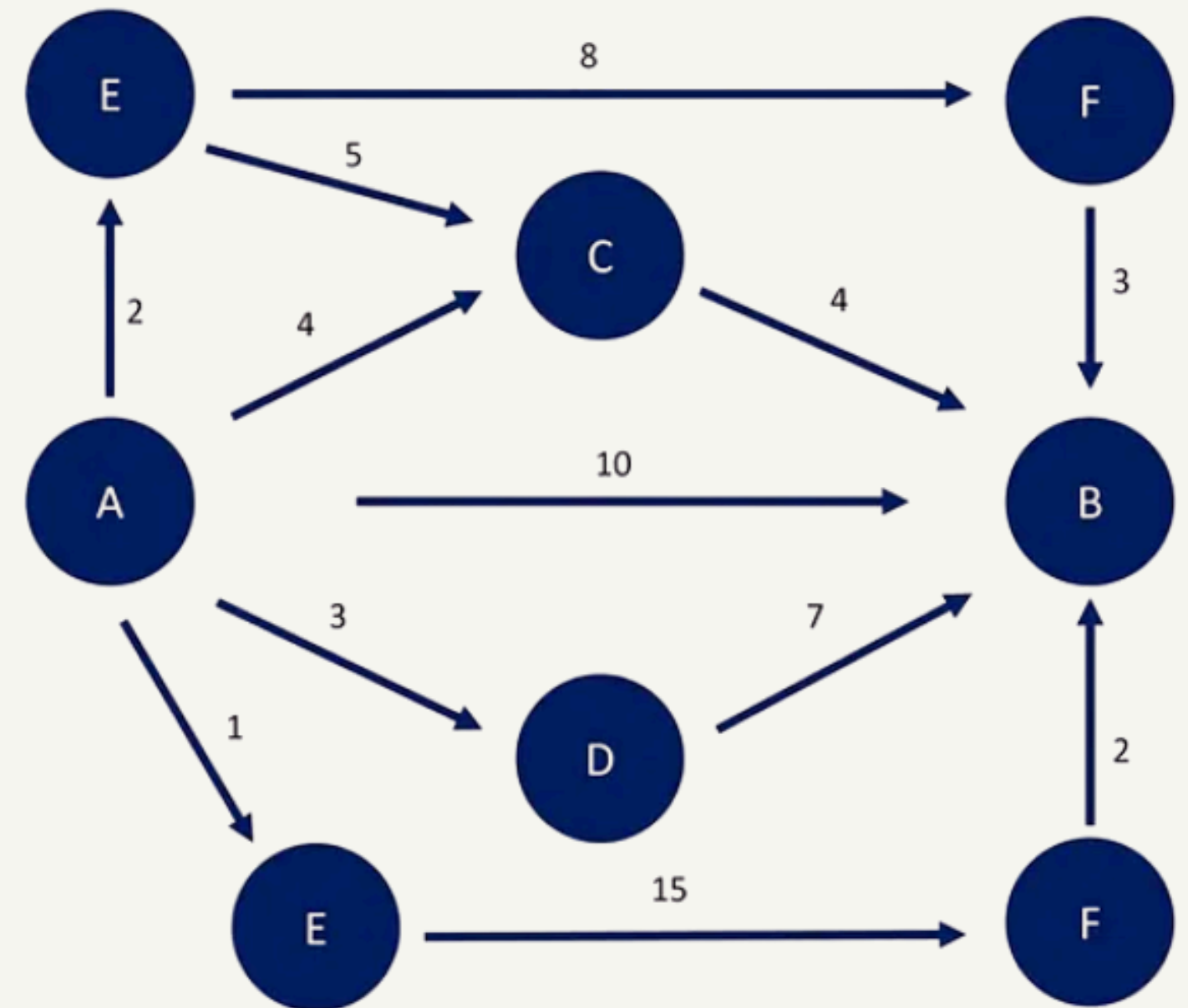
Métodos de Busca em Grafos

Têm como objetivo encontrar a melhor forma de ir de um nó a outro dentro do grafo.

Exemplo:

Quanto tempo demora para chegar meu iFood?

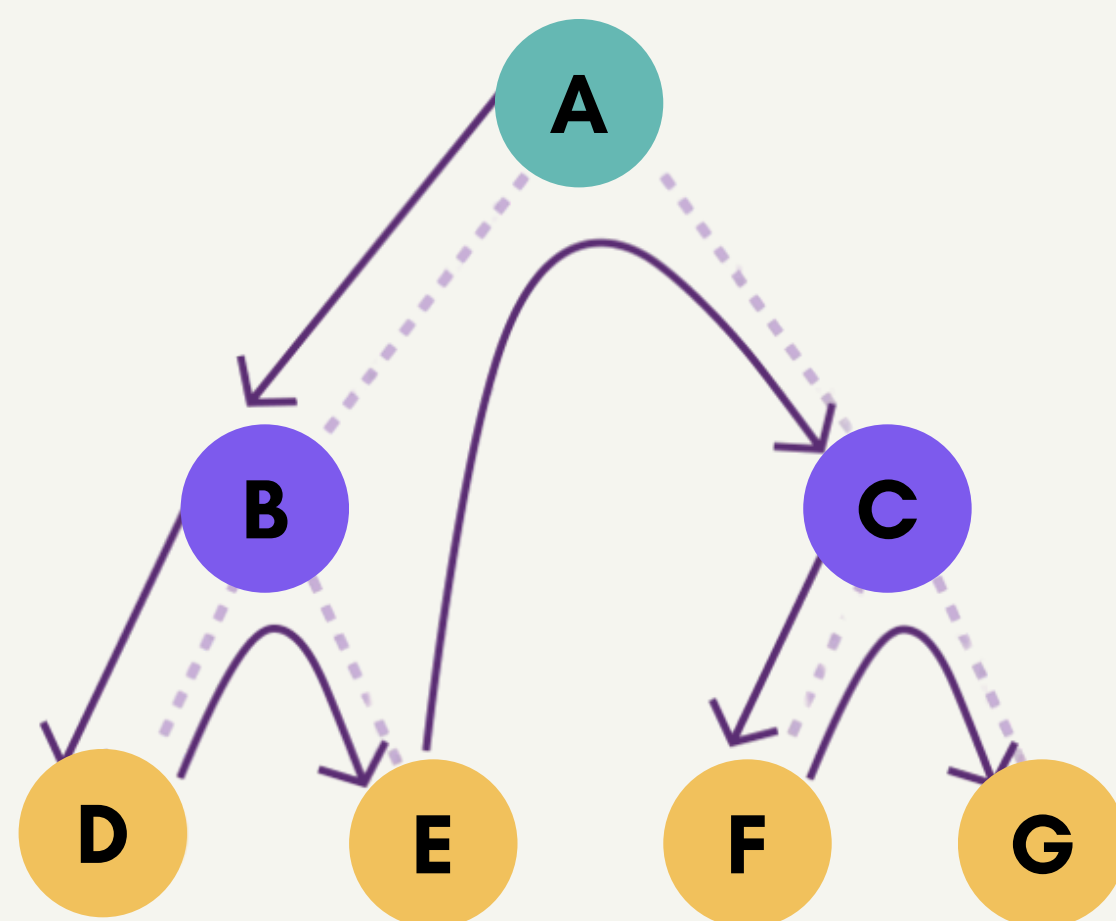
Qual a melhor rota para a minha viagem?



Métodos de Busca em Grafos

Busca em profundidade Depth First Search (DFS)

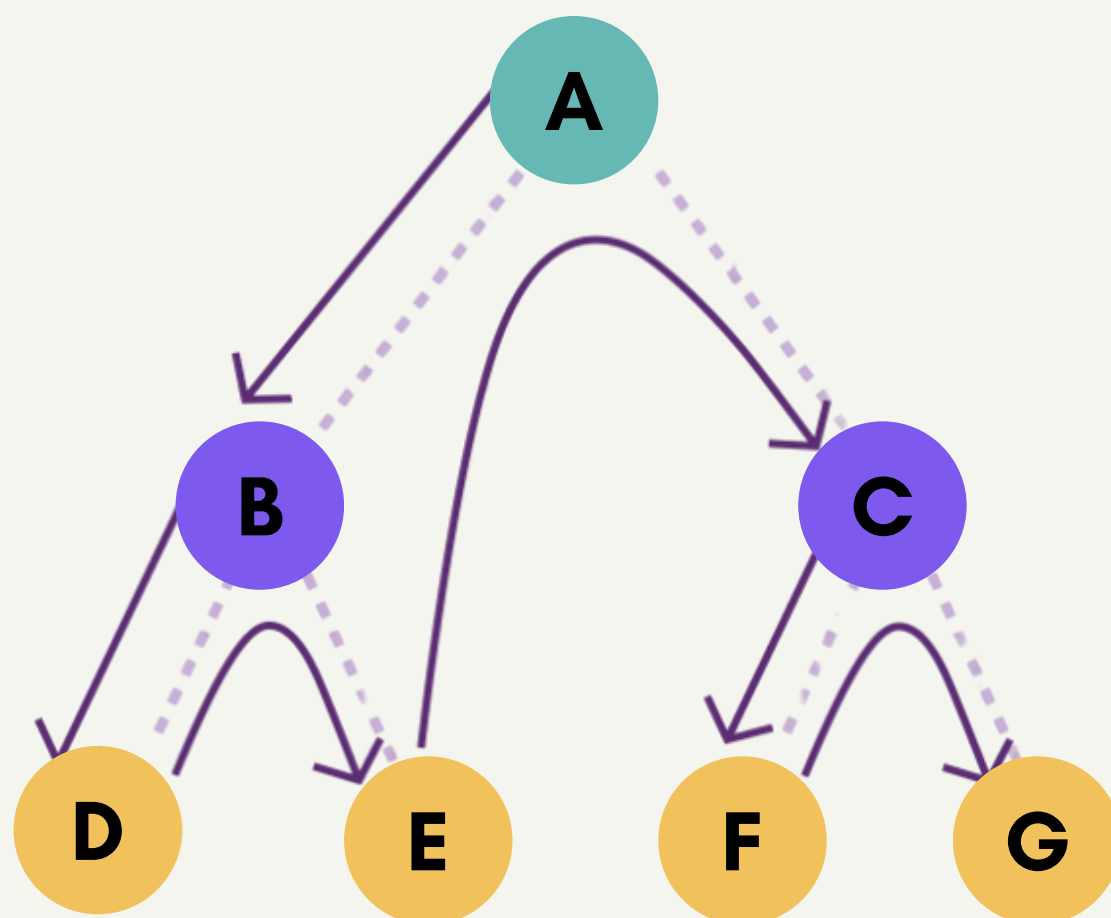
1. **Vou até a maior profundidade possível**
2. **Volto pro último nó filho não explorado e vou para a maior profundidade possível**



Métodos de Busca em Grafos

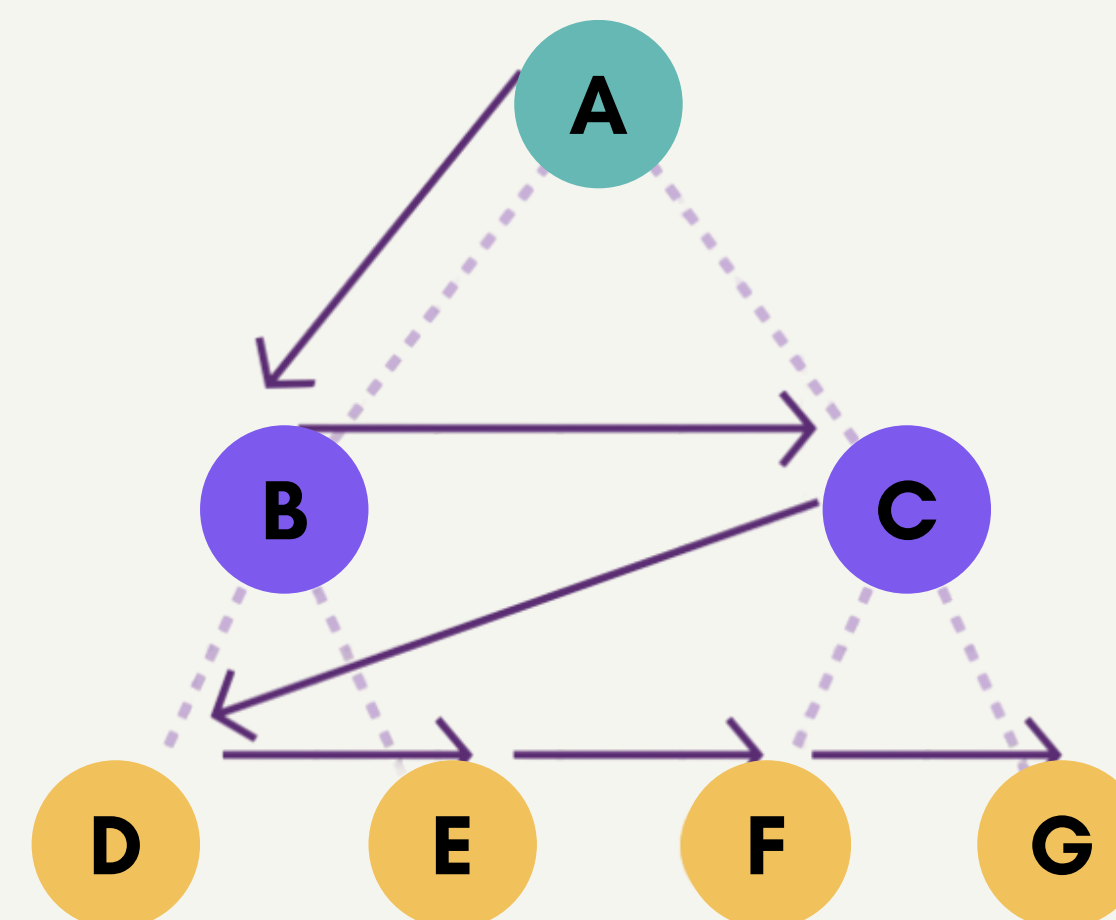
Busca em profundidade Depth First Search (DFS)

1. Vou até a maior profundidade possível
2. Volto pro último nó filho não explorado e vou para a maior profundidade possível



Busca em Largura Breadth-First Search (BFS)

Busco em cada profundidade por vez

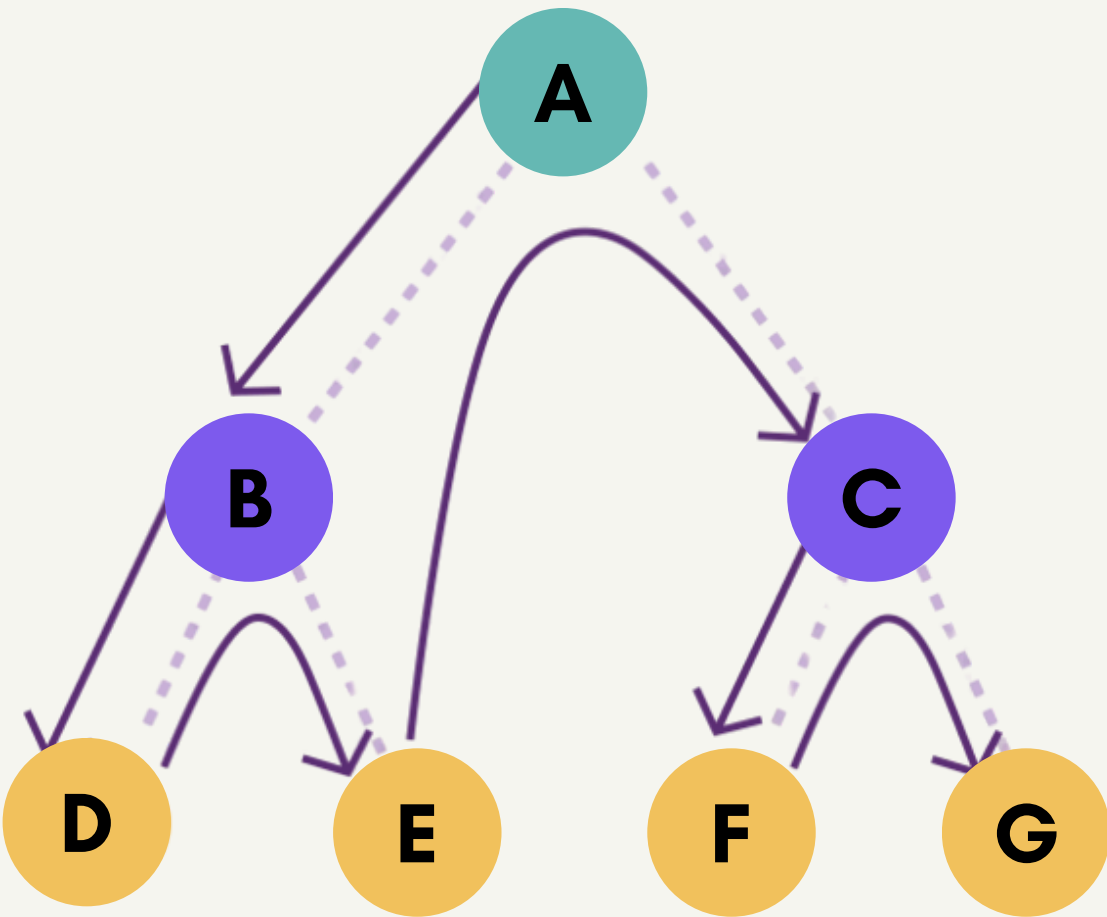


Métodos de Busca em Grafos

Busca em profundidade

Depth First Search (DFS)

- 1. Vou até a maior profundidade possível
- 2. Volto pro último nó filho não explorado e vou para a maior profundidade possível



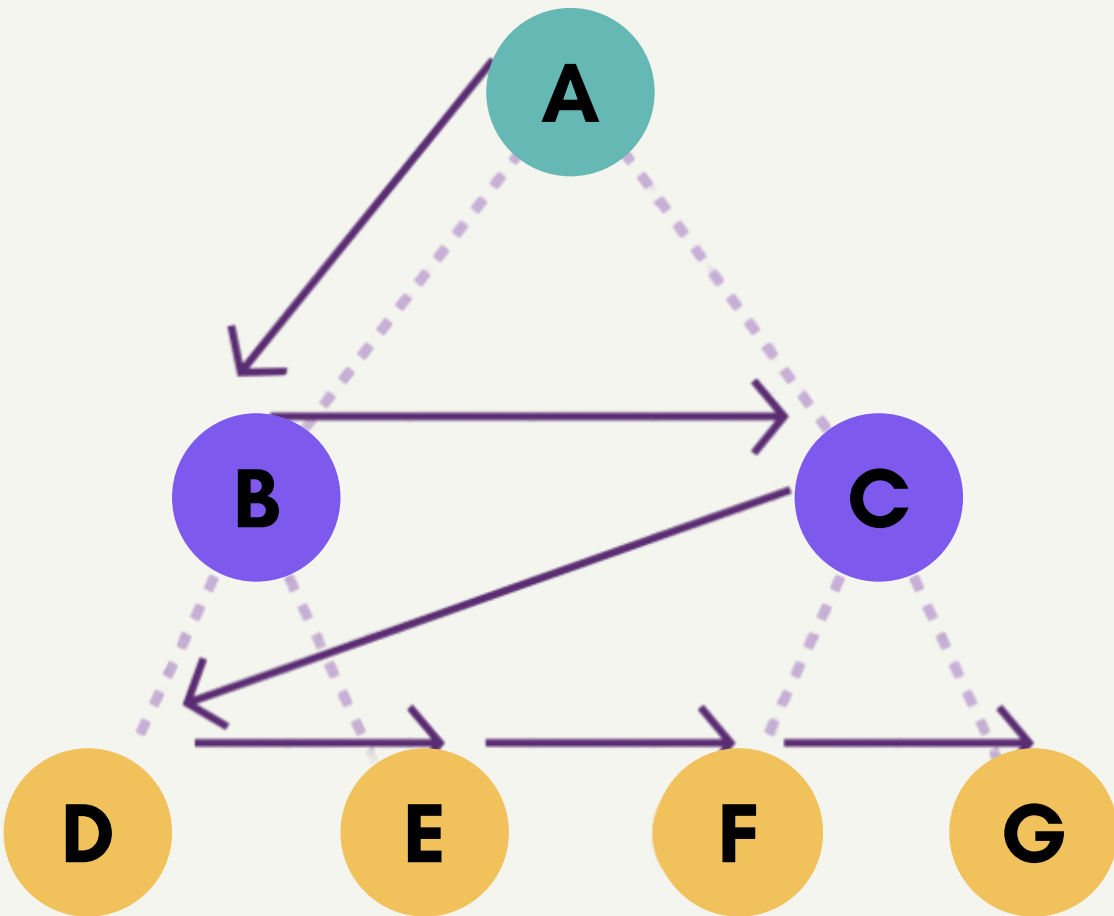
Utilizo apenas caminhos existentes.

Percursos Pós-ordem, em ordem, pré-ordem

Busca em Largura

Breadth-First Search (BFS)

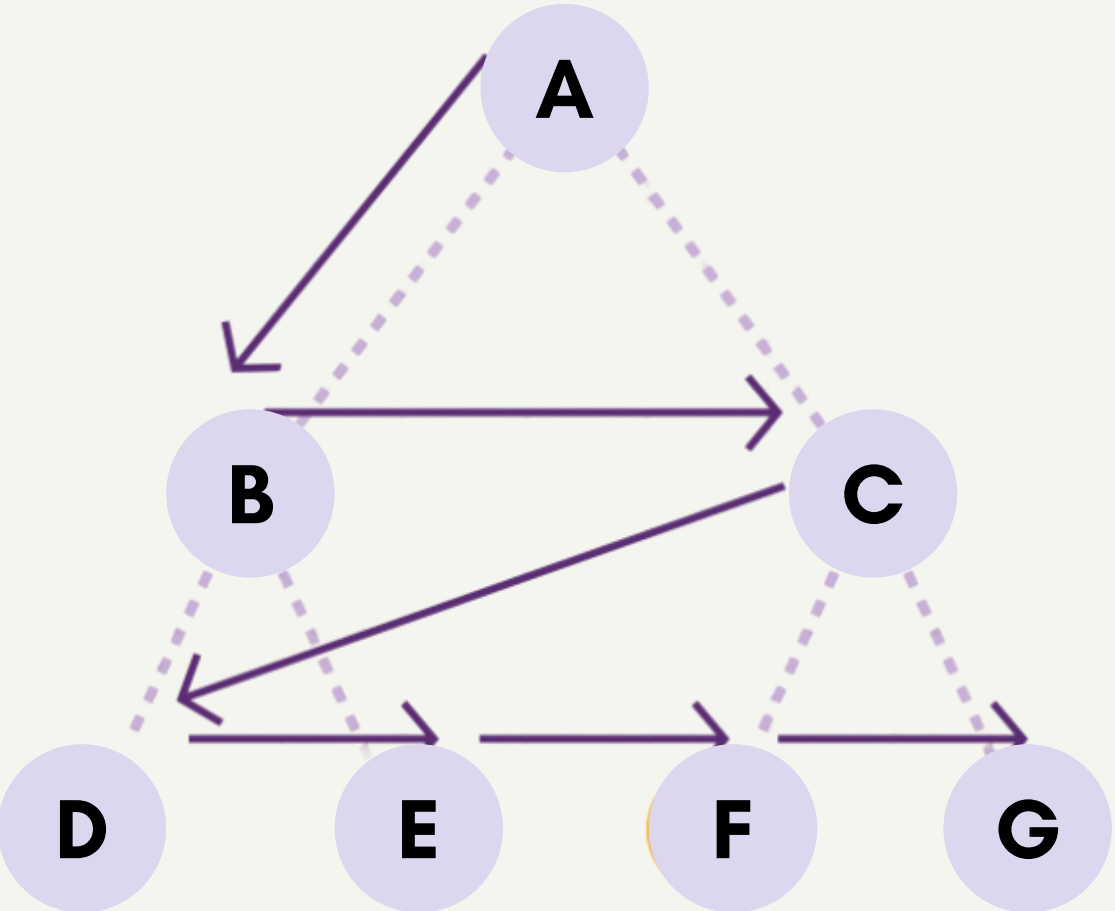
Busco em cada profundidade por vez



Não necessariamente são caminhos existentes

Busca em Largura (BFS)

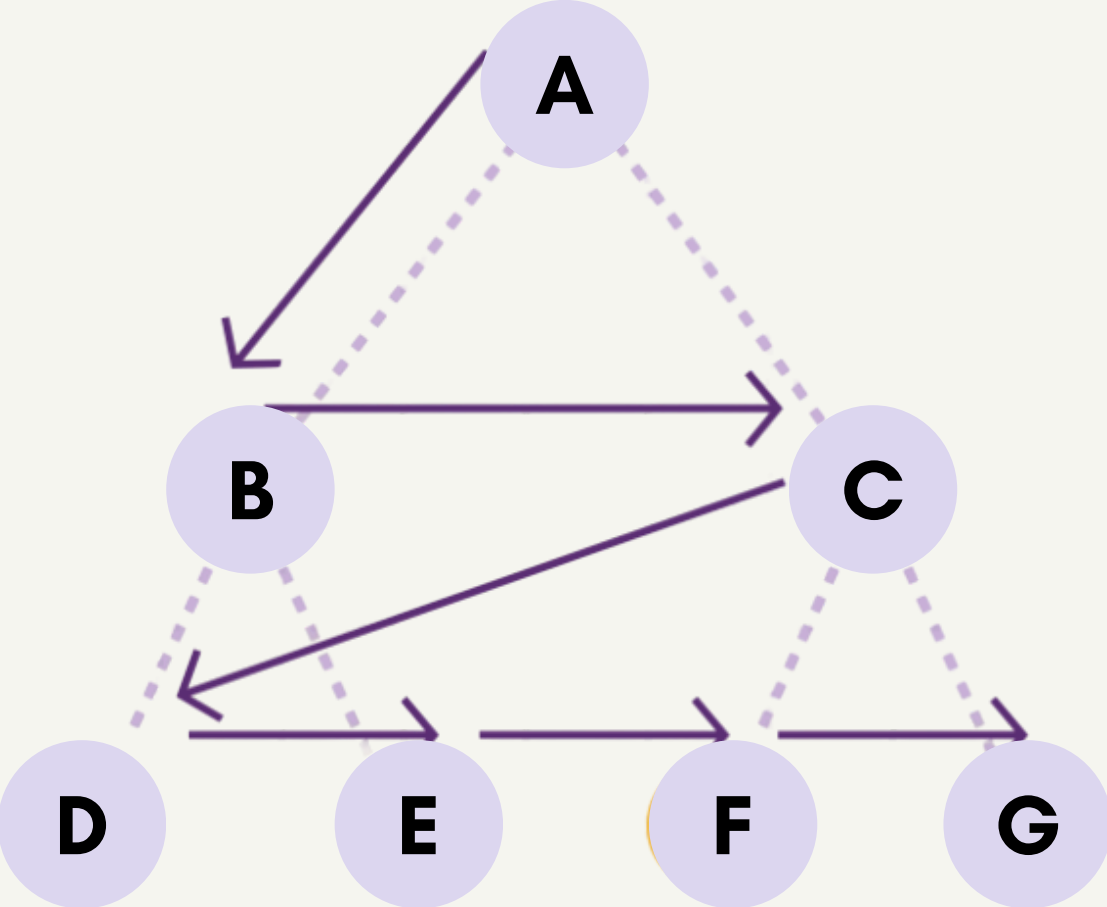
Estrutura auxiliar: FILA



Busca em Largura (BFS)

Estrutura auxiliar: FILA

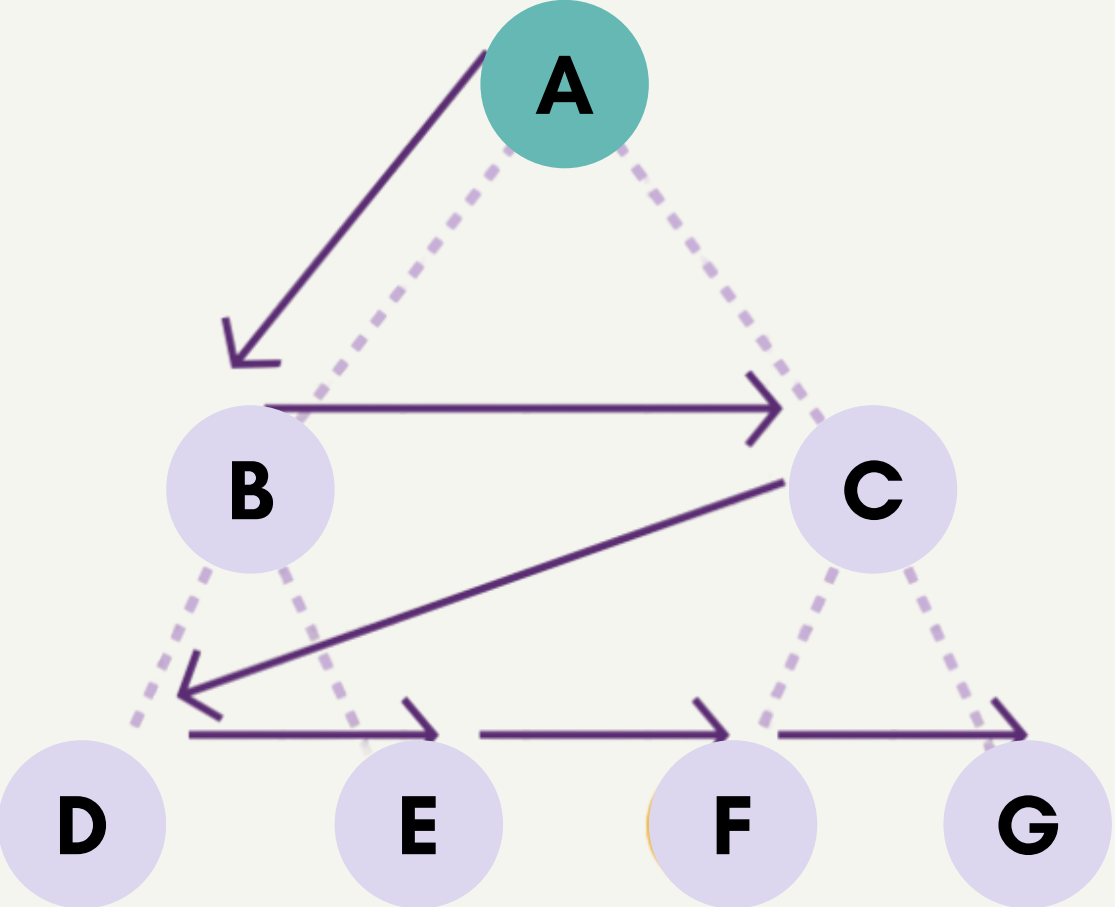
Vamos buscar o nó E



Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



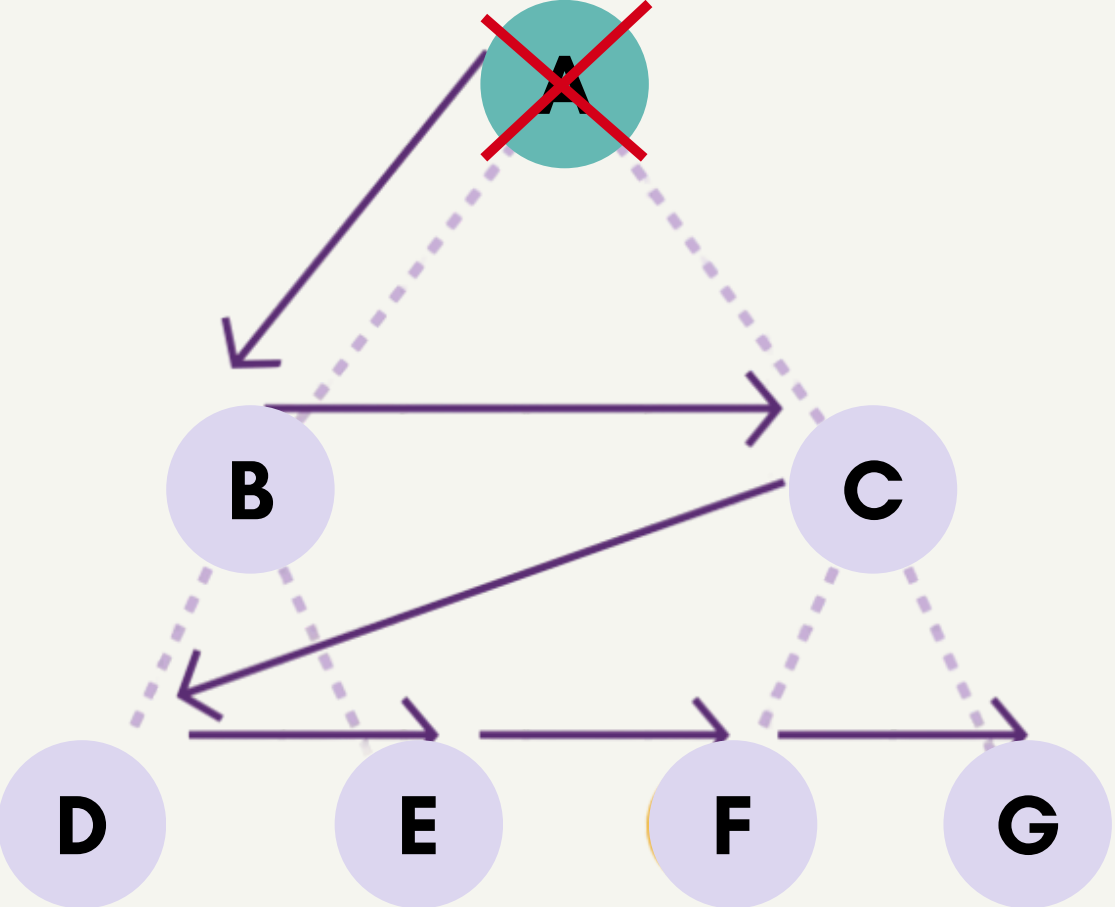
FILA:

A

Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



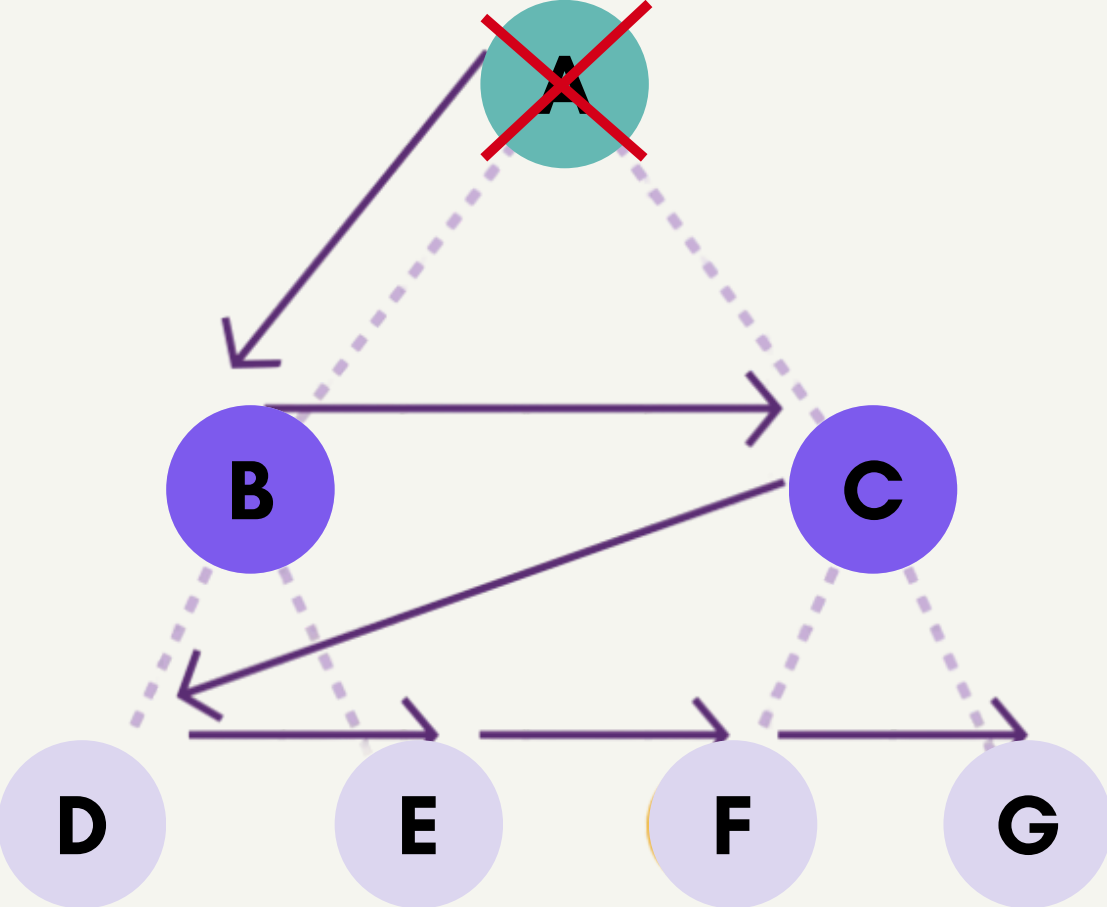
FILA:

~~A~~

Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



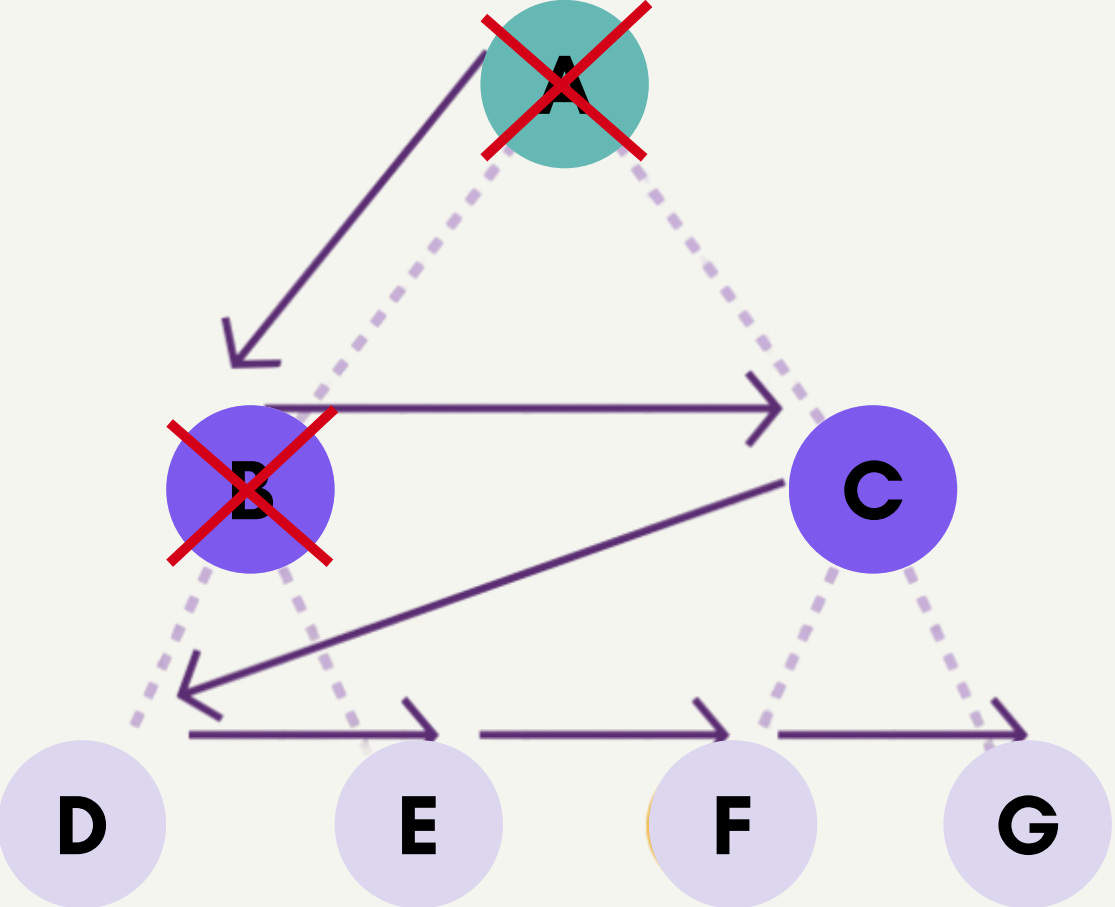
FILA:

C → B → ~~A~~

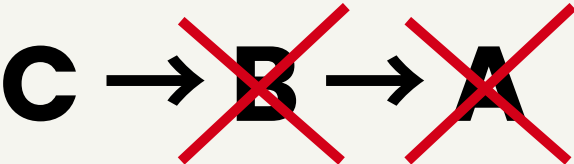
Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



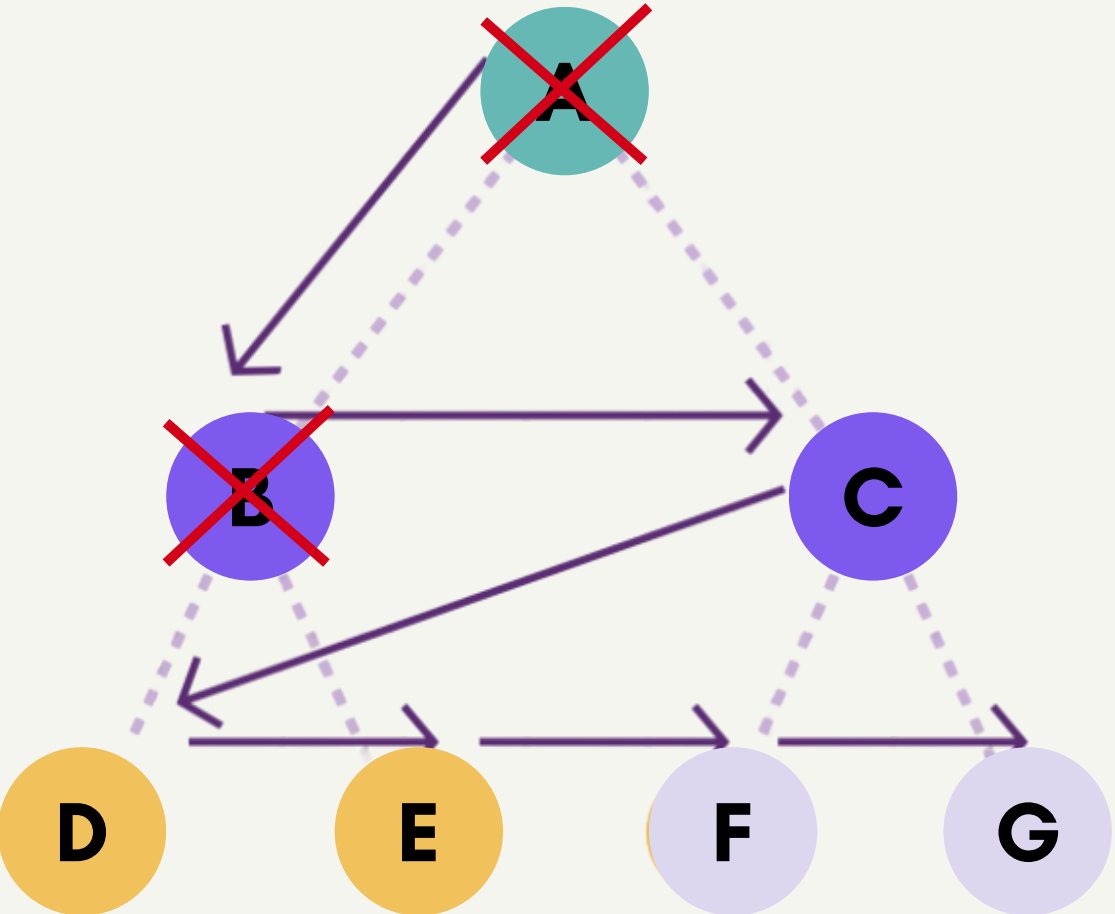
FILA:



Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



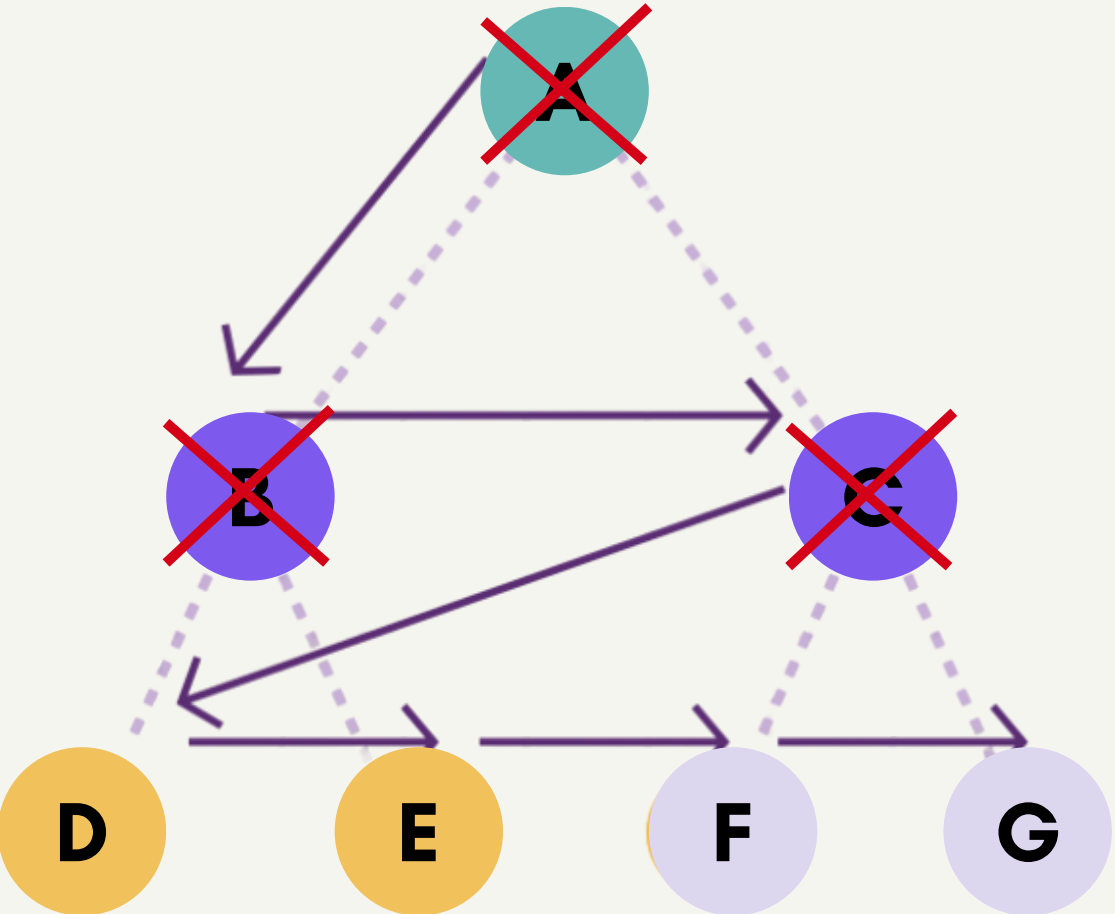
FILA:

E → D → C → ~~B~~ → ~~A~~

Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



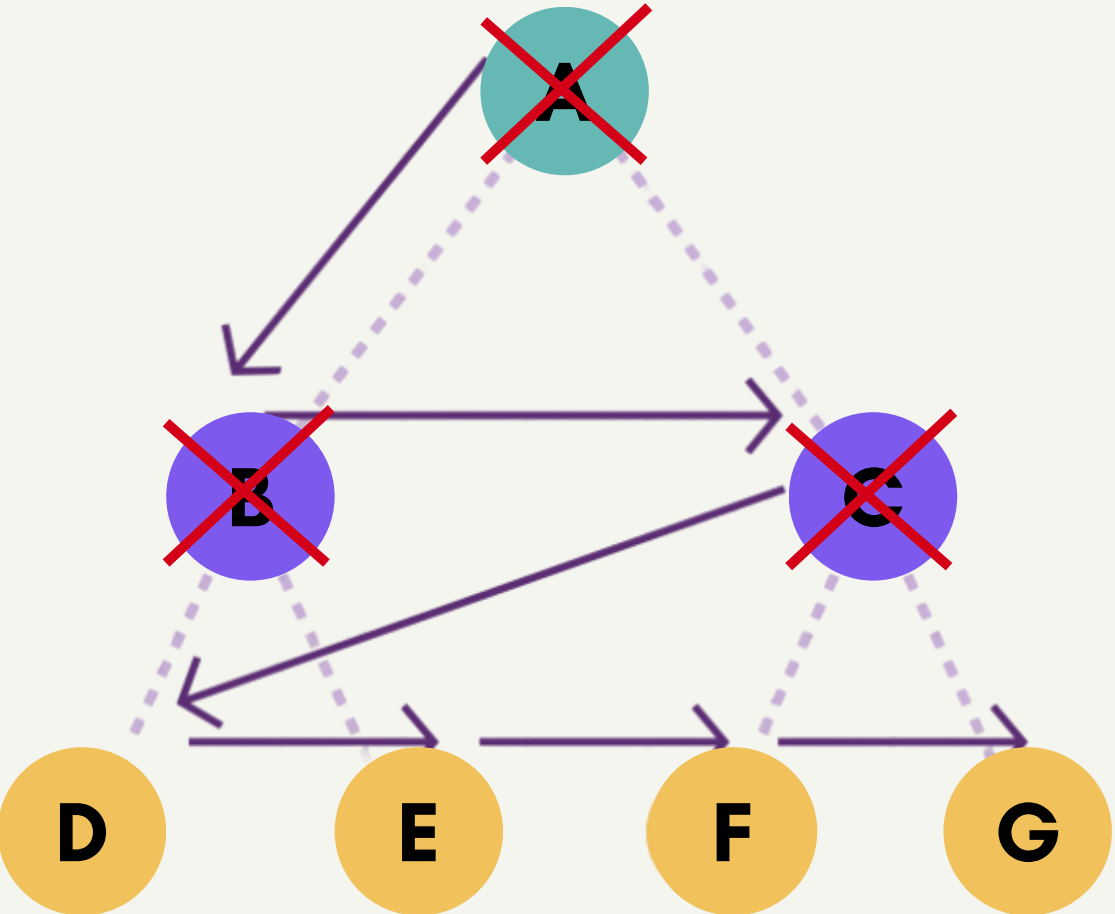
FILA:

$E \rightarrow D \rightarrow \cancel{C} \rightarrow \cancel{B} \rightarrow \cancel{A}$

Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



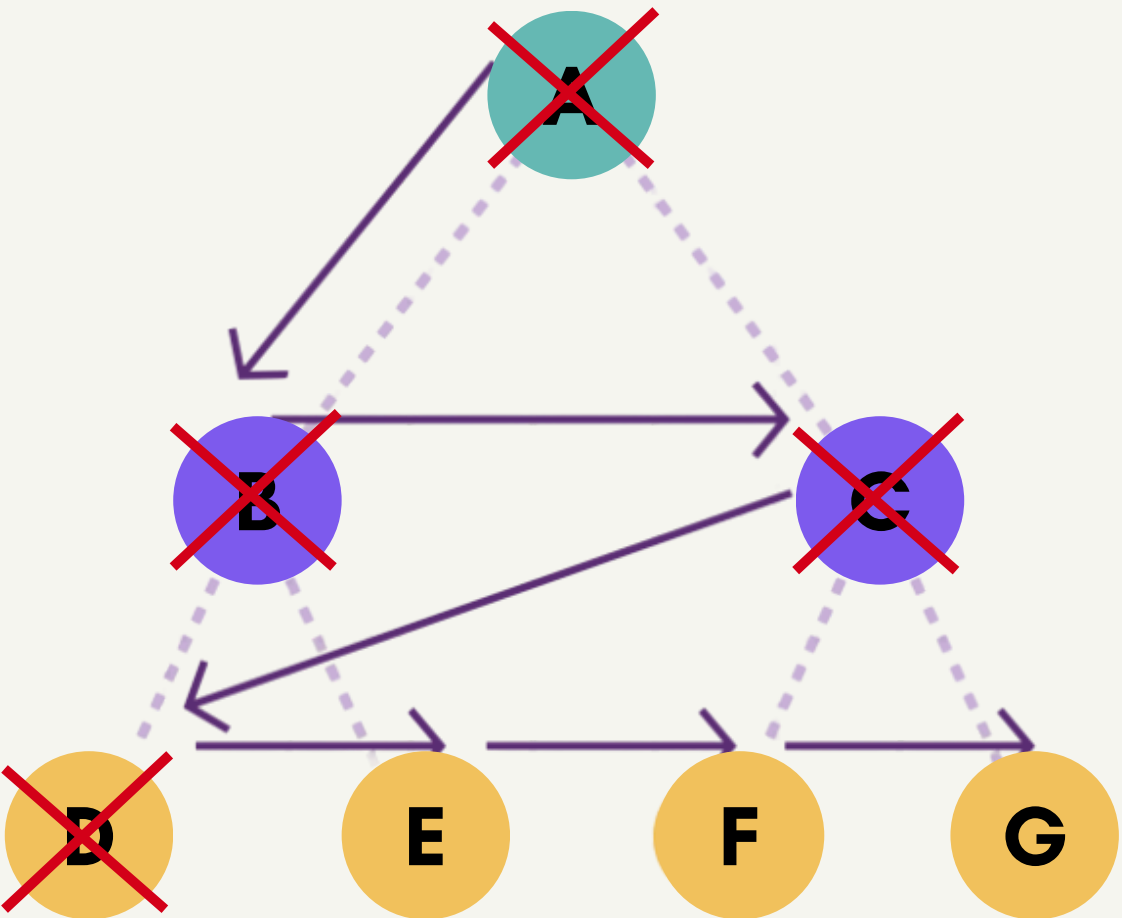
FILA:

G → F → E → D → ~~C~~ → ~~B~~ → ~~A~~

Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



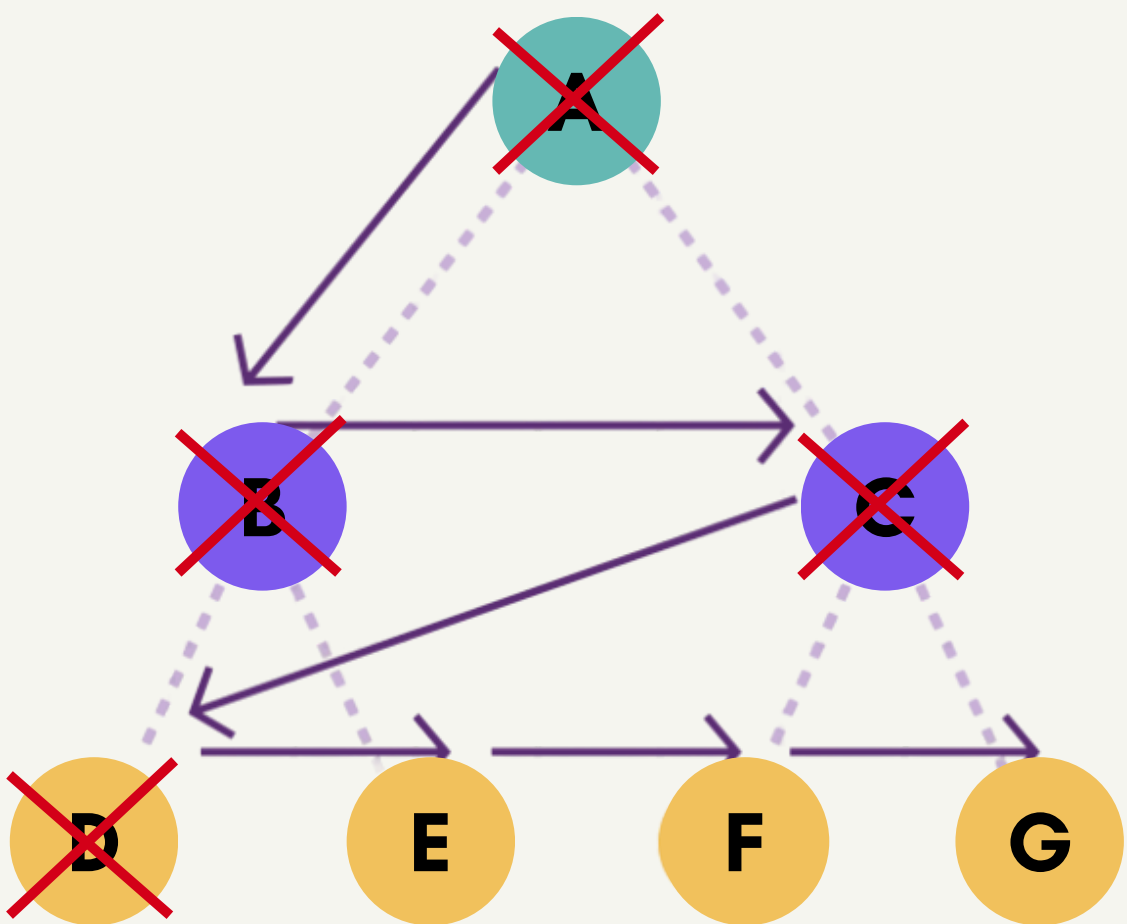
FILA:

G → F → E → ~~D~~ → ~~C~~ → ~~B~~ → ~~A~~

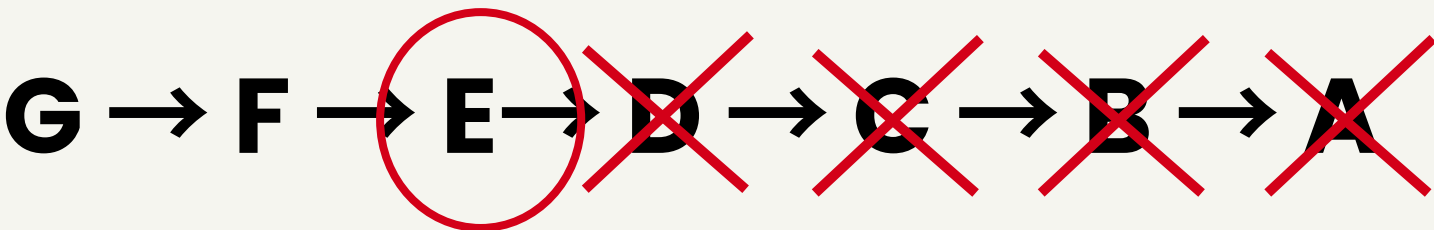
Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



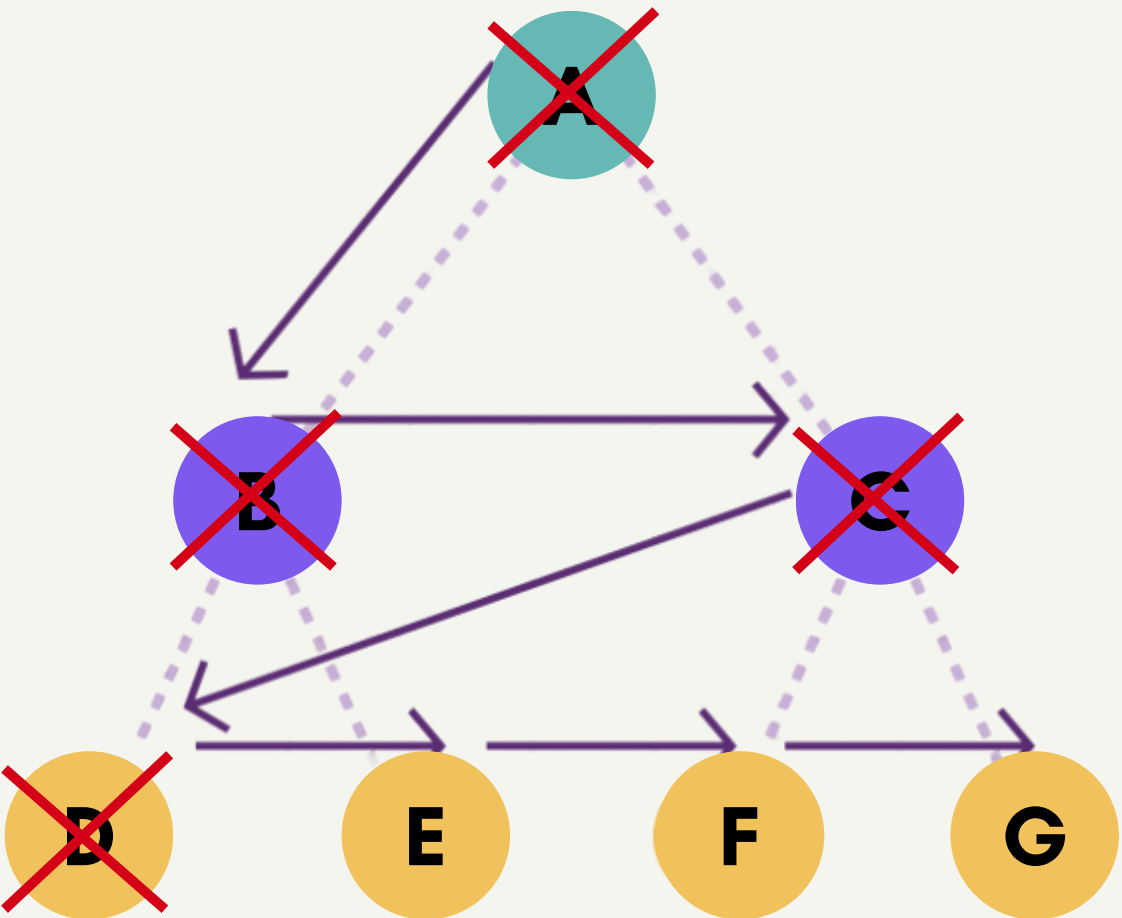
FILA:



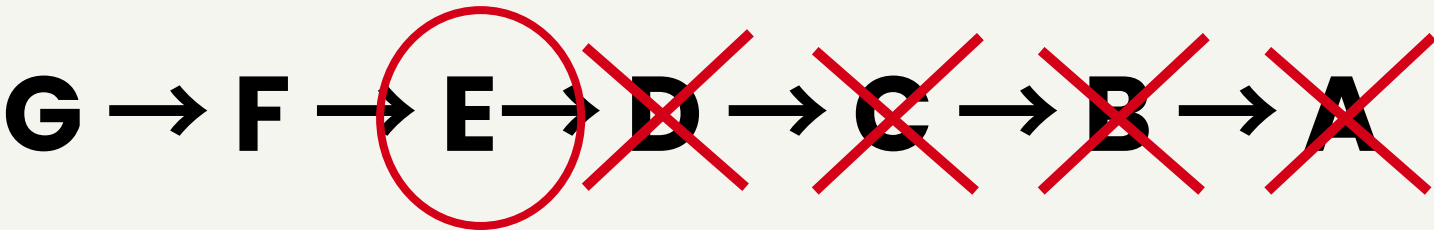
Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E

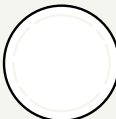


FILA:



Ordem de visita:

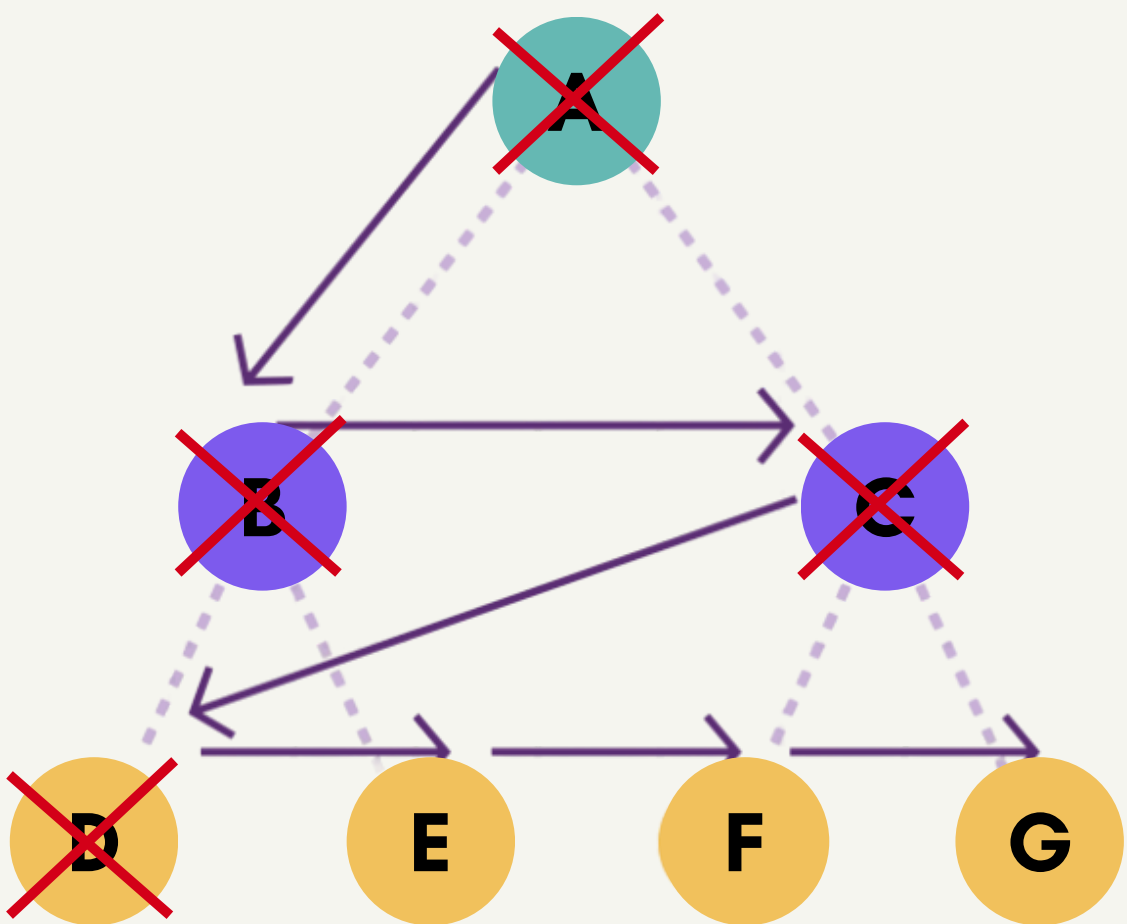
A → B → C → D → E



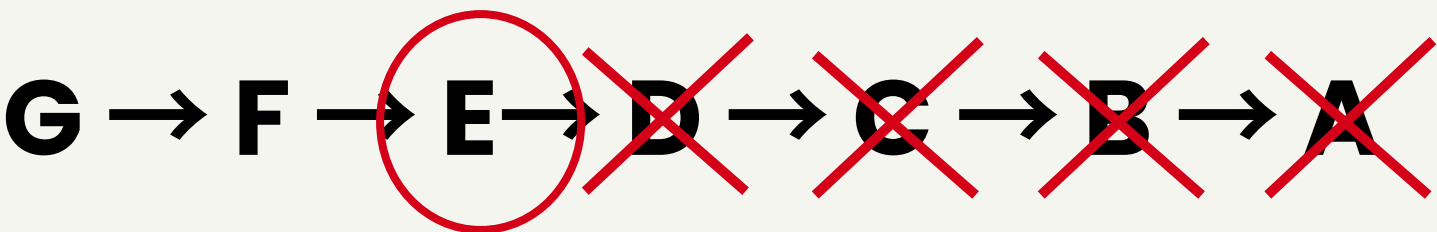
Busca em Largura (BFS)

Estrutura auxiliar: FILA

Vamos buscar o nó E



FILA:



Ordem de visita:

A → B → C → D → E

Percurso encontrado
(predecessores):

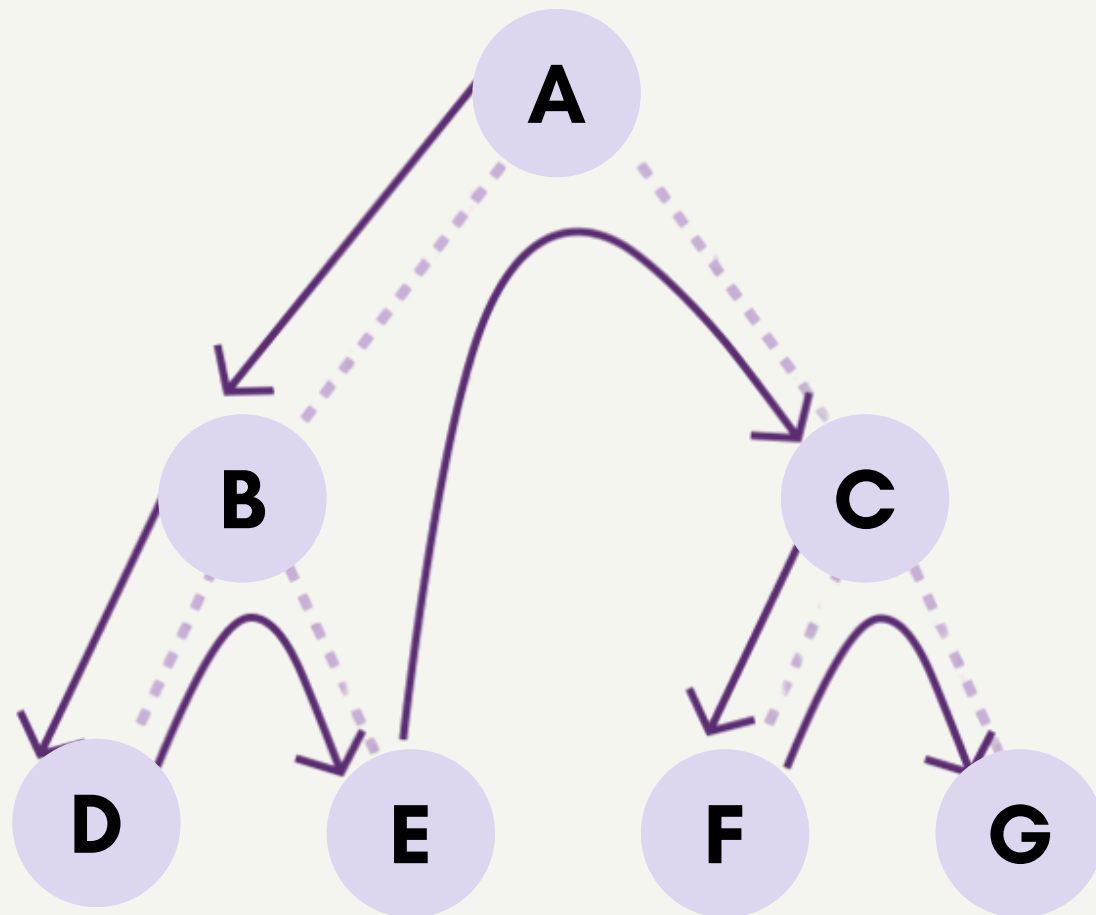
A → B → E

ACHOU O DESTINO?
VOLTA PELOS PREDECESSORES ATÉ A ORIGEM.
DEPOIS INVERTE.

Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

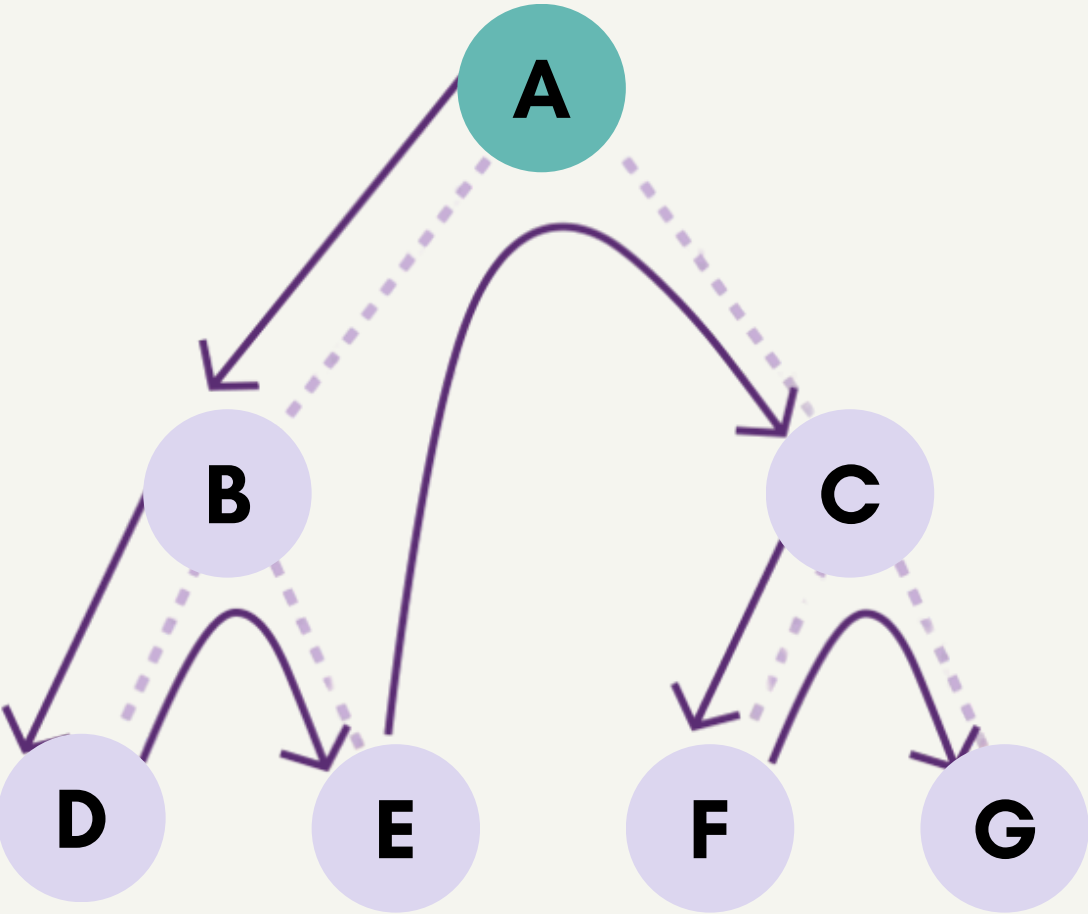
Vamos buscar o nó E



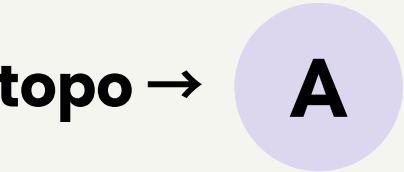
Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



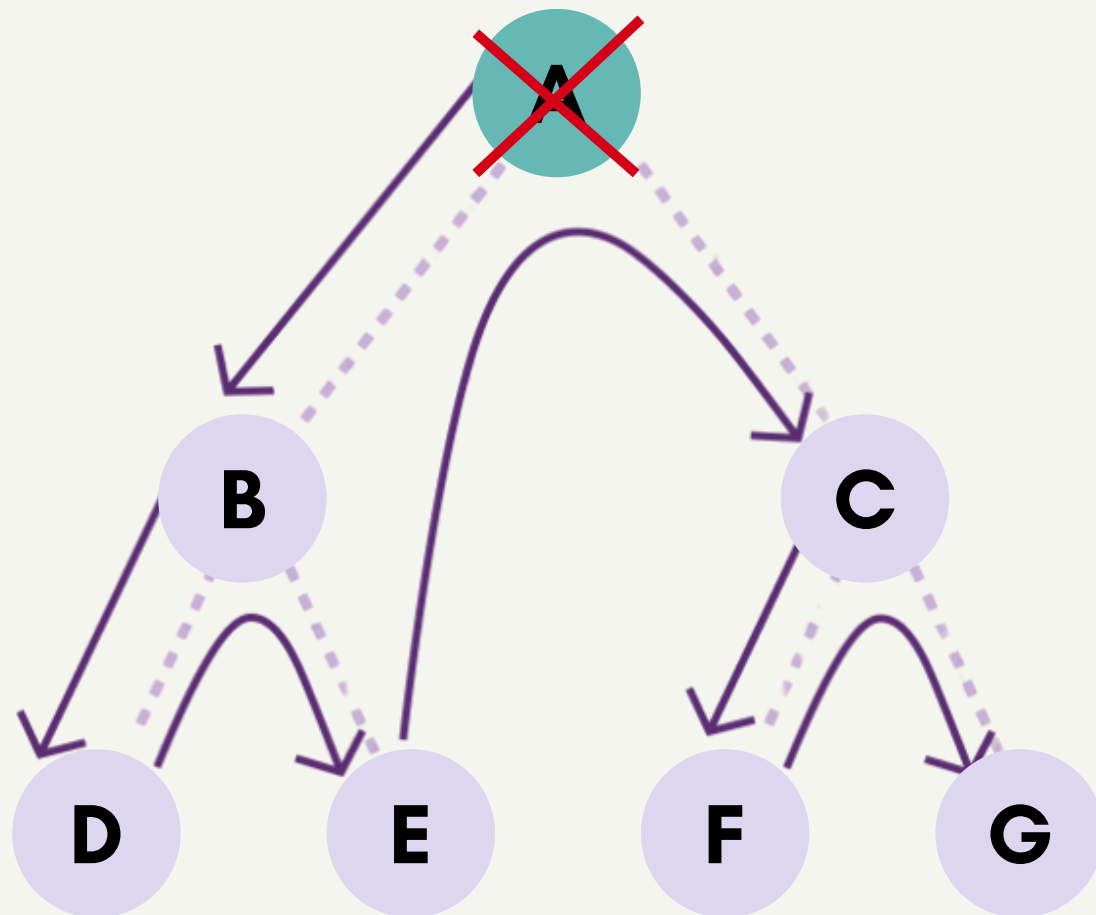
PILHA:



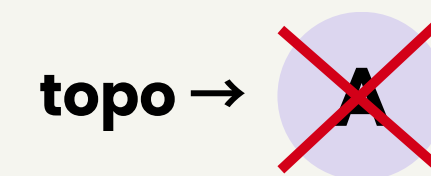
Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



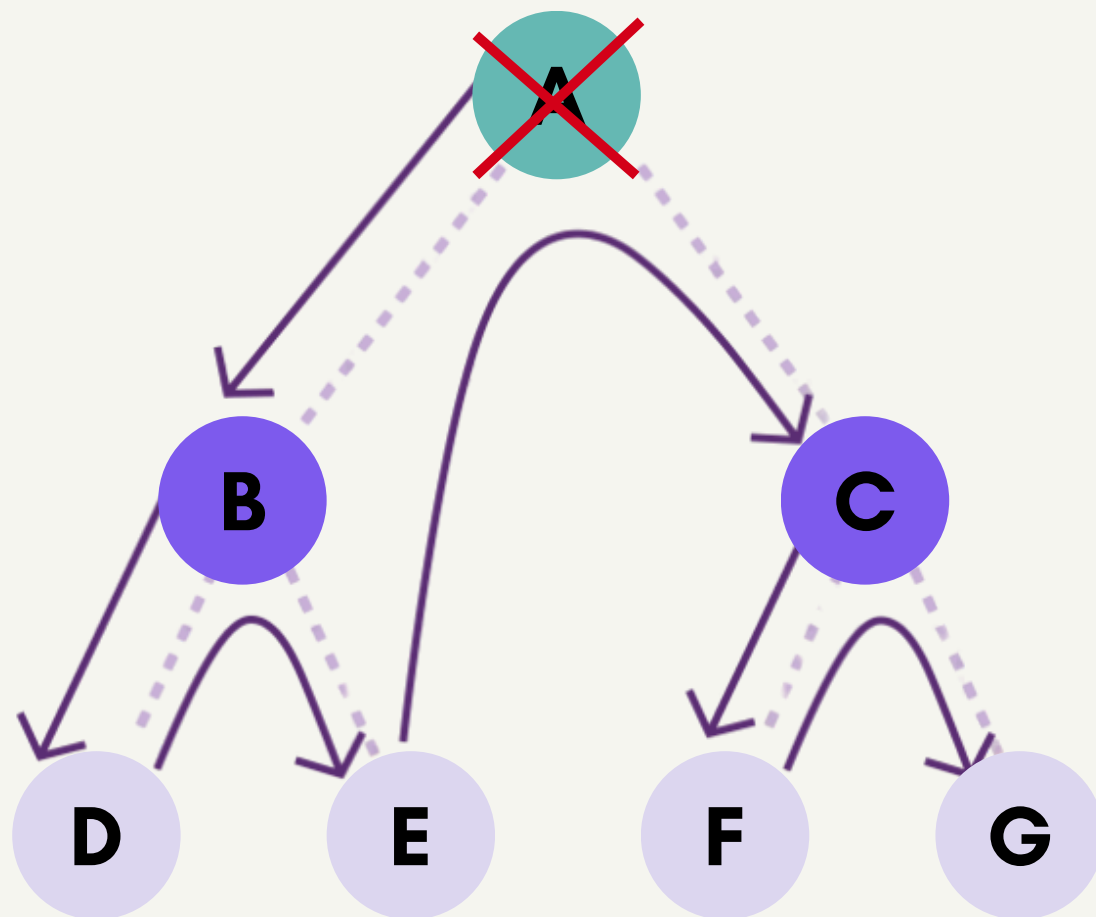
PILHA:



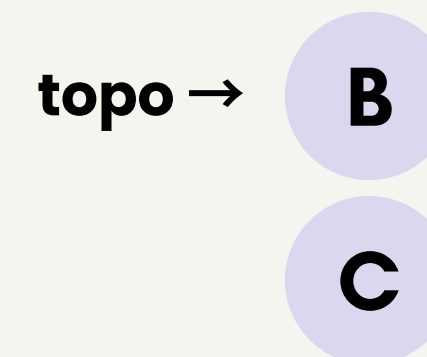
Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



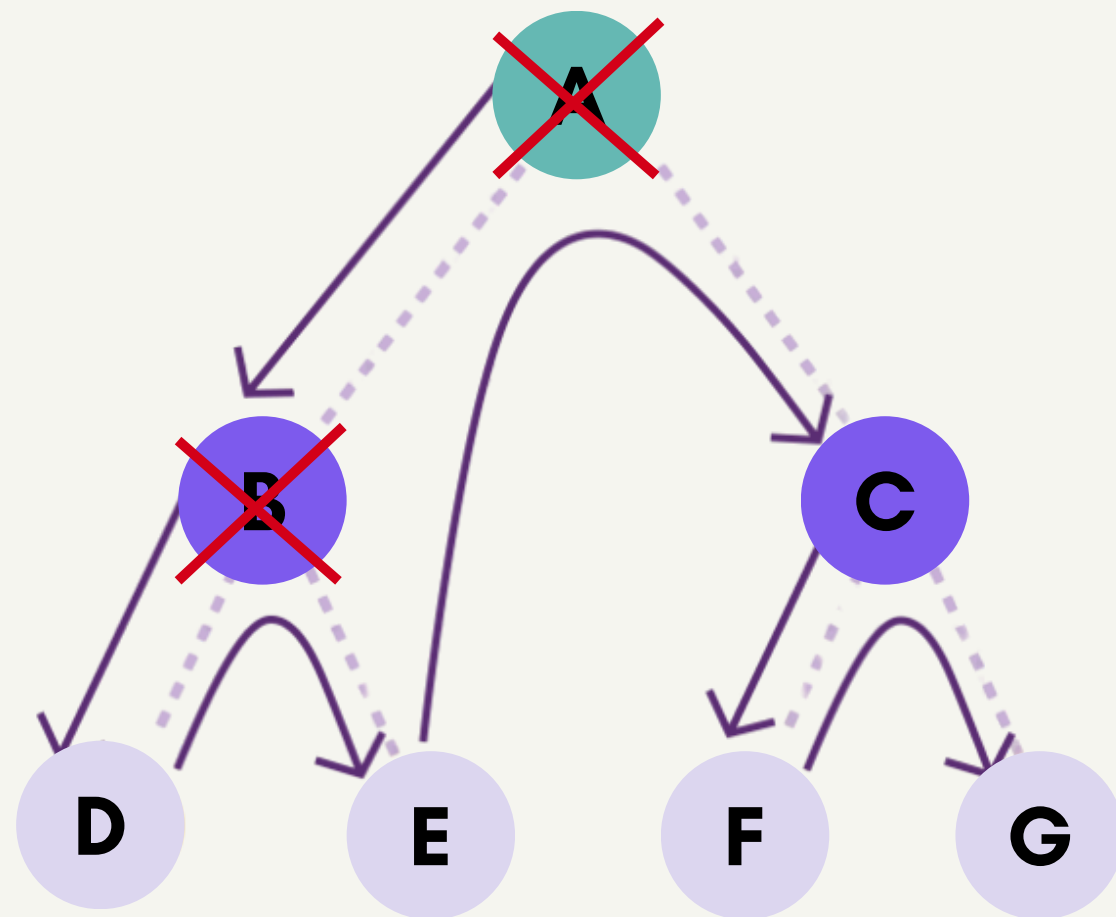
PILHA:



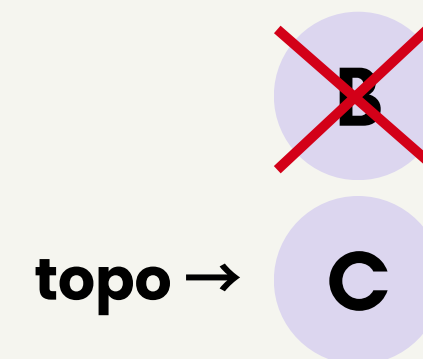
Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



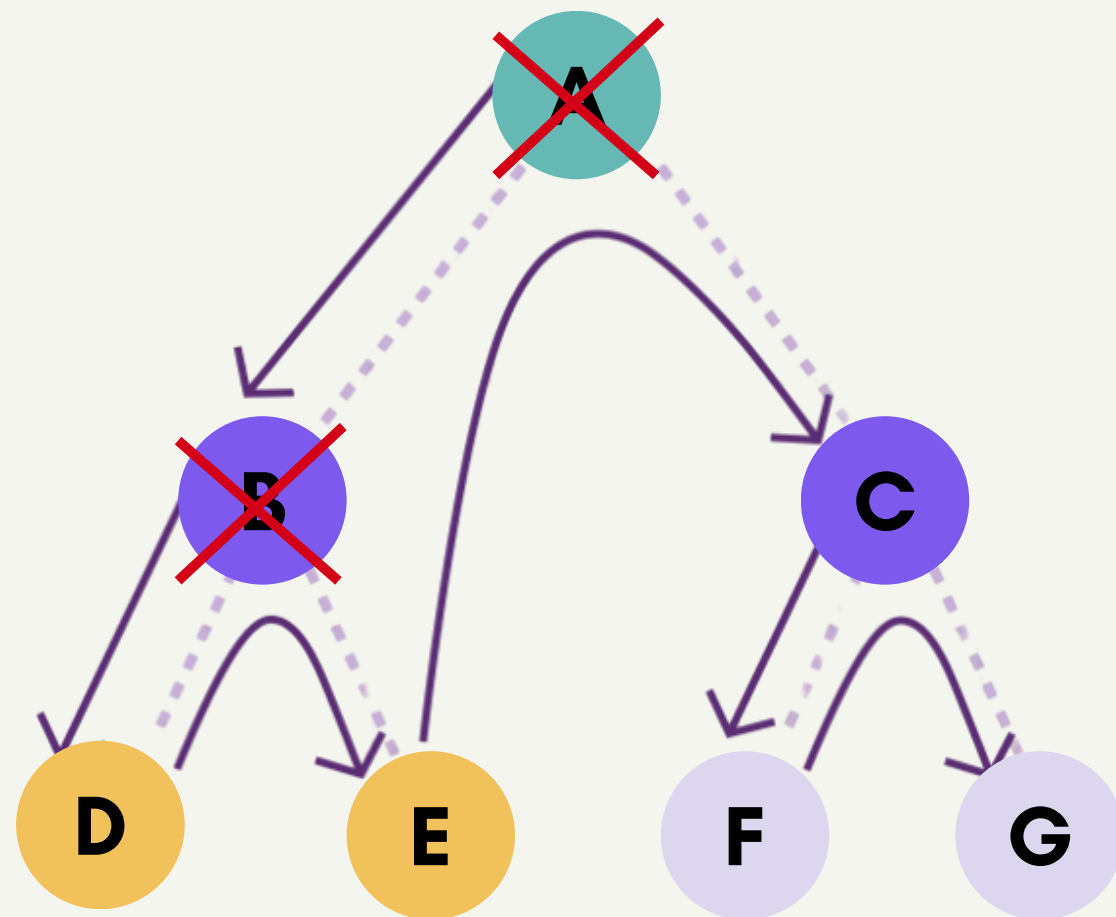
PILHA:



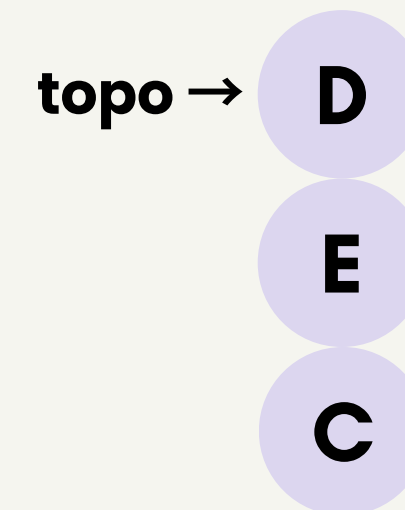
Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



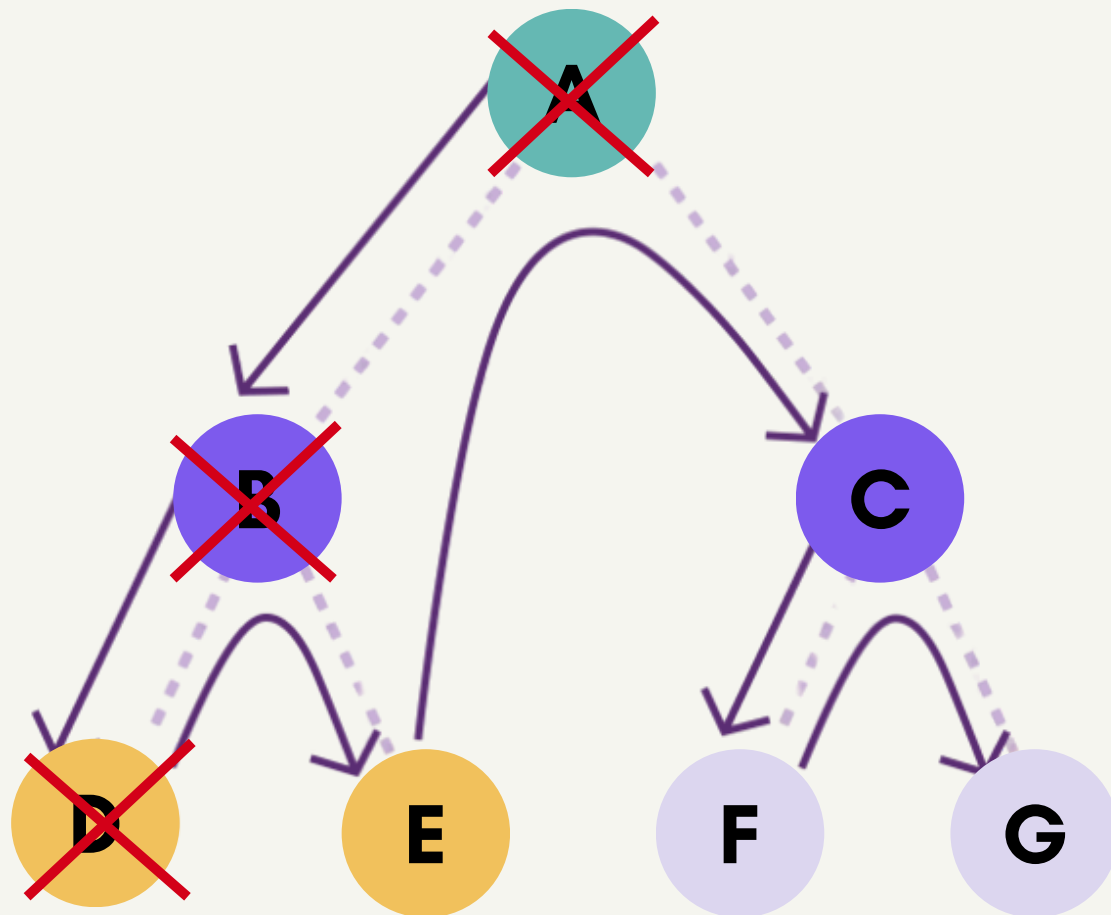
PILHA:



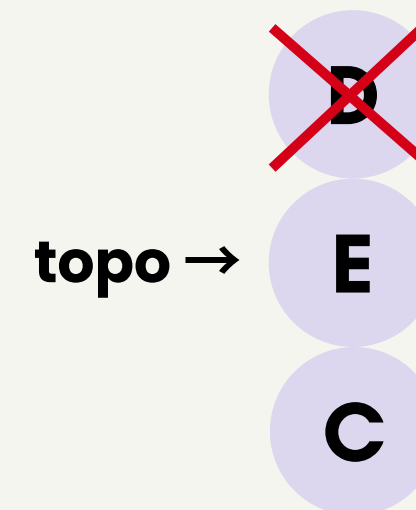
Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



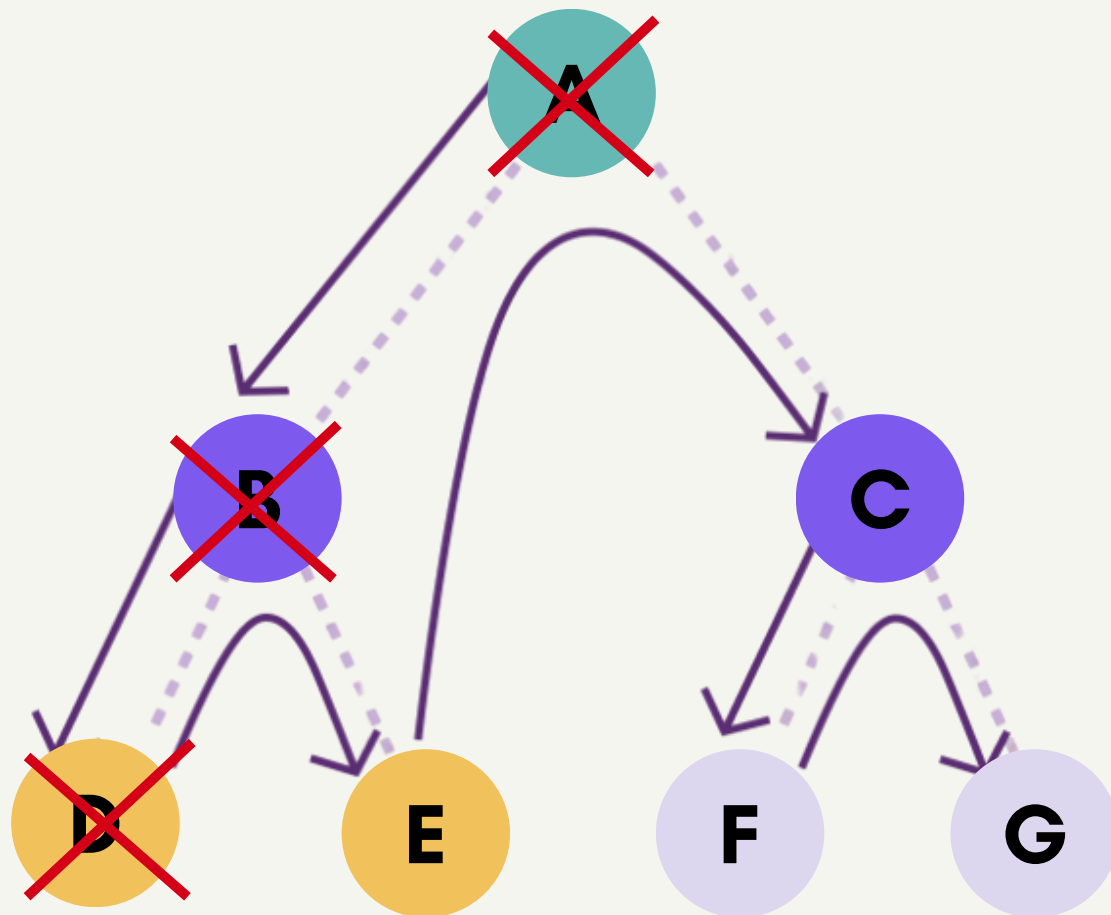
PILHA:



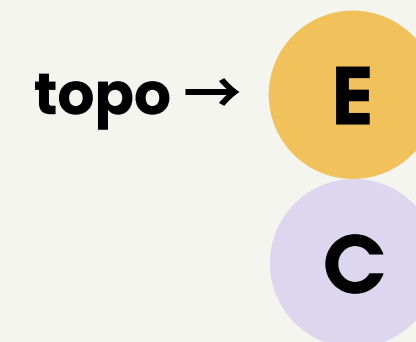
Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



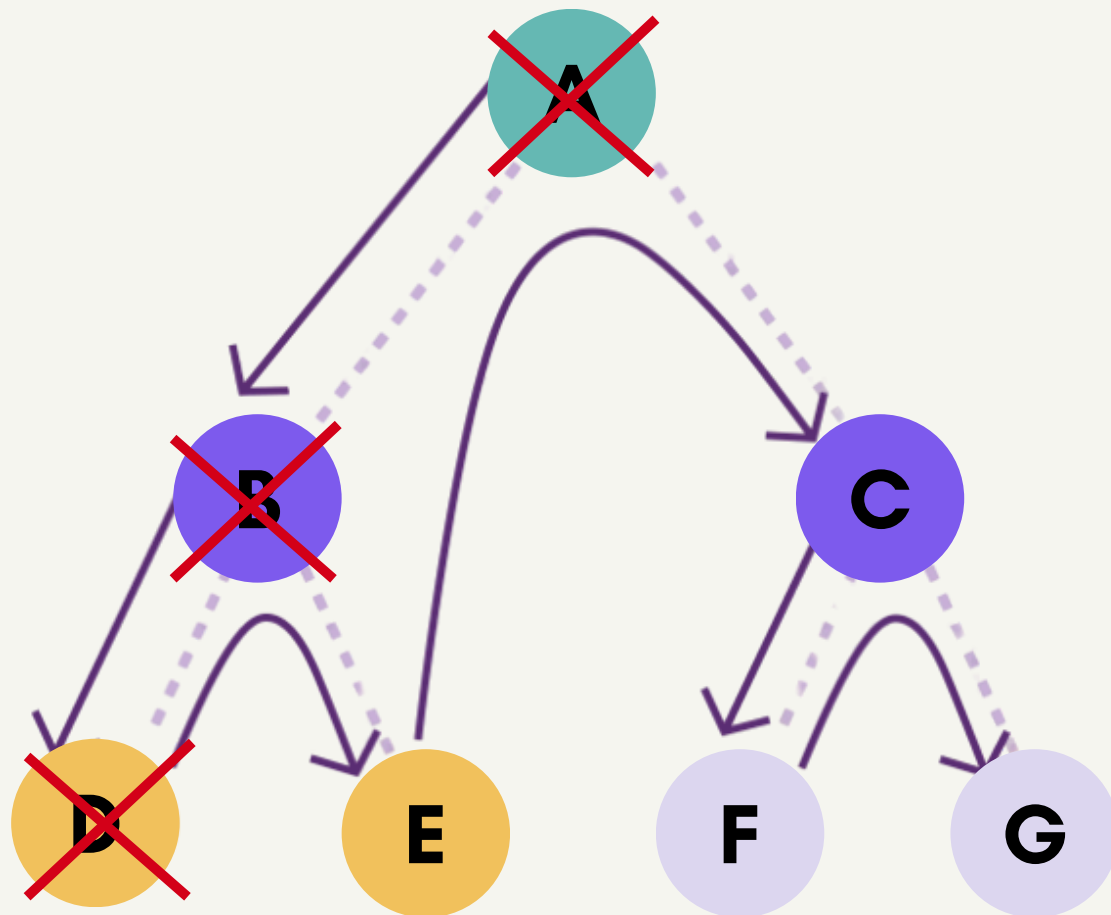
PILHA:



Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



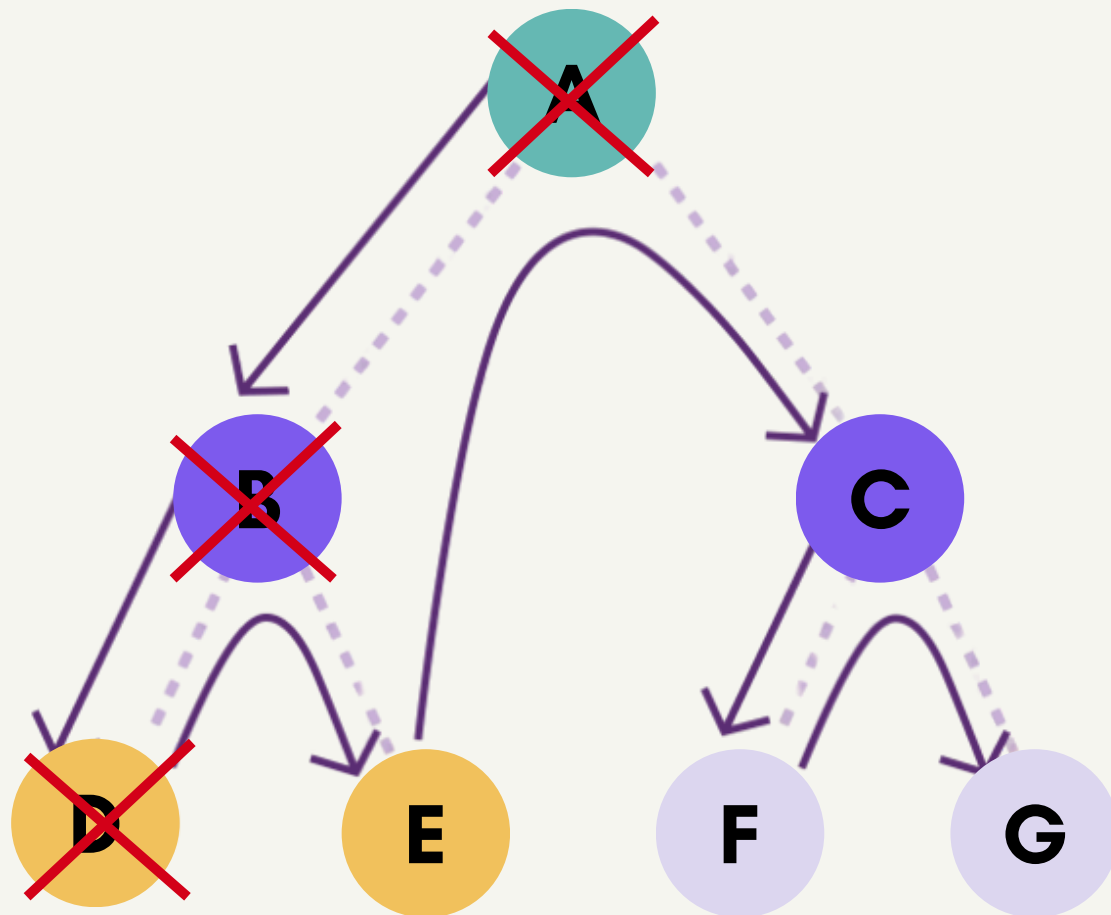
Ordem de visita:

$A \rightarrow B \rightarrow D \rightarrow E$

Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Vamos buscar o nó E



Ordem de visita:

$A \rightarrow B \rightarrow D \rightarrow E$

Percurso encontrado
(predecessores):

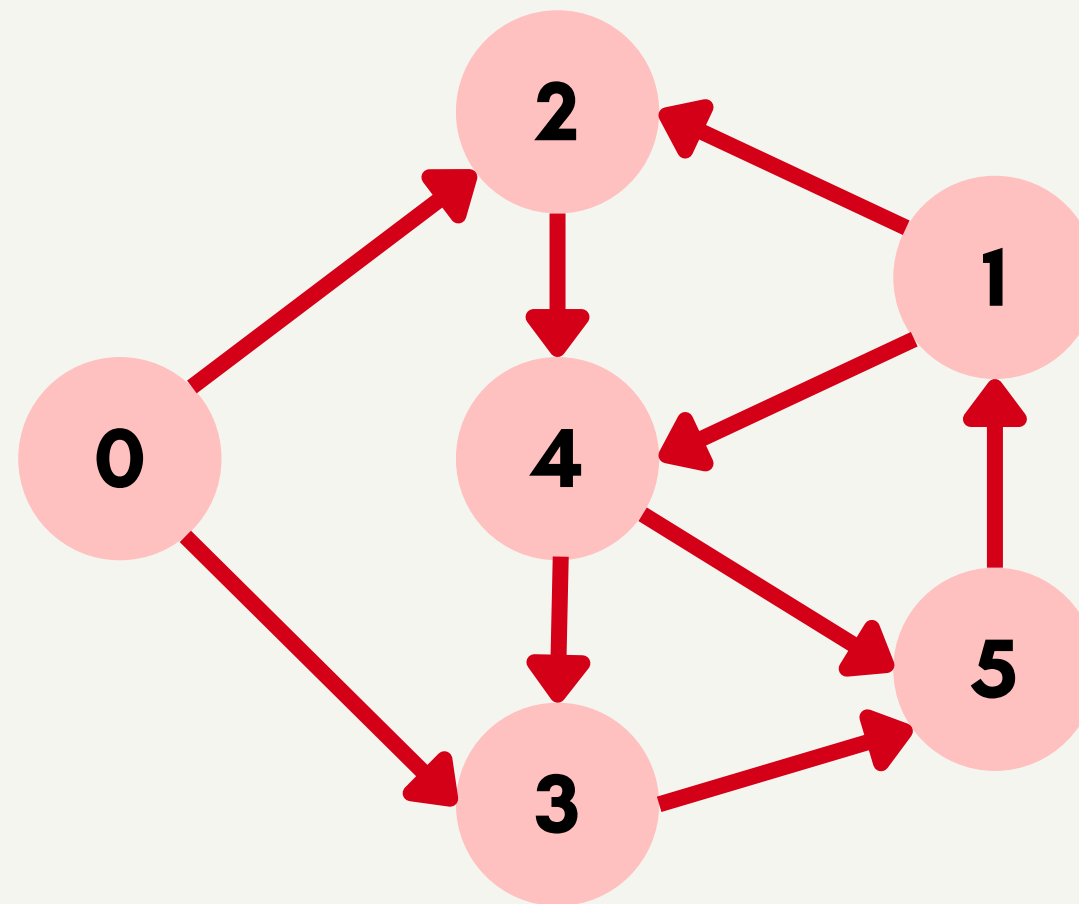
$A \rightarrow B \rightarrow E$

ACHOU O DESTINO?
VOLTA PELOS PREDECESSORES ATÉ A ORIGEM.
DEPOIS INVERTE.

Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Preciso verificar se existe caminho do 0 ao 4

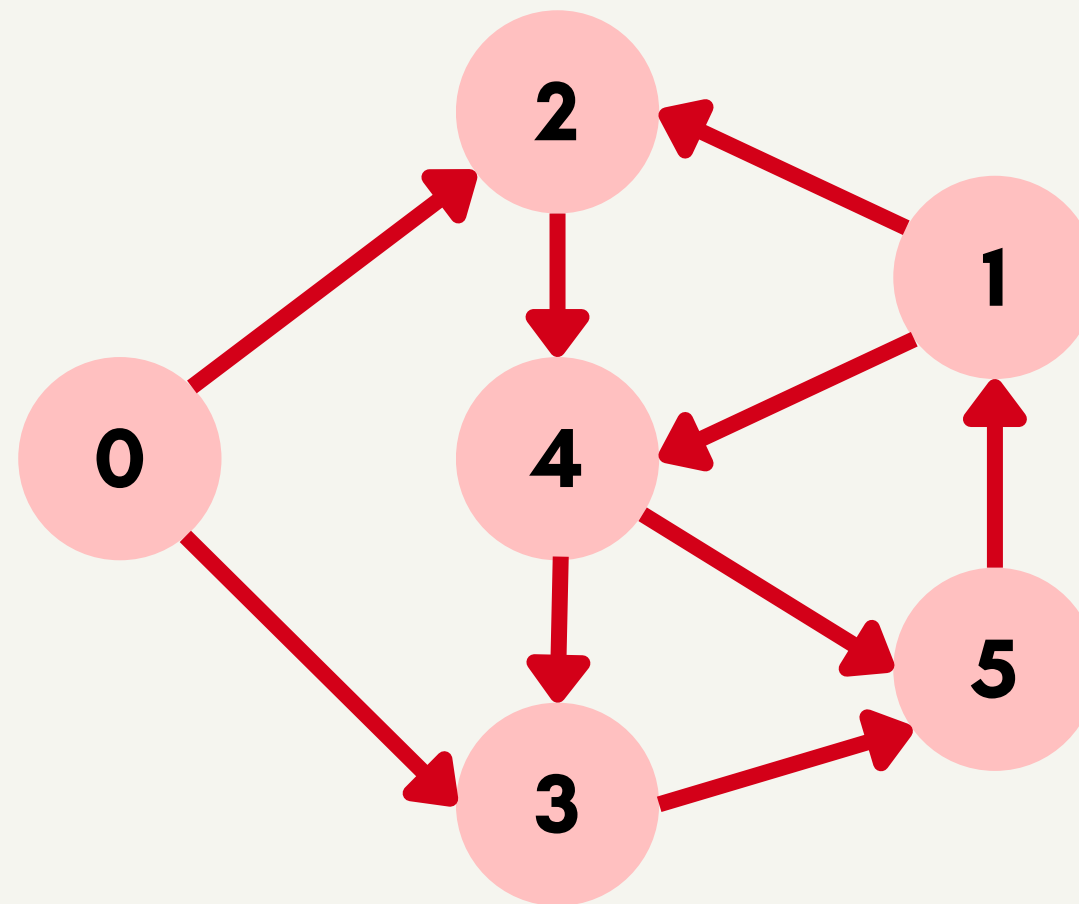


Início: 0
Fim: 4

Busca em Profundidade (DFS)

Estrutura auxiliar: PILHA

Preciso verificar se existe caminho do 0 ao 4



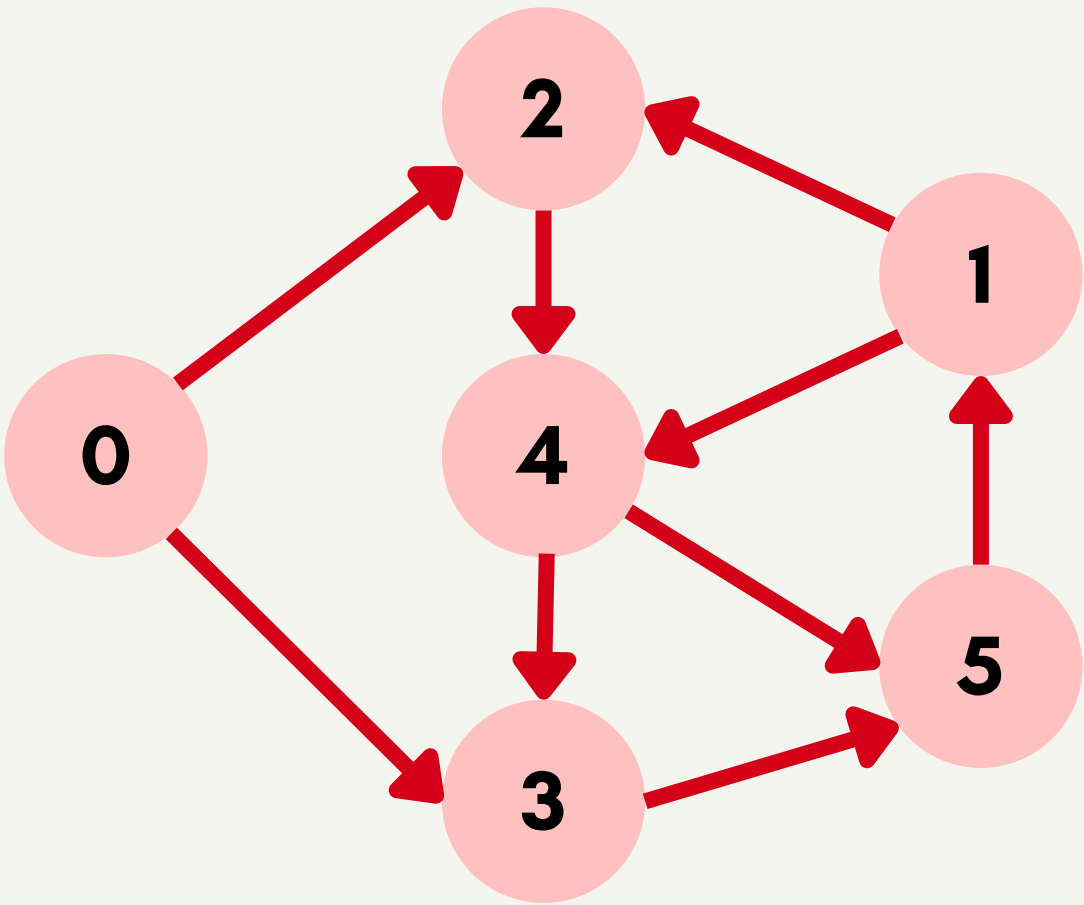
Início: 0
Fim: 4

Na **DFS**, nem sempre você acha o **menor** caminho (menor quantidade de arestas!!!)

Busca em Largura (BFS)

Estrutura auxiliar: FILA

Preciso verificar se existe caminho do 0 ao 4

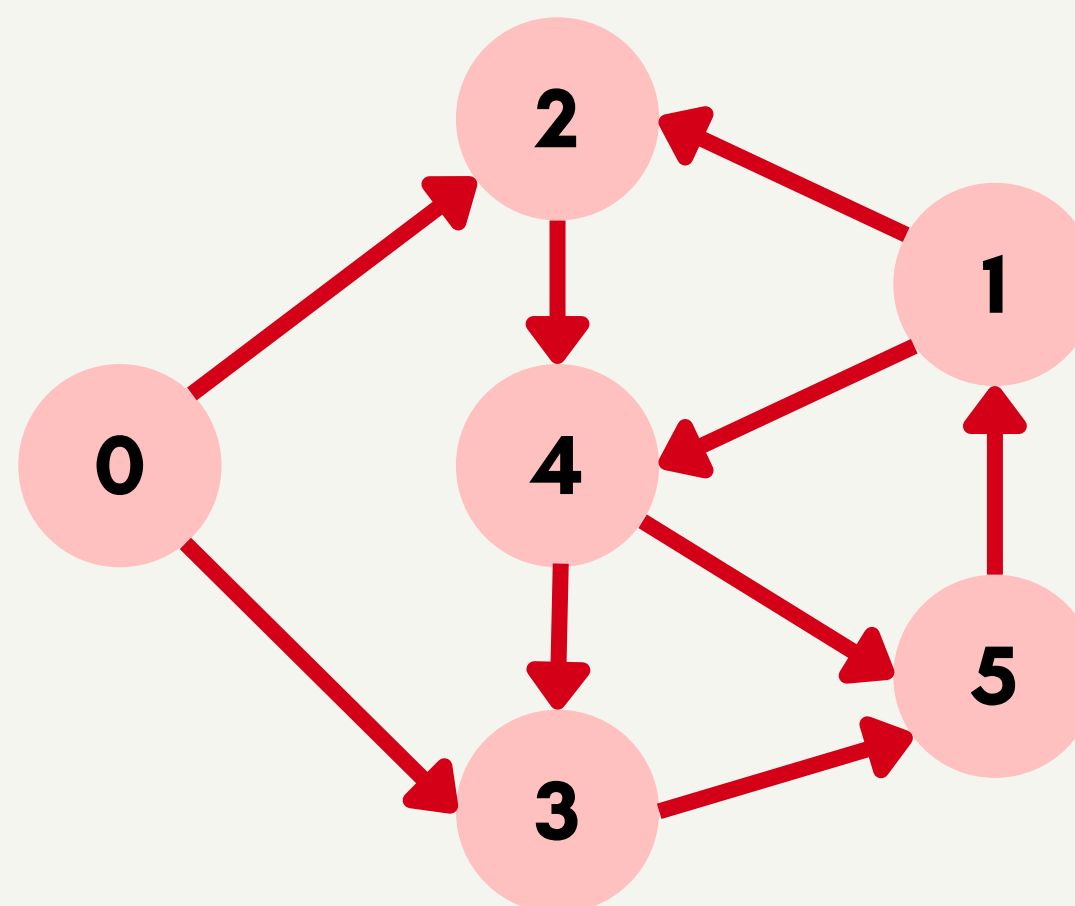


Início: 0
Fim: 4

Busca em Largura (BFS)

Estrutura auxiliar: FILA

Preciso verificar se existe caminho do 0 ao 4



Início: 0
Fim: 4

Na **BFS**, você acha o **melhor caminho** (menor quantidade de arestas!!!)

Observações:

GRAFOS NÃO PONDERADOS sem pesos nas arestas

Busca em LARGURA (BFS): fornece o **menor caminho** (em número de arestas)

Busca em PROFUNDIDADE (DFS): fornece apenas um caminho **possível**.

Exercício:

Altere o algoritmo de Grafos utilizando matriz de adjacências desenvolvido para implementar um grafo DIRECIONADO